DRAFT

# Semantic Governance

## Services, Architecture & Integration

**Version 2.1** · Draft · October 26, 2026


**PART I** **Semantic Governance Services**
Why semantic governance matters and how it governs agentic AI; the operating model, inbound services, deterministic knowledge, demand management, the neuro-symbolic foundation, and context engineering.

**PART II** **Semantic Governance Architecture & Integration**
The MVP1 solution architecture (Federated RAG 2.0), industry alignment, process flows, features, vector database design, the reusable inbound microservices, agents, and search facade, and the processing and outbound services, orchestration, and AI circuit breaker gateway.

**PART III** **Knowledge Engineering**
Design principles, naming and URI standards, change control, governance against ISO and IEEE standards, the single source of truth for rules and logic, the target operating model, the term definition standard, labels and personas, the taxonomies of relations and attributes, BFO and RDF foundations, semantic asset metadata, services and processes, and personas, jobs, and demand.

**APPENDIX A** **Finance (FP&A) Domain**
An illustrative, end-to-end example of the framework applied to Financial Planning & Analysis: domain model and ontology, requirements and change management, LKIF mappings, vector databases, sequence diagrams, use cases, and MCP integration.

**APPENDIX B** **Conversational AI**
A design and reference architecture for conversational AI that lets subject-matter experts talk to their data: components, layers, phased delivery plan, KPIs, and the context engineering module.

**APPENDIX C** **Simulation and Causal Reasoning**
Emerging techniques that extend context engineering: simulation-driven context with the Simulator, Reflector, and Curator pattern, and causal reasoning over ontologies with CausalGraph2LLM.

**APPENDIX D** **Statement of Work (MVP1 Prep)**
The Statement of Work for the 12-week build of semantic services supporting the HR 360 agent: scope, SKL footprint, services, measurement, traceability, service level improvement, deliverables, and acceptance criteria.

**APPENDIX E** **Applying Semantic Standards in the KE Workbench**
A stepwise guide to applying the Part III standards with the KE Workbench, with checklists, known behaviors, and the coverage of every standard by a workbench page.

**APPENDIX F** **Terms & Acronyms**
Definitions of the acronyms and specialist terms used throughout this document.

# Contents


**PART I Semantic Governance Services**

1.1 Overview
1.2 Value and Agentic Governance
1.3 Semantic Governance Operating Model
1.4 Inbound Semantic Services Design
1.5 Deterministic Knowledge in the Knowledge Fabric
1.6 Demand Management and Agility
1.7 Neuro-Symbolic Foundation for AI
1.8 Context Engineering

**PART II Semantic Governance Architecture & Integration**

2.1 Solution Architecture
2.2 Industry Alignment
2.3 Primary Process Flows
2.4 Feature and Functional Breakdown
2.5 Vector Database Design
2.6 Inbound Microservices, Agents, and Search
2.7 Processing and Outbound Services, and Orchestration

**PART III Knowledge Engineering**

3.1 Design
3.2 Standards
3.3 Change Control
3.4 Governance
3.5 Single Source of Truth
3.6 Target Operating Model
3.7 Term Definition Standard
3.8 Labels and Personas
3.9 Attributes, Properties and Relations
3.10 BFO and RDF Foundations
3.11 Semantic Asset Metadata
3.12 Services, Processes, and the Reference Registry
3.13 Personas, Jobs, and Demand

**APPENDIX A Finance (FP&A) Domain**

A.1 Finance Domain
A.2 FP&A Ontology
A.3 FP&A Ontology: Process-Centric Model
A.4 Ontology Requirements Specification (ORSD)
A.5 KGCL-to-ORSD Mapping
A.6 KGCL-to-ORSD Mapping (Detailed)
A.7 Mapping FP&A Metrics to LKIF Statements
A.8 Mapping FP&A Classes to LKIF Statements
A.9 Scenario Mapping in Vector Databases
A.10 Vector Database Integration with Ontologies
A.11 FP&A Sequence Diagrams

A.12 FP&A Use Cases
A.13 MCP and Microservices Integration

**APPENDIX B Conversational AI**

B.1 Conversational AI Design
B.2 Conversational AI Architecture

**APPENDIX C Simulation and Causal Reasoning**

C.1 Simulation for Context Engineering
C.2 Causal Reasoning with CausalGraph2LLM

**APPENDIX D Statement of Work (MVP1 Prep)**

D.1 Statement of Work (MVP1 Prep)

**APPENDIX E Applying Semantic Standards in the KE Workbench**

E.1 Purpose, Roles, and Setup
E.2 Stepwise Guide
E.3 Checklists and Tool Behaviors
E.4 Workbench Coverage of the Standards

**APPENDIX F Terms & Acronyms**

F.1 Terms & Acronyms

# Document Control

| Item | Value |
|---|---|
| Title | Semantic Governance: Services, Architecture & Integration |
| Version | 2.1 |
| Status | Draft |
| Date | October 26, 2026 |
| Versioning | MAJOR.MINOR. A minor version adds or revises content within the existing structure; a major version changes the structure or scope, or marks formal approval. |

## Revision History

| Version | Date | Status | Changes |
|---|---|---|---|
| 1.0 | October 2026 | Draft | Combined reference: Part I Services, Part II Architecture & Integration, Part III Knowledge Engineering (Sections 3.1–3.6), Appendices A–E; review feedback applied. |
| 1.1 | October 13, 2026 | Draft | Added versioning and document control. Part III: added Section 3.7 Term Definition Standard, 3.8 Labels and Personas, 3.9 Attributes, Properties and Relations, 3.10 BFO and RDF Foundations. Appendix E: added IAO, Persona, RDF 1.2, RO, SKOS-XL. |
| 1.2 | October 13, 2026 | Draft | Section 3.3 Change Control: added the change log model (change records in a dedicated append-only graph, added and removed triples per change set, reporting and lineage), the rule for versions in identifiers, and the release pipeline (build once, promote one package through development, test, stage, and production, deployment records, safe loading and rollback, drift prevention). Section 3.6: traceability chain extended to releases and deployments. |
| 1.3 | October 13, 2026 | Draft | Added Section 2.6 Inbound Microservices, Agents, and Search: Kafka-based microservice catalog for the Section 1.4 inbound flow, naming conventions, standard service contract and logging, agent catalog and guardrails, exception handling and resiliency patterns, and search as a Semantic Knowledge Layer utility with a facade of microservices and federated-query agents. Cross-reference added to Section 1.4; Appendix E updated. |
| 1.4 | October 13, 2026 | Draft | Section 1.4: replaced the step-by-step inbound flow listing with a six-stage diagram showing steps, microservices, the Kafka event backbone, and the agents. |
| 1.5 | October 13, 2026 | Draft | Section 2.1: replaced the semantic governance and homogeneity diagram with a Keel and North Star diagram, establishing it as the foundational architecture, and added a table mapping the section's layers to the vessel layers. |
| 1.6 | October 13, 2026 | Draft | Section 1.5: added Developing the Knowledge Fabric with an Agentic Fabric (component design, federated star topology, development agents, development loop, and diagram) and Alignment with Keel and North Star. Added Appendix F, Applying Semantic Standards Using RDF Studio, a stepwise guide. Appendix E updated. |

| Version | Date | Status | Changes |
|---|---|---|---|
| 1.7 | October 13, 2026 | Draft | Moved Terms & Acronyms to the last appendix (now Appendix F); Applying Semantic Standards Using RDF Studio is now Appendix E. Cross-references updated; entries above refer to the appendix letters in use at the time. |
| 1.8 | October 13, 2026 | Draft | Appendix D, Section D.1: replaced the 2025 Marketplace Statement of Work with a 12-week Statement of Work for semantic services supporting the HR 360 agent rollout (Employee Benefits & Retirement Sponsorship): scope by domain, SKL footprint (core, domains, personas, SSOT), semantic services, monitoring and measurement, traceability demonstration, before-and-after service levels, and Strategic Outcome metrics, with roadmap and architecture diagrams. Appendix D summary, Sections 2.4 and 3.4 cross-references, and Appendix F (CQ, HR 360, MTBH, SKL) updated. |
| 1.9 | October 15, 2026 | Draft | Part III: added Section 3.11 Semantic Asset Metadata, the metadata field set for terms, vocabularies, taxonomies, and ontologies used by the Semantic Modeling Workbench forms and the metadata registry, with each property tied to ISO and IEEE standards; a terms cross-reference for each artifact; identifier, version IRI, and named graph patterns; taxonomy concepts that draw meaning from several vocabulary terms (polymorphism), with the new core properties core:hasPrimaryTerm, core:formalizedBy, and core:hasTermType; and controlled values for lifecycle status and term type, mapped to BIBO, ISO/IEC 11179-6, and IEEE LOM. Section 3.2: added the named graph URI pattern. Sections 1.1, 3.7, 3.9, and E.3: cross-references updated. Appendix F: added BIBO, BT / NT / RT, DCMI, IEEE, LOM, Named graph, PT / NPT, Polyhierarchy, Polymorphism, RA, SN, TBX, TLO, TT, VAEM, and VANN. |
| 2.0 | October 24, 2026 | Draft | MAJOR edition: identifier, URI, and named graph rules change. Section 3.2: one identifier rule for classes, concepts, and terms (lower-case primitives joined by hyphens), asset titles, URI patterns for core facets, operational areas, and nested domains, named graph = asset IRI, and instance identifiers (proposed). Section 3.1: BFO-based core on the asset genus; operational areas, containers, insulation, and the enterprise service catalog replace the operational module. Section 3.3: the PROV-O change record profile with required governance extensions, versioning before 1.0.0, Candidate entry criteria (proposed), current load practice. Section 3.4: implemented quality gates and process policies P01–P09. Section 3.6: Operations Dashboard and measured agent assurance (proposed). Section 3.7: primitives and polymorphic terms. Sections 3.8 and 3.10: the persona model; reference ontology stack and load order. Section 3.11: named graphs and edition conformance. New Section 3.12 Services, Processes, and the Reference Registry and Section 3.13 Personas, Jobs, and Demand. Section 1.3: the KE Workbench and the operating structure. Appendix E rewritten for the KE Workbench, with E.4 Workbench Coverage of the Standards. Appendices A and D and all examples re-keyed to the v2.0 identifiers. Appendix F: 25 entries added. |
| 2.1 | October 26, 2026 | Draft | Added Section 2.7 Processing and Outbound Services, and Orchestration: processing microservices for mapping, inferencing and learning; outbound microservices for search, contextual relevancy (prompts reverse engineered and mapped to domain holons and to persona, job, job step and desired outcome instances), conversational AI, publishing and APIs; topic |

| Version | Date | Status | Changes |
|---|---|---|---|
| | | | namespaces for processing, outbound, release, feedback, decision and subscription events; one orchestrating agent per tier (the Section 2.6 orchestration-agent is now inbound-orchestration-agent); and decision evaluation by the AI circuit breaker gateway for every agent and human decision, with ground truth the released knowledge layer plus the measured data of the sources. Modeling is independent of any one AI tool. Section 3.6: answer traceability target set to 100% in the service level table. Appendix E.4: coverage updated; retrofit item G20 resolved by Section 2.7. Appendix F: new terms. |

# Overview

This reference brings together the Company's semantic governance work. Semantic governance harmonizes meanings, ontologies, and taxonomies across the enterprise so that people, analytics, and AI share one machine-readable business language, turning knowledge into a governed asset that grounds AI and makes cause and effect traceable.

## Architecture at a glance (detail in Section 2.1)

- **Knowledge fabric:** core and domain ontologies authored in the Semantic Modeling Workbench (implemented as the KE Workbench, Section 1.3) and stored in GraphDB.
- **Operations Workbench:** a low-code layer for monitoring, visualization, impact analysis, and human-in-the-loop review.
- **Intelligence layer:** domain-aligned vector databases for semantic search and AI agents that map, enrich, and reason.
- **Integration backbone:** microservices and the Model Context Protocol (MCP) for secure, standard communication.
- **Governance for AI:** policies expressed in the semantic layer, with SHACL validation, registries, and monitoring.

## How this document is organized

### Part I — Semantic Governance Services

Sections 1.1–1.8

Value and agentic governance, the operating model, inbound services, deterministic knowledge, demand management, the neuro-symbolic foundation, and context engineering.

### Part II — Semantic Governance Architecture & Integration

Sections 2.1–2.7

The MVP1 solution architecture (Federated RAG 2.0), industry alignment, process flows, features, and vector database design.

### Part III — Knowledge Engineering

Sections 3.1–3.13

How knowledge is designed, named, changed, governed, and operated: the process-step-centric ontology design, naming and URI standards, change control, governance, the single source of truth for rules and calculations, the target operating model, and the detailed modeling standards for term definitions, labels and personas, relations, and attributes, the metadata recorded for every vocabulary, taxonomy, and ontology, the service and process model, and personas, jobs, and demand.

### Appendices — Applying the Framework

Appendix A is an illustrative, end-to-end example of the framework applied to Financial Planning & Analysis, the pattern other domains would follow. Appendix B covers conversational AI, Appendix C simulation and causal reasoning, and Appendix D the Statement of Work for the MVP1 build.

Appendix E is a stepwise guide to applying the semantic standards with the KE Workbench, with the coverage of every standard by a workbench page, and Appendix F defines terms and acronyms.

# Value and Agentic Governance

A semantic layer gives enterprise AI structured, business-aware knowledge: what terms mean, how concepts relate, which actions are permitted, and which outcomes matter. That knowledge turns AI from a passive analysis tool into an agent that can interpret intent, plan, and act within business rules. Semantic governance keeps the layer accurate and trusted, and increasingly it is also how the agents themselves are governed.

## How a semantic layer enables autonomous AI

- **Contextual reasoning.** Business logic, definitions, and relationships are encoded as ontologies and knowledge graphs, so AI can reason about how concepts connect rather than match patterns.
- **Consistent, trustworthy action.** Standard definitions for terms, metrics, and processes give every agent a single source of meaning and remove errors caused by ambiguous metadata.
- **Cross-system automation.** Entity resolution across sources (a CRM customer is the billing client) lets agents orchestrate approvals, reconciliation, and root-cause analysis across systems.
- **Goal-driven planning.** Models of cause and effect, rules, and hierarchies let agents plan multi-step actions toward business objectives and explain their rationale in business terms.
- **Machine-accessible business logic.** APIs and query languages let agents retrieve context, transform data, and trigger processes in line with formal semantics.
- **Guardrails.** Access rules, compliance constraints, and permitted actions defined at the business-concept level keep autonomy within policy, regulatory, and ethical bounds.

## Where semantic governance fits

Semantic governance does not replace other governance disciplines; it supplies the business meaning that connects them.

| Discipline | What it governs | What semantic governance adds |
|---|---|---|
| Data governance | Quality, access, privacy, and compliance of raw data | Bridges technical fields to business meaning |
| Metrics governance | How KPIs are calculated and displayed | Resolves ambiguity beyond the numbers |
| Analytic governance | Analytics tools and dashboards | Explains how a metric fits the business context |
| AI governance | Ethical, explainable models; bias prevention | Feeds AI trustworthy, meaningful concepts |
| Infrastructure | Pipelines, scale, performance, uptime | Adds meaning and trust, not just capacity |
| Semantic governance | Definitions, metadata, relationships, traceability | Unified, actionable business logic across all of the above |

## Business value and how to measure it

| Value | What it means | How to measure |
|---|---|---|
| Smarter, more reliable AI and analytics | AI and BI get the right words, meanings, and context | Fewer AI/BI errors; faster decisions; higher tool adoption |
| A single business language | Sales, finance, and operations share definitions | Fewer metric disputes; consistent KPIs across reports |
| Standards-based knowledge | Knowledge integrates with new systems and partners | Faster onboarding; lower integration cost |
| One semantic layer | Metrics mean the same thing in every tool | Tools on the shared layer; fewer duplicate definitions |
| Living knowledge | Glossary and graph track how the business operates | Update frequency; speed of adapting to change |
| Explainable, causal answers | Every answer traces to a business term and its causes | Time from question to decision; decision accuracy |

## Alignment with industry direction

- **Analyst guidance.** Gartner research on data and analytics governance stresses active metadata, glossaries, catalogs, lineage, and a semantic layer to reduce AI hallucinations and improve explainability.
- **Standards.** ISO 11179 (metadata registries), ISO 704 (concepts and terminology), and the OMG Ontology Definition Metamodel provide the formal basis for consistent, reusable semantic assets.
- **Practice.** Leading organizations link business glossaries to catalogs and knowledge graphs, and treat semantic layers in BI tools and data fabrics as governed assets.

## Governing agentic AI

As AI agents gain autonomy, governance must move from static rules to machine-readable policy that agents can interpret and that can be checked at run time. The semantic layer supports this in six ways:

- **Contextual policies:** rules, ethical constraints, and compliance requirements expressed in a form agents can apply.
- **Explainability:** decisions linked to the semantic framework produce traceable, human-readable explanations.
- **Lifecycle integration:** a central record of each agent's purpose, data use, and risk profile across build, deploy, and monitor.
- **Risk detection:** semantic reasoning flags behavior that departs from defined acceptable operation.
- **Interoperability:** shared terms and protocol semantics make multi-agent communication secure and compliant.
- **Scalable automation:** compliance checks and escalation run without constant human intervention.

## Governance requirements for autonomous agents

| Requirement | What it involves |
|---|---|
| Accountability and traceability | Clear ownership of AI-driven decisions; immutable audit trails with unique agent and human identifiers |
| Scope and boundaries | Explicit limits on data access, tools, and decision authority; escalation of ambiguous or high-risk cases |
| Transparency | Intermediate steps, assumptions, and reasoning disclosed in a form suited to each stakeholder |
| Identity-centric security | Verifiable agent identities with zero-trust, least-privilege access that adapts to behavior |
| Continuous monitoring | Behavioral thresholds, anomaly detection, and the ability to shut down or constrain an agent |
| Risk-based frameworks | Privacy, safety, and fairness controls extended for agents, aligned with ISO/IEC 42001 and the NIST AI RMF |
| Continuous governance | Regular evaluation and red-teaming, with feedback loops that update policy |
| Cross-functional coordination | Legal, technical, security, and compliance teams working from shared definitions |

The risk is not hypothetical. Agents already coordinate through open protocols such as MCP and Agent-to-Agent (A2A), switch between protocols, and improve themselves. If monitoring and controls do not keep pace, inter-agent communication could bypass human-imposed guardrails, and analysts expect agents to run entire business functions with minimal oversight by around 2030. Observability, auditability, and dynamic security across agent communication are therefore core requirements, not add-ons.

## Agent registries

Registries are the authoritative catalog of AI agents and the backbone of agent governance. They:

- **Identify every agent** as a verifiable, identity-bound entity with its capabilities, risk level, and permissions.
- **Track the lifecycle** of each agent, including purpose, version history, and compliance checks before deployment.
- **Classify risk** by autonomy and criticality so that oversight and escalation scale with exposure.
- **Hold audit trails** of behavior, communication, and decisions for monitoring and incident investigation.
- **Secure discovery** through cryptographic identity and federated approaches such as MCP and A2A agent cards.
- **Feed governance KPIs** such as trust, autonomy, efficiency, and incident frequency, and connect to CI/CD, access control, and incident response.

## Frameworks and protocols to build on

| Category | Examples | Governance role |
|---|---|---|
| Agent protocols | Model Context Protocol (MCP), | Structured, policy-aware messages carrying |

| Category | Examples | Governance role |
|---|---|---|
| | Agent-to-Agent (A2A) | agent identity, capabilities, and constraints |
| Orchestration frameworks | Semantic Kernel, Copilot Studio, LangGraph | Embed machine-readable policies and validation checks in multi-step workflows |
| Multi-agent frameworks | AutoGen, CrewAI, SuperAGI, smolagents | Role taxonomies, delegated permissions, and semantic task coordination |
| Formal ontologies | RDF, OWL, SHACL | Explicit vocabularies for governance concepts, risk levels, and agent capabilities |
| Evaluation consoles | Conversational agent consoles with observability | Benchmark agent responses against semantic policy standards |

# Semantic Governance Operating Model

This section describes the Operations Workbench that monitors, visualizes, and governs lifecycle changes to knowledge artifacts such as vocabularies, taxonomies, and ontologies in a graph database. The workbench is designed for this type of federated, dynamic knowledge graph environment, with strong integration and visualization capabilities.

## Workbench Capabilities

## Unified Knowledge Graph Modeling

The Operations Workbench includes a schema editor for designing or importing knowledge graph models directly from existing graph databases or RDF sources. Ontology-based data (RDF, OWL, RDFS) can be bound to the schema, enabling visualization and monitoring of knowledge artifacts from multiple repositories without duplicating the data.

## Real-Time Data Integration and Change Binding

The workbench supports automatic schema extraction and real-time integration from federated data sources, including Neo4j, Amazon Neptune, Apache TinkerPop, and SPARQL endpoints. These connectors allow the workbench to display live data views of ontologies and vocabulary relationships, making it easy to detect and visualize changes as they propagate across taxonomies or linked models.

- Dynamic Binding: Updates in the graph database automatically refresh within workbench visualizations.
- Change Detection: Schema metadata can include timestamps or version identifiers, allowing new ontology versions or modified relationships to be highlighted.

## Visualization and Impact Analysis

The platform provides rule-based, interactive graph visualizations and dependency mapping tools for analyzing ontology relationships. Capabilities include:

- Node-based impact graphs to see how a taxonomy update affects dependent vocabularies or instances.
- Centrality, clustering, and pathfinding analytics to show semantic influence or dependency strength.
- Overlay metadata (like version number, date, or steward) via styles or color-coding for governance oversight.

## Lifecycle and Governance Monitoring

Graphs can be augmented with attributes such as:

- Version history
- Approval status
- Lifecycle state (draft, validated, deprecated)

- Authorized steward or domain owner

Using workbench dashboards and schema-linked attributes, administrators can monitor governance workflows, visualize change approval pipelines, and assess compliance against ontological or taxonomic policy guidelines. These dashboards transform abstract RDF governance rules into a live operational interface.

## Example Workbench Configuration

| Function | Implementation in the Operations Workbench |
|---|---|
| Change Monitoring | Link version metadata to nodes/edges and use visual cues to show modifications or pending approvals |
| Impact Visualization | Pathfinding analysis to display dependency propagation when a concept or class changes |
| Artifact Versioning | Visual panels to compare ontology versions and highlight added/removed entities |
| Governance Workflow View | Dashboard combining schema state, change history, and steward assignments |
| Integration with Graph DBs | Real-time API integration to Neo4j, Neptune, or any SPARQL endpoint |
| Role-based Access Control | Configurable access layers to manage who can modify or approve ontology changes |

## Architecture Integration

The Operations Workbench is deployed as an operations layer above the semantic stack:

Knowledge Repositories: Neo4j / GraphDB / Amazon Neptune

Integration Layer: Workbench schema bindings and federated connectors

Visualization Layer: Workbench UI built with the workbench designer

Governance Framework: Lifecycle metadata (status, steward, policy) stored directly within RDF or external SG system

Event Source (Optional): Kafka or graph triggers for asynchronous updates

## Practical Outcome

With this setup, the Operations Workbench becomes:

- A real-time control panel for ontology evolution and data integrity oversight.
- A semantic impact workspace showing downstream effects of vocabulary refinement.
- A governance-driven visualization suite allowing SG teams to monitor, validate, and authorize knowledge artifact changes as they occur.

In practice, the Operations Workbench integrates with SPARQL-compatible graph databases, detects and visualizes ontology-level changes, and manages governance workflows within an interactive operations environment.

## Implementation: the KE Workbench

The KE Workbench is the Company's implementation of the Operations Workbench described in this section and of the Semantic Modeling Workbench named elsewhere in this document. It is a set of browser pages served next to GraphDB, grouped as Model & standards, Rules & logic, Agents & operations, Customer 360, Products & Services, and Reference. Every page reads from and writes to the graph; Turtle and TriG files are exported releases of the graph, not a second place to edit (Section 3.3). Appendix E is the procedure for each page, and Appendix E.4 maps every page to the standard it applies.

| Function | KE Workbench pages |
|---|---|
| Change monitoring | Operations Dashboard (change log, causality, root cause); Model Manager; SSOT Editor; Mapping Agent Dashboard |
| Impact visualization | Operations Dashboard (causes and effects of a change set); Model Editor (Ontology) tiers and views; Service Assembly (copy number, Used by); Agent & Service Map |
| Governance workflow | Ingestion; Conformance Review (dispositions, load gates); PEO Outcome Review (SME decisions); Demand Gateway; PLM Lifecycle Dashboard |
| Artifact versioning | Model Manager; Model Editors; Conformance Review (version bump per change set); Release Notes |
| Integration with graph databases | All pages, through the workbench server and its GraphDB proxy; Ingestion › Reference ontologies for BFO and RO |

## Operating Structure: Areas, Domains, and the Service Catalog

Knowledge is organized the way the enterprise works. The enterprise is divided into four operational areas: Business Operations, Commercial Operations, HR Service Operations, and Technical Operations. Each area aggregates domains (departments), and a domain may nest up to three levels. Each domain delivers services and runs the processes that enable them. Two service portfolios meet in one enterprise service catalog above the areas: external services, which clients and their worksite employees receive, follow the HR Services & PEO Enterprise Business Model; internal domain services, which each department provides, are built on the APQC Process Classification Framework.

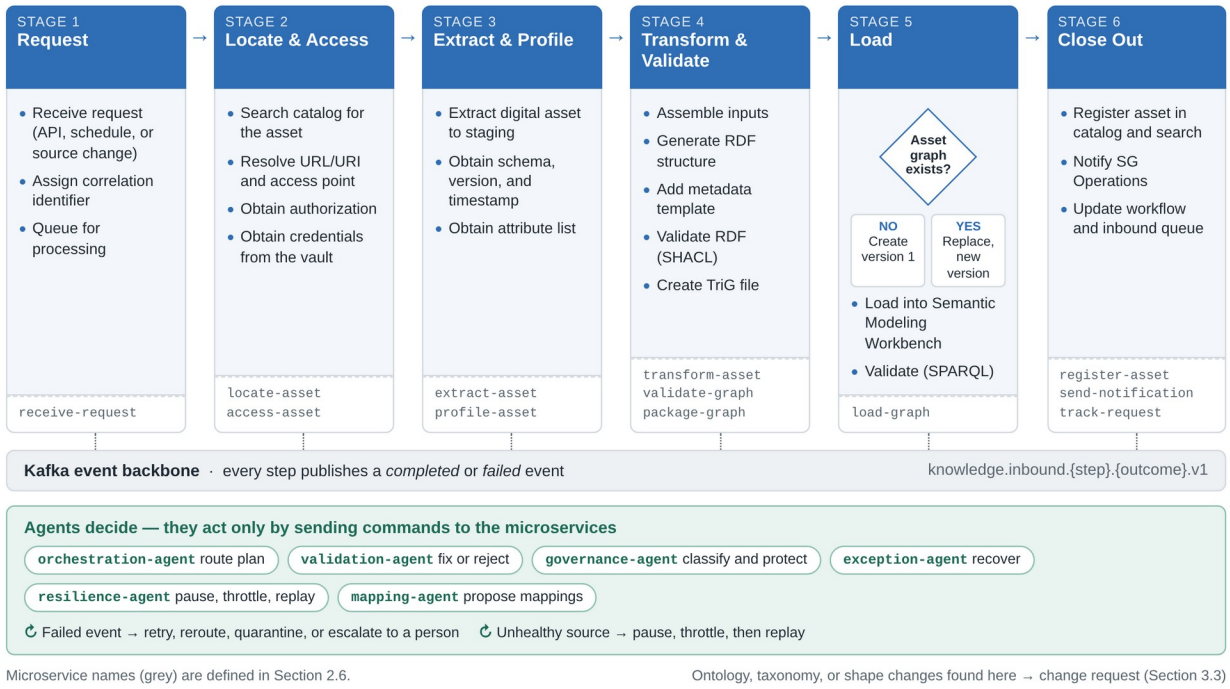
Enterprise › Operational area › Domain (L1 › L2 › L3) › Service › Process › Process step

Section 3.1 sets the modeling rules for areas and domains, Section 3.2 their names and identifiers, Section 3.12 the service and process model, and Section 3.13 the personas and jobs the services are measured against.

# Inbound Semantic Services Design

### Inbound Flow

Inbound processing moves each asset through six stages, from request to close-out. Microservices perform each step and publish an event to the Kafka backbone; agents make the decisions and handle exceptions. Section 2.6 defines each microservice and agent.




Microservice names (grey) are defined in Section 2.6.

Ontology, taxonomy, or shape changes found here → change request (Section 3.3)

A robust automation architecture for Inbound Processing in Semantic Governance can be built using a network of Python-based microservices, coordinated AI Agents for decision-driven orchestration, and semantic tools like RDFLib integrated with Semantic Modeling Workbench's APIs. Each service handles one atomic function for clear traceability and schema compliance, while agents use orchestration logic to sequence the services, handle exceptions and retries, and enforce governance checkpoints between steps.

This framework merges microservices, agentic orchestration, and semantic governance automation, producing a repeatable and scalable inbound data handling model that conforms to MBSE-aligned digital engineering architectures.

## Architecture Overview

This pipeline aligns with semantic governance principles, combining AI orchestration, microservice modularity, and knowledge graph operations.

Core components include:

1. Microservices Layer (Python REST Services) for discrete operations.
2. AI Agent Layer using Semantic Kernel or LangChain for sequencing, reasoning, and error resolution.

3. Semantic Data Layer using RDFLib for RDF modeling, and Semantic Modeling Workbench Graph

   Store API for ingestion and validation.

## Workflow

1. Inbound request enters queue and invokes pipeline.
2. AI orchestrator calls microservices per order.
3. RDF file generation (`.TRIG`) occurs post validation.
4. File uploaded to Semantic Modeling Workbench Graph Store; ontology version validated.
5. Workflow entry updated; SG notified.

## AI Agents and Orchestration Logic

Using Semantic Kernel's multi-agent orchestration framework, three main AI agents can manage this pipeline:

- Orchestration Agent: Central decision unit that invokes sequential microservices (Sequential Pattern).
- Validation Agent: Evaluates ontology and SHACL constraint compliance.
- Governance Agent: Audits metadata and ensures policy conformance for schema naming and version control.

Each agent runs under Semantic Kernel orchestration, utilizing Sequential and Handoff patterns:

1. Scheduler → Asset Retrieval → RDF Generation → Validation → Loading → Ontology Versioning → Notification.
2. Agents share context stores to ensure schema, metadata, and governance coherence.

## Semantic Governance Compliance

Governance rules align roles, privileges, and review checkpoints:

- Policy Framework: Defines approval thresholds for ontology promotion.
- Metadata Governance: Ensures proper schema linkage and provenance attributes.
- Lifecycle Tracking: Captures versioning and timestamps in RDF metadata.
- Audit Trail: Updates Semantic Governance (SG) workflow with activity logs.

## Microservices Breakdown

| Service | Function | Tools / Libraries |
|---|---|---|
| Request Scheduler | Queue inbound requests, assign IDs, and trigger workflow. | Celery + Redis |
| Search Digital Asset | Query asset repository with metadata filters. | ElasticSearch / Custom API |
| Search Catalog | Locate relevant schema or ontology source. | Semantic Modeling Workbench Query Catalog API |
| Extract Digital Asset | Download or parse content from stored archive. | Python requests , JSON / XML parser |
| Obtain URL/URI & | Resolve persistent identifiers. | W3C URIRef via RDFLib |

| Service | Function | Tools / Libraries |
|---|---|---|
| Access Point | | |
| Obtain Authorization & Credentials | Token exchange with OIDC or API keys. | Authlib / Keycloak |
| Obtain Schema, Version, Datetime | Retrieve ontology meta-data and constraints. | SPARQL Query via RDFLib |
| Obtain Attributes (list) | Parse RDF predicates or class properties. | RDFLib Graph.triples() |
| Assemble Inputs → Generate RDF | Serialize data into RDF triples. | RDFLib Graph.serialize() |
| Add Metadata for Ontology Template | Append custom annotations. | RDFLib Namespace DCTERMS , , OWL RDFS |
| Validate RDF Structure | Apply SHACL constraints within Semantic Modeling Workbench. | SHACL API in Semantic Modeling Workbench |
| Create .TRIG file | Use RDFLib .serialize(format="trig") . | RDFLib |
| Load Digital Asset to Semantic Modeling Workbench | POST to Graph Store API ( ). /rdf-graphs/service | Semantic Modeling Workbench Graph Store API |
| Ontology Versioning Service | Check and auto-increment ontology versions. | SPARQL Query + Version registry |
| Validate Ontology (SPARQL) | Check consistency, domain/range types. | RDFLib SPARQL engine |
| Queue & Workflow Updating | Update inbound and completed job states. | REST DB / Kafka topic |
| Notification Service | Notify SG Operations of completion. | SMTP / Slack API |

*Section 2.6 turns this breakdown into a reusable catalog of Kafka-based microservices and decision-making agents, with standard naming, logging, exception handling, and a search facade; where the tools listed here differ, Section 2.6 applies.*

# Deterministic Knowledge in the Knowledge Fabric

Incorporating deterministic knowledge through a top-down knowledge fabric—built with taxonomies, ontologies, federated graphs, and AI—enables strict policy, controls, and the emergence of adaptive behaviors within business processes.

### Top-Down Knowledge Fabric Construction

- Taxonomies as Foundation: Taxonomies classify concepts and entities hierarchically, providing clear categories and preferred vocabulary for enterprise data and business objects. They are invaluable for structuring information and supporting efficient search and retrieval.
- Ontologies for Enriched Semantics: Ontologies map relationships, attributes, and rules governing how business objects connect and behave, turning static classifications into a dynamic semantic network. Unlike pure taxonomies, ontologies add context, constraints, and reasoning capabilities, crucial for deterministic policy enforcement and compliance.
- Federated Knowledge Graphs: By connecting multiple ontologies (internal and external) into federated graphs, organizations create a unified semantic layer. This enables contextual policies and enterprise-wide controls to propagate dynamically, supporting federated compliance, audit trails, and knowledge-sharing.
- AI for Automation and Emergence: Embedding taxonomies and ontologies in AI platforms—especially LLMs, semantic tagging, and graph-based reasoning—enables automatic validation, classification, and the surfacing of emergent behaviors. AI tools can enrich taxonomies, suggest new concepts from unstructured data, and enforce semantic rules in real-time process automation.

### Delivering Policy, Controls, and Emergent Behaviors

- Policy and Controls: Deterministic knowledge models enable rule-based compliance, auditability, and controlled vocabularies. Semantic tagging and graph relationships ensure that assets, decisions, and user actions are governed by centrally defined policies and ontological rules.
- Emergent Process Behaviors: When business processes are modeled within an ontology-driven fabric, AI can leverage relationships, constraints, and semantic context to adapt workflows in response to dynamic triggers—such as shifting requirements, compliance checks, or new business insights.
- Unified Traceability and Responsiveness: Every process touchpoint, asset, or decision tagged to the knowledge graph reflects the current policy landscape. Automated reasoning embedded in the graph fabric enables deterministic action

  with room for emergent adaptation aligned to best practices and business outcomes.

## Implementation Highlights

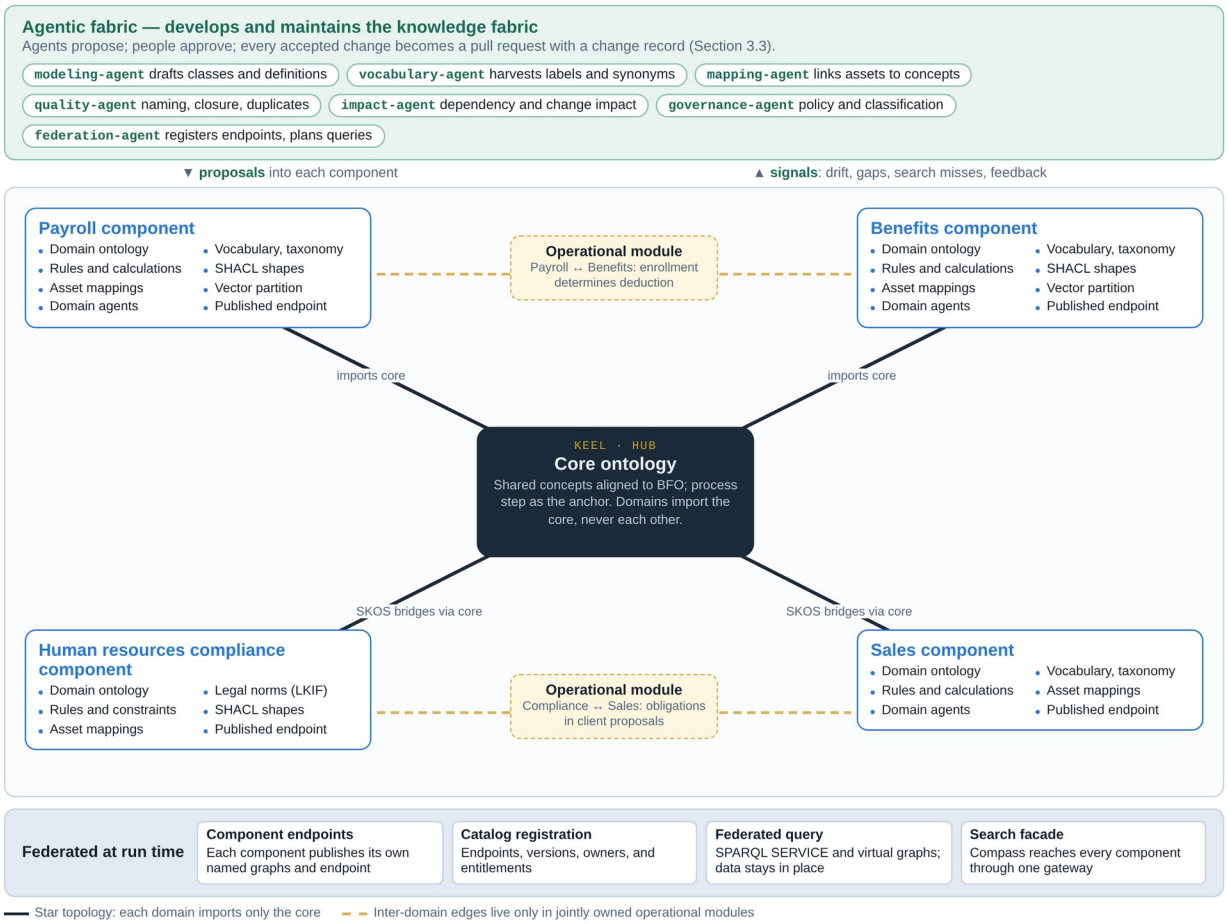
- Transform existing taxonomies into enriched ontologies and integrate them into federated knowledge graphs.

- Use Semantic Web standards (like RDF, OWL, SKOS) to link vocabularies and support automated reasoning.
- Integrate AI-powered entity extraction and tagging to surface new relationships and ensure policy adherence in real time.
- Map all process activities to ontology classes so contextual policy and controls are enforced automatically while enabling workflows to evolve responsively.

This approach delivers highly regulated, transparent, and adaptable processes—utilizing deterministic knowledge frameworks and AI to balance compliance and innovative emergent behaviors in service delivery

## Developing the Knowledge Fabric with an Agentic Fabric

The **knowledge fabric** is the governed, federated body of meaning described in this section: ontologies, vocabularies, taxonomies, rules, and mappings, connected across domains. The **agentic fabric** is the governed set of AI agents that help build, maintain, and use it. In development, agents do the repetitive work of drafting, harvesting, checking, and connecting; people decide what becomes authoritative. Two design choices make this work at enterprise scale: a **component design** that packages each domain as a self-contained, versioned unit, and a **federated topology** that connects those components through the core rather than to each other.



### Component Design

Each business domain is delivered as a knowledge component with the same standard parts, so any team can build, test, release, and reuse a domain the same way:

| Part | Contents | Owner |
|---|---|---|
| Domain ontology | Classes and intra-domain properties that extend the core (Section 3.1) | Domain knowledge engineer |
| Vocabulary and taxonomy | SKOS concept schemes, preferred, alternate, and persona labels (Section 3.8) | Domain data steward |
| Rules and calculations | The domain's entries in the single source of truth (Section 3.5) | Domain subject-matter expert |
| Shapes and norms | SHACL shapes for data quality; LKIF norms where regulation applies | Domain knowledge engineer |
| Asset mappings | Mappings from source schemas and digital assets to domain concepts | Domain data steward |
| Vector partition | Domain retrieval partition anchored to ontology IRIs (Section 2.5) | Platform team |
| Domain agents | Agents scoped to the domain, with their allowed actions and thresholds | Domain owner and Architecture |
| Published interface | Named graphs, endpoint, Decision service rules, and search registration | Platform team |
| Component manifest | Version, dependencies on the core and its area container, owners, and release notes | Domain owner |

- **Depends on the core and its area only:** a component imports the core ontology and its area container, and never imports another domain component.
- **Released as a unit:** each component has its own semantic version, source repository or folder, code owners, and release through the pipeline in Section 3.3.
- **Replaceable and reusable:** a component can be upgraded, rolled back, or reused by another business unit without changing the others, as long as its published interface stays compatible.

### Federated Topology

Components are connected in a **star topology** with the core ontology at the hub. The same shape applies at every level, which keeps the number of connections linear as domains are added, rather than growing with every pair of domains:

| Level | Topology rule |
|---|---|
| Logical (imports) | Every domain imports the core and its area container; domains in different areas connect only through enterprise service catalog entries (Section 3.1) |
| Terminology (mappings) | Each domain maps its terms to the core through reviewed SKOS bridges; domains never map directly to each other (Section 3.5) |
| Physical (storage) | Each component publishes its own named graphs and endpoint, registered in the catalog with version, owner, and entitlements |
| Run time (queries) | The federation-agent plans federated queries across registered endpoints and virtual graphs; data stays in place (Section 2.6) |

### Agentic Fabric for Development

The development agents extend the run-time agents in Section 2.6 and follow the same guardrails: they act only by proposing, every proposal is reviewed, and every accepted change becomes a pull request with a change record.

| Agent | Development task | Approved by |
|---|---|---|
| modeling-agent | Drafts classes, definitions, and properties from approved requirements (ORSD, Appendix A.4) using the term definition standard (Section 3.7) | Domain knowledge engineer |
| vocabulary-agent | Harvests candidate alternate and persona labels from search logs, documents, and source systems | Domain data steward |
| mapping-agent | Proposes mappings from new source schemas and assets to domain concepts | Domain data steward |
| quality-agent | Checks naming conventions, missing definitions, orphan terms, and near-duplicate concepts before review | Automated gate |
| impact-agent | Lists every rule, mapping, metric, service, and agent affected by a proposed change | Change reviewer |
| governance-agent | Applies classification, access, retention, and licensing policy to new components and assets | Governance owner |
| federation-agent | Registers component endpoints in the catalog and keeps the federation plan current | Platform team |

### Development Loop

1. A requirement is approved with its competency questions (ORSD).
2. Development agents draft the change in a sandbox copy of the component.
3. The quality-agent and SHACL shapes check the draft; the impact-agent lists what it affects.
4. The owners review and accept, amend, or reject each proposal.
5. Accepted changes are committed as a pull request with a change record and released through the pipeline (Section 3.3).
6. The released component is registered for federation and search; run-time signals (drift, search misses, feedback) return to step 1 as new requirements.

## Alignment with Keel and North Star

The knowledge fabric and the agentic fabric are expressions of the Keel and North Star foundational architecture (Section 2.1). The fabric is the keel; the agents work on the bridge, in the hull, and at the waterline; governance is the ballast; and every outcome is measured against the North Star.

| Fabric element | Vessel layer | Alignment |
|---|---|---|
| Core ontology and domain components | Keel | Governed meaning, authored once and versioned; holds no run-time state |
| Operational modules and SKOS bridges | Keel | Cross-domain meaning made explicit and owned, never implied in code |
| Federated endpoints, virtual graphs, mappings | Hull | Connect the keel to moving sources and keep source changes out of the meaning |

| Fabric element | Vessel layer | Alignment |
|---|---|---|
| Run-time agents (search, answers, exceptions) | Bridge and hull | Read meaning from the keel; hold none of their own |
| Development agents and drift signals | Waterline | Detect change and turn it into proposals and change requests |
| Change control, approvals, provenance | Ballast | Keep the fabric upright: nothing becomes authoritative without review |
| Measured outcomes | North Star | Trustworthy AI, drift-proof integration, audit-grade lineage, faster delivery |

- **Meaning lives only in the keel:** agents, services, and applications reference keel IRIs; none defines meaning of its own.
- **Stack-independent:** components are standard RDF, OWL, SKOS, and SHACL, so any platform or vendor tool can consume them through the published interfaces.
- **Steer by the North Star:** every component release reports against the North Star outcomes, measured in the target operating model (Section 3.6).

# Demand Management and Agility

To build an agile workforce and intelligent service delivery platform, implement unified demand management that integrates external requirements with internal customer needs, systematically vet demands through compliance and corporate priorities, and rationalize these into actionable solution design. This approach enables responsive adaptation while maintaining strategic alignment and governance.

## Unified Demand Management Framework

- Centralized Demand Hub: Establish a platform where all external demands (clients, regulatory, partners) and internal needs (operational teams, business units) are captured, tracked, and integrated for visibility and traceability.
- Prioritization Engine: Apply decision criteria to rank and vet demands—beginning with internal customer impact, then subjecting shortlisted demands to compliance checks (risk, regulatory, security) and corporate strategic needs (growth, cost management, ESG).
- Governance and Rationalization: Introduce a governance workflow where compliance, regulatory, and executive teams review, rationalize, and re-prioritize demand pools, ensuring only high-value, feasible initiatives progress to solution design.
- Agile Solution Design Integration: Connect prioritized demands directly to agile solution teams and service architects, who build and extend platforms around the most urgent validated needs, with continuous feedback loops for adaptation and improvement.

## Agile, Intelligent Workforce Enablement

- Real-time Orchestration: Use telemetry-driven orchestration mechanisms (for example, event-driven workflow engines fed by operational telemetry), enabling intelligence to dynamically allocate resources, reconfigure processes, and manage workloads based on the current mix of demands and operational realities.
- Transparent Resource Allocation: Managers and teams utilize dashboards and portfolio planning tools to match workforce capacity and expertise to the rationalized priority list, fostering agility and cross-team collaboration.
- Compliance-Led Adaptability: Security, compliance, and policy controls operate seamlessly within this framework, ensuring every solution adheres to standards while empowering innovation.

## Steps to Implement

1. Consolidate all demand streams (external, internal) into a unified digital hub.
2. Apply multi-layered prioritization: internal value, compliance, and strategic alignment.
3. Governance teams vet and rationalize, resolving conflicts and ensuring cohesiveness.
4. Feed prioritized, rationalized demands to agile solution design teams.
5. Deploy intelligent orchestration and adaptive resource management for workforce and service delivery.

This approach creates a dynamic, compliant, and value-driven ecosystem for extending agile workforce capabilities and intelligent service delivery, tightly aligning technology solutions with real business needs in a fast-evolving environment.

# Neuro-Symbolic Foundation for AI

Enterprise AI needs two kinds of intelligence. Neural models, such as large language models and the embeddings held in vector databases, bring fluency, pattern recognition, and the ability to work with unstructured content. Symbolic models, such as ontologies, knowledge graphs, and rules, bring explicit meaning, logical constraints, and reasoning that can be inspected. A neuro-symbolic architecture combines the two so that AI can think with knowledge rather than only learn from data. Semantic governance is what makes the symbolic half trustworthy: it decides which meanings are authoritative, how they change, and how they are enforced.

## Why the symbolic layer matters

- **Explainability.** When relationships, hierarchies, and constraints are encoded explicitly, an AI answer can be traced back to the facts and rules that produced it, rather than to opaque model weights.
- **Contextual intelligence.** Pattern recognition alone cannot tell an agent what a term means in this business, which actions are permitted, or how two concepts relate. The knowledge graph supplies that context.
- **Interoperability.** Open W3C standards let internal and external data, ontologies, and applications connect without bespoke translation for every pair of systems.

## RDF: the atomic unit of meaning

In RDF every fact is a triple of subject, predicate, and object, each identified by a global IRI. Triples are small enough to update one fact at a time, yet any triple can link to any other, so the graph grows without breaking. This is how the core ontologies (Process, Service, Asset, Product, Party, Location, Time, Event, Issue) and the domain ontologies built on them become one connected fabric, and how structured and unstructured sources are unified without forcing them into a single schema.

## From facts to knowledge: RDFS, OWL, and SHACL

RDF Schema and OWL add meaning on top of facts. Class hierarchies and property characteristics allow the graph to infer relationships that were never stated directly, for example that a variance report is a kind of financial analysis. SHACL adds the governance half: shapes declare what valid data looks like and which policies apply, so changes can be validated before they are accepted. In this architecture SHACL acts as a policy gate for agents and workflows, as shown in the FP&A sequence diagrams (Section A.11).

## SPARQL: querying with graph thinking

SPARQL queries patterns across the graph rather than joining tables. Multi-hop questions, such as which cost centers depend on a driver that changed last quarter, become a single query. The same ability to extract a precise, context-rich subgraph is what grounds retrieval-augmented generation: the Federated RAG 2.0 architecture (Section 2.1) retrieves governed facts by meaning before the language model writes its answer.

## RDF and property graphs

Property graphs suit fast, localized traversal and analytics inside one application; RDF is built for ecosystems, with standard identifiers, inference, validation, and federation. Keep authoritative meaning in the governed RDF fabric and project it into property graphs where an application needs them.

## How the pieces fit in this architecture

| Layer | Role | Where it appears |
|---|---|---|
| Symbolic knowledge | Core and domain ontologies, taxonomies, and SHACL policies authored in the Semantic Modeling Workbench and stored in the graph database. | Sections 2.1, 1.3 |
| Neural intelligence | Language models and embeddings in domain-aligned vector databases for semantic search and generation. | Section 2.5 |
| Bridge | The semantic scaffold links each embedding to ontology IRIs; domain agents maintain mapping ontologies. | Sections 2.5, 2.1 |
| Governance | ORSD requirements and KGCL change management keep the symbolic layer versioned and auditable. | Sections A.4–A.6 |

## Design principles

- **Make meaning explicit.** Define business concepts once, in the ontology, and reference them everywhere.
- **Validate before acting.** Route agent actions and data changes through SHACL policy checks.
- **Ground every answer.** Retrieve governed facts before generation, and cite the IRIs used.
- **Stay standards-based.** Use RDF, RDFS, OWL, SHACL, and SPARQL to keep the fabric portable and federated.
- **Evolve incrementally.** Record every schema change as a KGCL statement tied to an ORSD entry.

# Context Engineering

Context engineering is the practice of assembling everything a model sees at inference time: instructions, retrieved knowledge, memory, tool outputs, system state, and policy. It is where the governed knowledge described in this document is put to work. Prompt engineering is one part of it; vector databases are one of its sources.

## Three complementary disciplines

| | Semantic governance | Knowledge engineering | Context engineering |
|---|---|---|---|
| Purpose | Policy, compliance, audit | Formalize domain knowledge | Assemble dynamic context for AI |
| Produces | Rules, constraints, metadata layers | Ontologies, taxonomies, knowledge bases | Context pipelines, prompt templates, memory |
| Owned by | Governance and compliance teams | Knowledge engineers, ontology managers | AI engineers, context designers |
| Role in AI | Sets the rules and trust framework | Supplies structured meaning | Delivers the right context at the right time |

## From prompts to context

| | Prompt engineering | Context engineering |
|---|---|---|
| Focus | Wording of a single prompt | The full information environment at inference |
| Scope | One-off or short interactions | Multi-turn workflows and agents |
| Inputs | Instructions and examples | Prompts, retrieval, memory, tools, system state, policy |
| Scalability | Brittle as tasks grow | Designed to be reusable, governed, and robust |

## Components of a production pipeline

- **Composition:** select inputs such as retrieved documents, tool definitions, instructions, session history, and metadata.
- **Ranking and filtering:** score relevance by similarity, keywords, and recency; remove duplicates and noise.
- **Memory management:** summarize prior turns and recall preferences and history across sessions.
- **Tool integration:** bring results from code, queries, and APIs into the context in a usable format.
- **Instruction orchestration:** versioned, reusable prompt templates for multi-step and branched flows.
- **Dynamic retrieval:** fetch information as the task evolves rather than front-loading static context.
- **Format optimization:** clear sections, labels, and summaries to improve comprehension and token use.

- **Evaluation and feedback:** test context relevance and output quality; diagnose context versus model failures.
- **Policy and compliance:** guardrails, compliance checks, and audit metadata built into the pipeline.
- **Cost and scale controls:** scoping, compression, and prioritization to balance completeness with latency and cost.

## Context engineering and vector databases

Vector databases remain the semantic retrieval layer: they store embeddings and return relevant documents, memories, and data quickly and at scale. Context engineering sits above them. It combines what is retrieved with memory, tool output, and policy; manages token limits; and adapts what the model sees at each step of a multi-turn or multi-agent workflow. Used together, they address the limits of retrieval alone, such as information overload, hallucination, and context-window pressure. Section 2.5 describes the vector database design, and Appendix C extends context engineering with simulation.

# Solution Architecture

### Solution Architecture: Federated RAG 2.0 with Semantic Modeling Workbench & Gemini

This architecture creates an intelligent, self-optimizing system for managing enterprise knowledge. It establishes a dynamic link between abstract business concepts and concrete digital assets, powered by a component-based design that promotes scale and domain-specific learning. It is built on the Keel and North Star foundational architecture described at the end of this section.

## Top-Down Knowledge Layer (Core & Domain Ontologies)

This is the foundation of the architecture. It's the central source of business truth, defining what the company is, how it operates, and what it knows.

- **Core Ontologies:** A foundational set of ontologies (Process, Service, Asset, Product, Party, Location, Time, Event, Issue) that provides a consistent, high-level vocabulary for the entire enterprise. All domain-specific ontologies will extend and reuse these core concepts.
- **Domain-Specific Ontologies:** For each business domain (e.g., Sales, Marketing, HR), an ontology is built on top of the core ontologies. This layer enriches the core concepts with domain-specific terms, relationships, and business rules, representing the organization's unique knowledge in a machine-readable format.
- **Semantic Modeling Workbench's Semantic Modeling Assistant:** This agent is used to author and manage the core and domain-specific ontologies. It provides a visual, user-friendly interface for knowledge engineers and subject matter experts to create and extend these top-down structures.

## Bottom-Up Knowledge Layer (Digital Assets)

This layer represents the raw data and metadata from across the enterprise. It's the physical reality that the top-down layer seeks to model.

- **Schemas and Metadata:** This includes database tables, columns, business intelligence metrics, hierarchies, and other metadata from all digital assets (e.g., Salesforce, ERP, data lakes).
- **Digital Asset Inventory:** The schemas and metadata are automatically or semi-automatically ingested into a central repository (e.g., Data Plex in GCP), creating a comprehensive inventory of all digital assets.

## Mapping Layer & Federated Agents

This is the most critical and intelligent part of the architecture, where the connection between the top-down and bottom-up layers is established and continuously improved.

- **Mapping Ontologies:** Each bottom-up schema (e.g., `users_table`, `sales_transactions`) will have its own dedicated mapping ontology. This ontology formally defines the relationships (e.g., `isRepresentedBy`, `hasAttribute`) between specific top-down ontological classes and the bottom-up schema elements. This granular approach supports polymorphic meaning.
- **Semantic Modeling Workbench's Search & Discover Agent:** This agent acts as a federated interface. It uses the top-down and mapping ontologies to enable users to search for data using

business terms. It translates a business query like "Find customer emails for the Q3 sales campaign" into the technical query required to retrieve `users_table.email_address` from the data source.

- **Domain-Specific AI Agents:** An individual AI agent is assigned to each business domain (Sales, Marketing, HR, etc.). These agents are responsible for two core functions:

1. **Mapping at Scale:** They continuously learn and propose new mappings between top-down concepts and bottom-up schemas, accelerating the manual process. They identify patterns and suggest relationships that human experts might miss.
2. **Extending Knowledge:** They analyze unstructured data and user queries within their domain to propose extensions to the domain-specific ontologies, vocabularies, and taxonomies, ensuring the knowledge layer remains comprehensive and up-to-date.

## Federated RAG 2.0 with Domain Vector Partitions ("Virtual SLMs")

**Terminology note:** in this document, a "virtual SLM" means a domain-specific vector database partition that gives a single general-purpose LLM the knowledge a specialized small language model would otherwise hold. The MVP1 target topology deploys **one LLM plus domain vector partitions**; it does not deploy separate SLM instances. Dedicated SLMs remain an optional future optimization for high-volume domains.

This layer explains how the system interacts with a single large language model (Gemini) to provide grounded, context-aware responses without the need for multiple, costly specialized language models (SLMs).

- **Vector Databases per Domain:** A separate vector database is created for each business domain. These are not "virtual SLMs" in the traditional sense, but they serve as a virtual, domain-specific knowledge cache. They store vectorized representations of:

1. The top-down and mapping ontologies for that domain.
2. Conversational queries and their successful mappings.
3. Positive and negative feedback from users.

- **The Single LLM (Gemini):** A single, powerful LLM acts as the core reasoning engine. It doesn't need to be specialized for each domain because it has the knowledge layer to rely on.
- **Federated RAG 2.0 Workflow:**

1. **Query from User:** A user submits a query to the conversational AI interface (e.g., "What are our most profitable products?").
2. **Query Decomposition & Domain Routing:** The conversational AI and a separate routing agent (a simple rule-based agent or a specialized LLM call) analyze the query and identify the most relevant business domains (e.g., Sales, Finance).
3. **Federated Retrieval:** The query is sent to the vector databases of the relevant domains. The system retrieves the most relevant semantic mappings, scenarios, and terms. This is the **Retrieval** step of RAG.
4. **Enriched Context for LLM:** The retrieved information (the mappings, relevant ontology classes, and past successful queries) is combined with the user's original question to create a highly specific and grounded prompt.

5. Generation & Response: Gemini generates a response based on this enriched, grounded prompt. The response is then traced back to the mappings and ontology terms it used; only responses that pass the feedback controls below update the vector database.

### Feedback Validation Controls

To keep incorrect or hallucinated answers from being learned back into the vector space, feedback is written only through a controlled path:

1. **Traceability check:** every answer must cite the ontology IRIs and mappings it used. Answers that cannot be traced are never stored as feedback.
2. **Policy validation:** the cited mappings are validated with SHACL against the domain ontology before any update.
3. **Confidence threshold:** feedback is written automatically only when the mapping confidence is at or above the domain's threshold (initially 0.85) and the user has confirmed the answer; anything lower is queued for review.
4. **Human-in-the-loop review:** a Knowledge Engineer or Data Steward approves queued items, and every new mapping proposed by an agent requires approval before it becomes authoritative.
5. **Quarantine and rollback:** new feedback is versioned and held in a staging partition for one release cycle; it is promoted only if retrieval quality metrics do not degrade, and can be rolled back by version.
6. **Audit:** every write records the source query, response, reviewer, and version, so feedback can be traced and reversed.

Thresholds and the staging period are initial values to be calibrated during MVP1.

## Recursive Learning & Governance

The system is designed for continuous improvement and strong governance.

- **Recursive Learning:** Every successful query and mapping is stored as positive feedback in the domain-specific vector database. When a new query is submitted, the system first consults its learned mappings, continuously improving its accuracy and speed. If a query fails, the negative feedback is captured, allowing the AI agents to propose corrections or extensions to the ontologies or mappings.
- **Component Design:** The architecture is modular. Each domain is a self-contained component with its own ontology, vector DB, and AI agents. This allows for scalability and independent development without impacting other domains.
- **Central Governance:** The core ontologies and the mapping process are governed by a central team. This ensures consistency and prevents data fragmentation. A governance dashboard tracks the health of each domain's knowledge layer, monitors mapping confidence scores, and flags areas for human intervention.
- **LookML Integration:** The metadata for the schemas is automatically generated from the LookML models in Looker, ensuring that the bottom-up layer is always in sync with the live analytics layer. This data is then consumed by the Semantic Modeling Workbench agents and vector databases.

## Foundational Architecture: Keel and North Star

The solution architecture is built on the Keel and North Star architecture, which is the foundational architecture for this document. The **keel** is the governed, deterministic semantic foundation: ontologies, vocabularies, the single source of truth for rules and calculations, shapes, and metric definitions, authored once and versioned. The **North Star** is the bearing every layer steers toward: every decision, insight, and AI answer rests on governed, traceable meaning. Between them, the hull mediates between the keel and moving sources, the bridge holds the two core capabilities (Compass for finding and explaining meaning, Copilot for answering and acting on it), and the ballast of governance keeps the vessel upright.




Foundational architecture: Keel and North Star. Section references show where each layer is specified in this document.

The layers of this section map onto the vessel as follows:

| Layer in this section | Vessel layer | Role |
|---|---|---|
| Top-down knowledge layer (core and domain ontologies) | Keel | Governed meaning that every other layer reads |
| Bottom-up knowledge layer (digital assets) | The sea, described in the keel | Sources stay where they are; their RDF descriptions live in the keel |
| Mapping layer and federated agents | Hull | Links assets to concepts and answers queries in place |
| Federated RAG 2.0 with domain vector partitions | Bridge (Copilot) and hull | Grounded answers from one LLM, retrieval anchored to ontology IRIs |
| Search and discover agent | Bridge (Compass) | Finds assets by meaning through the search facade (Section 2.6) |
| Feedback validation controls | Waterline and ballast | Nothing is learned back without traceability, validation, and approval |
| Recursive learning and governance | Ballast | Change control, release pipeline, and monitoring (Part III) |

# Industry Alignment

**Why Sales Operations here?** The research alignment and the vector database design in this part (Sections 2.2 and 2.5) were developed against **Sales Operations** (leads, quotes, orders) as the first pilot domain. Appendix A uses **FP&A** as a second, fully worked domain. The pattern is domain-agnostic: core ontologies, a domain ontology, mapping ontologies, a domain vector partition, and domain agents. Any domain, including Sales Operations, FP&A, Marketing, or HR, follows the same structure.

This document outlines how our approach to building domain-specific knowledge—leveraging top-down ontologies mapped to bottom-up metadata via a process-centric model—aligns with the concepts and methodologies presented in the following two authoritative resources:

- Pliks et al., 2010, "Knowledge management and decision support in adaptive case management platforms" (pulled from IEEE - link below https://ieeexplore.ieee.org/abstract/document/7321628?signout=success)
- (alternate link): https://www.academia.edu/72947572/Knowledge_Management_and_Decision_Support_in_Adaptive_Case_Management_Platforms?email_work_card=view-paper
- Search-O1 Project, Enterprise Semantic Search Framework (https://search-o1.github.io/) (recommended by Rajender R Nednur)

## Overview of Our Approach

Specifically, our approach:

- Combines top-down ontology engineering with bottom-up metadata extraction, reflecting the dual-layer knowledge modeling advocated by Pliks et al. (see Sections 2 and 5);
- Leverages a process-centric semantic model aligned with Pliks' emphasis on business process impact on ontology structure (Section 4);
- Incorporates iterative validation and refinement mechanisms consistent with Pliks et al.'s recommendations (Section 6);
- Implements semantic vector embeddings and unified ontology-schema mappings that enable semantic and conversational search, which aligns strongly with Search-O1's architecture and goals (as detailed in their Concepts and Architecture pages);
- Enforces modular subdomain decomposition and context filtering, vital in Search-O1 and supported by Pliks et al. for scalable, domain-aware knowledge management;
- Provides a knowledge fabric foundation that enables intelligent agents to operate under a consistent top-down knowledge topology at enterprise scale.

This combined framework delivers a resilient, explainable, and scalable semantic search and conversational agent environment, practical for complex domains such as sales operations.

## Integration of Top-Down and Bottom-Up Ontologies

- Pliks et al. (Section 2, p. 3-5) highlight the critical value of integrating *expert-curated top-down ontologies* with *bottom-up extracted knowledge* to comprehensively capture enterprise semantics.
- Our approach mirrors this by utilizing a domain-specific sales operations ontology (top-down) mapped explicitly to metadata from digital asset schemas (bottom-up). This hybrid data model maintains semantic rigor while grounding ontology classes in practical, data-driven instances.

## Process-Centric Semantic Modeling

- In Section 4 (p. 9-11), Pliks et al. emphasize modeling business processes and workflows to appropriately structure ontologies, improving alignment with operational semantics and data source contexts.
- Our architecture organizes sales operations into subdomains (lead management, pipeline, order management), using a process-centric model to clarify concept relationships and improve mapping accuracy—a direct application of these principles.

## Iterative Mapping Validation and Refinement

- Section 6 (p. 14-16) describes the importance of continuous validation of ontology-to-data mappings supported by evidence and confidence metrics, suggesting iterative refinement to maintain semantic integrity.
- Our database captures mapping confidence scores, validation statuses, evidence texts, and training labels (positive/negative) to enable this ongoing refinement, ensuring conceptual accuracy and robustness.

## Ontology-Driven Semantic Search

- Pliks et al. discuss how ontology-driven integration supports semantic querying and retrieval (Abstract, Section 7, p. 17-18). Semantic layers allow for *context-aware*, *concept-based search* transcending simple keyword matching.
- Our use of dense vector embeddings aligned with ontology vocabulary and mapping tables operationalizes semantic search, empowering intelligent agents to understand queries at a conceptual level rooted in the knowledge fabric.

## Modularity and Scalability

- The paper advocates modular ontology design to accommodate the complexity of enterprise domains and simplify maintenance (Section 5, p. 12-13).
- Our design incorporates subdomain indexing and modular service areas to manage complexity modularly, in line with these recommendations.

## Semantic Embeddings and Vector Databases

- Search-O1's core architecture rests on transforming ontological and schema knowledge into vector spaces to support semantic similarity search (Concepts page).
- Our use of vector embeddings for ontology entities and schema metadata directly implements this principle, enabling robust semantic retrieval crucial for enterprise-scale knowledge access.

## Unified Ontology-Schema Mapping

- Search-O1 stresses mapping domain ontologies to heterogeneous metadata schemas for uniform enterprise insight and searchability (Architecture page).
- The mapping tables in our model reflecting entity correspondences, types, confidence, and evidence match this, providing organized, machine-readable links to unify disparate data assets.

## Contextual Subdomain Filtering

- The project promotes sophisticated knowledge layer modularization where subdomains or domains can be scoped and filtered during query or agent interactions.
- Our subdomain index and process-centric model embody this concept, enabling agents to operate with semantic precision by focusing on the most relevant domain slices.

## Intelligent Agents and Conversational Search

- Search-O1 envisions agents leveraging the knowledge fabric to power natural language search and complex workflows (Agent and Conversational Search demos).
- Our curated knowledge fabric, enriched with validated mappings, enables agents to perform contextually informed conversational search, consistent with their vision for scoping, reasoning, and retrieval.

## Capability Map & Alignment with Best Practices

| Capability | Supported By Our Approach | Corresponding Article Evidence |
|---|---|---|
| Comprehensive, accurate domain representation | Top-down ontology + bottom-up metadata mapping | Pliks et al. Sections 2, 5, 6 |
| Context-aware, process-centric semantic relations | Process-centric domain modeling | Pliks et al. Section 4; Search-O1 modular domain design |
| Semantic vector search and retrieval | Entity embeddings with vector indexes | Search-O1 vector space architectures; Pliks et al. semantic querying |
| Iterative validation and refinement | Mapping confidence, labeling, and evidence tracking | Pliks et al. Section 6 |
| Unified enterprise knowledge fabric | Schema-ontology mappings for digital asset consistency | Search-O1 unified ontology-schema architecture |
| Intelligent agent & conversational search support | Semantic layers plus mapping driven agent guidance | Search-O1 agent-based conversational frameworks |

# Primary Process Flows

## Ingestion & Initial Mapping

This phase describes how a new or updated data source is introduced into the system and how the initial mapping process is triggered.

1. **Source Schema Ingestion**: The process begins when a new data source schema is ingested or an existing one is updated. This event triggers the start of the lifecycle.
2. **Process Path Selection**: The system evaluates whether the ingested schema is completely new or an update to an existing one. This decision point, or exclusive gateway, directs the flow down one of two paths:

- **Manual Mapping**: If it's a new schema, a **Knowledge Engineer** or Data Steward must manually initiate the mapping process. This approach ensures human oversight for complex, new data structures.
- **Automated Mapping**: If it's an update, the process automatically begins an automated mapping routine. This path is used for efficiency when changes are minor and can be handled by AI agents.

## Domain-Specific Mapping & Guidance

Regardless of whether the process is manual or automated, the core mapping work is assisted by specialized agents to ensure accuracy and context.

3. **Mapping Assistance**: Both the manual and automated paths lead to a sub-process where the **Semantic Modeling Workbench** and its agents work to map the schema. This step is guided by a **Company Domain Agent**, a specialized AI agent trained on a specific domain. The domain agent provides critical context and guidance to ensure the mapping aligns with internal business knowledge.
4. **Mapping Execution**: The Semantic Modeling Workbench system, with the help of the domain agent,

performs the actual mapping. It links the new or updated schema to existing ontologies and identifies new relations. The output of this stage is a mapped schema ready for further processing.

## Parallel Post-Mapping Tasks

Once the core mapping is complete, the process branches into several parallel tasks to propagate the new knowledge throughout the system.

5. **Concurrent Updates**: A parallel gateway activates four concurrent service tasks to update and extend various components of the semantic ecosystem:

- **Vector Database Update**: The new semantic embeddings and mappings are immediately used to update the **Vector Database**, ensuring that the knowledge graph is current for retrieval-augmented generation (RAG).
- **Semantic Model Extension**: The core semantic models and ontologies are extended with the new relationships, concepts, and mappings. This maintains a centralized and authoritative source of knowledge.

- **AI Model Training**: The new data is fed to the **AI Training System** to further refine the models, improving their ability to understand, classify, and generate insights based on the updated knowledge.
- **Monitoring Metrics Capture**: Key metrics related to the mapping process—such as success rates, time taken, and quality scores—are captured and fed to the **Monitoring & Dashboards** system for analysis.

## Quality Assurance & Iteration

After all parallel tasks are complete, the system performs a critical quality check to ensure the integrity of the knowledge.

6. **Quality Assurance**: The concurrent tasks join at another gateway, and the process moves to a quality check. The system evaluates whether the mapping quality is satisfactory.
7. **Iterative Feedback Loop**: This is a key decision point in the process:

- If the mapping is **not satisfactory**, the process is routed back to the **Manual Mapping** step. A Knowledge Engineer must then review and correct the issues, re-initiating the cycle.
- If the mapping **is satisfactory**, the process proceeds to the final publishing stage.

**Optional extensions:** at the Quality Assurance gateway, proposed mapping changes can be tested in simulation before approval (Appendix C.1), and the causal relationships in the ontology can be checked with causal reasoning (Appendix C.2). Both are optional enhancements to steps 6 and 7, not prerequisites for MVP1.

## Publishing & Release

This final phase ensures that the approved, high-quality knowledge is formally published and made available to the broader organization.

8. **Scheduled Publishing**: The process waits for a **weekly timer event**. This ensures that approved mappings are released in a controlled, predictable manner rather than immediately.
9. **Ontology Publishing**: When the timer triggers, the **Publishing & Release Management** system publishes the new and updated ontologies. This makes the approved knowledge available for consumption by data products, applications, and end-users.
10. **Completion**: The process concludes once the publishing is successfully completed.

# Feature and Functional Breakdown

## Functional Features

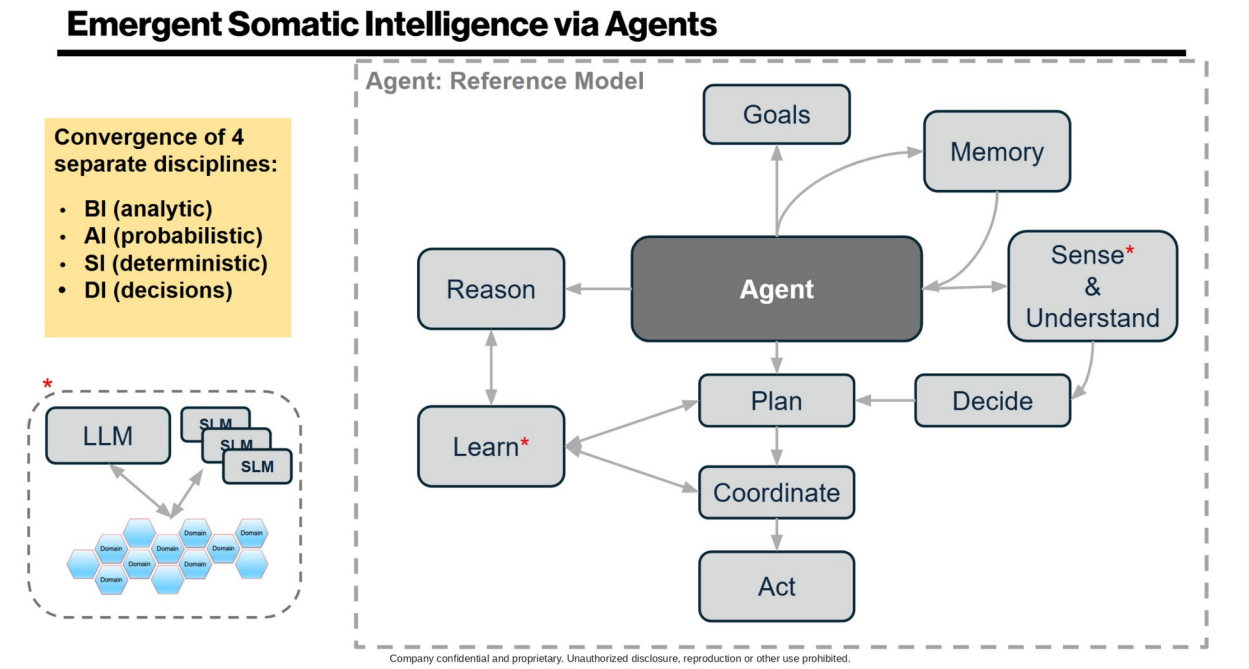
Functional features describe what the system does to support each step of the process.

- **Schema & Metadata Ingestion**: The ability to import data schemas and associated metadata from various sources (e.g., APIs, databases, files) into the platform.
- **Automated Mapping Engine**: An AI-powered service that automatically identifies, evaluates, and proposes mappings between ingested data schemas and existing ontologies.
- **Manual Mapping Interface**: A user interface that allows Knowledge Engineers to manually review, edit, and create mappings.
- **AI Agent Guidance**: A system for specialized AI agents (like the Company Domain Agent) to provide context and suggest mappings for a given domain.
- **Ontology Extension Service**: A service that validates and applies new classes, properties, and relationships to the core semantic models and ontologies.
- **Vector Embedding Generation**: The capability to convert structured data and relationships into high-dimensional vectors for use in a vector database for Retrieval Augmented Generation (RAG).
- **AI Model Fine-Tuning**: A process that updates and trains large language models (LLMs) with new data and mappings to improve their performance.
- **Dashboard & Metrics**: A user interface that visualizes key performance indicators (KPIs) related to data quality, mapping progress, and system health.
- **Publishing & Release Workflow**: An automated or semi-automated system that publishes approved ontologies and mappings to a production environment on a set schedule.

## Non-Functional Features

Non-functional features describe how the system operates, focusing on its performance and reliability.

- **Scalability**: The system's ability to handle an increasing volume of schemas and mapping operations without a degradation in performance.
- **Performance**: The speed and responsiveness of the system, particularly for automated mapping, AI training, and data retrieval tasks.
- **Reliability**: The system's ability to operate continuously with minimal downtime, including robust error handling and recovery mechanisms.
- **Security & Access Control**: Features that ensure data integrity, confidentiality, and proper authorization, such as role-based access to the mapping interface and data encryption.
- **Usability**: The ease of use of the platform's interfaces, especially for Knowledge Engineers and Data Stewards responsible for mapping and quality assurance.
- **Interoperability**: The capability of the system to integrate seamlessly with various external data sources and downstream applications.




**Note on the agent reference model:** the LLM and SLM boxes in this diagram are logical reasoning roles. In the MVP1 deployment both roles are served by the single LLM and the domain vector partitions described in Section 2.1; physical SLMs are not part of the initial topology.

## Functional Breakdown

### Top-Down Knowledge Knowledge Assembly

- Core Ontologies
- DS Ontologies
- DS Search Agent
- DS Mapping Agent
- DS Vector DB
- Orchestration Agent

### SG Operational Dashboard

- Demand Management (Intake)
- Lifecycle Management (all six SKL stages: ideation, development, testing, deployment, maintenance, retirement)

**Lifecycle terminology:** this document uses three distinct sequences. The **Semantic Knowledge Lifecycle (SKL)** has six stages (ideation, development, testing, deployment, maintenance, retirement) and governs every semantic asset (Section 2.4). The **mapping process flow** in Section 2.3 is the operational workflow that moves a schema through those stages. The **seven delivery phases** in Appendix B are a project plan for building the conversational AI solution, not an asset lifecycle.

- Service Level Management
  - Inbound
  - Processing
  - Outbound
- Issue Management
- Capacity Management
- Business Continuity Management

## AI Monitoring & Training Dashboard

- **For EACH Agent…**
- Monitoring
- Measurement
- Training (Learn = Pos + Neg)
- Transparency (Decide)
- Recommendations (Act)

## SG Scheduler

- Ingestion (initial, ongoing)
- Mapping at Scale (Bottom-Up)
- Extending Knowledge at Scale
- Batch Configuration
- Orchestration & Threading

## Prompt Engineering

- Sense
  - Intake
  - Reverse Engineering
- Understand
  - Relevancy (Domains)
  - Relevancy (Persona)
- Act
  - Orchestration
  - Search & Discover
  - Generate Results
    - Mapping Recommendations (Digital Schemas & Filtering)
    - Respond (tailoring and refining the results)

# Vector Database Design

This design uses Sales Operations as its worked example, consistent with Section 2.2. The same schema applies to FP&A; Appendix A.9 shows the FP&A variant of the vector table.

The following is a design for building vector databases to capture domain-specific context for guiding a sales operations agent—one constrained by a domain-specific ontology and tasked with mapping ontology classes to schema ontologies for digital assets—requires an architecture that integrates several nuanced components and processes.

## Core Architecture Overview

The database should be a hybrid system, combining:

- Vector DB (Domain) A vector store for semantic retrieval
- Mapping Ontology (Schema) - Relational/graph storage for explicit mappings and relations
- Metadata and embedding support for context-rich queries

## Key Components and Data Flows

- **Ontology Entities Table**: Stores classes and properties from both the sales operations domain ontology and schema ontologies for digital assets. Includes fields for entity type, metadata, embedding (vector representation), and links to sub-domains.
- **Embeddings Index:** Each ontology class/property (e.g., “Lead,” “Account,” “Opportunity,” “Contract” in sales ops) is embedded into a vector space, using semantic embedding models suitable for domain-specific alignment. **This supports the approximate nearest neighbor (ANN) search for mapping discovery.**
- **Mapping Table:** Explicit links between domain ontology classes and schema ontology classes representing digital assets, with fields for:
  - Source class
  - Target class
  - Mapping confidence (numeric or categorical)
  - Mapping type (equivalent, broader, narrower) (relation from Upper Ontology)
  - Validation status and history.
- **Sub-domain Context Index**: Sales operations consists of sub-domains (e.g., lead management, quoting, order fulfillment, pricing, sales analytics). Each sub-domain has distinct classes that may overlap or intersect. The index tracks associations, context vectors, and provides scope filtering for querying and agent guidance.

## Flow of Activities

1. **Entity Extraction:** Upon agent request, sales ops ontology classes are extracted and embedded. Metadata (name, description, relationships, sub-domain context) is stored.

2. **Semantic Mapping:** Matches between domain ontology classes and digital asset schema classes are proposed using vector similarity (semantic matching), then refined/ranked with syntactic, lexical, and structural features.
3. **Mapping Validation:** High-confidence mappings are validated and persisted with evidence; ambiguous mappings are flagged for review. Mappings are linked with sub-domain context for traceability.
4. **Sub-domain Navigation:** Agent queries can be scoped by sub-domain (e.g., “Order Fulfillment”), narrowing mapping search to relevant ontology sections.

### Example Schema

| Table | Key Fields | Purpose |
|---|---|---|
| Ontology_Entities | entity_id, name, description, type, embedding, subdomain | Stores ontology classes/properties with vectors and context |
| Schema_Entities | schema_id, name, type, description, embedding | Digital asset schema classes/properties |
| Entity_Mappings | mapping_id, source_entity_id, target_entity_id, confidence, type, status, evidence | Stores class-to-class mappings with context and validation |
| Subdomain_Index | subdomain_id, name, entities | Allows filtering and contextual search for agent queries |

## Sales Operations Sub-domains

Subset of “Level 1” sub-domains for the Domain (Sales Operations) can include:

- Lead Management
- Quote & Pricing
- Contract Administration
- Order Processing
- Sales Analytics
- Commission Management

Each sub-domain should have its own section of the ontology and mappings, supporting modular and scalable expansion.

The following is a design specification for integrating domain-specific ontologies with vector databases and agentic AI, built to improve accuracy, relevancy, and recall. The standard references ISO, IEEE, and OMG specifications to ensure technical rigor and interoperability.

## Design Specification: Sustainable Semantic Scaffold for Agentic AI and Vector Databases

## Purpose

The specification defines a modular architecture for domain-centric ontological integration with vector databases in support of agentic AI applications. The goal is to enable robust scenario-

mapping, semantic search, high recall, and precision, leveraging international standards to maximize interoperability and sustainability.

## Key Steps for Integration

- Use the OMG Ontology Definition Metamodel (ODM) for designing conceptual models and mapping UML/ER schemas to OWL and RDF/JSON-LD, guided by ISO/IEC/IEEE 24765 nomenclature for consistent terminology.
- Transform ontology classes/properties into vector embeddings using formalized graph-based sequence models mapped according to recommended OMG and W3C vocabularies, extending DCAT for metadata.
- Employ the OMG Distributed Ontology, Model and Specification Language (DOL) to unify models across heterogeneous data sources, supporting cross-domain interoperability and ontology linking.
- Ensure conformance by specifying two conformance levels:
  1. *Specification-level*: Applications formally import the ontologies (owl:imports) and resolve logical consistency.
  2. *Linked Data-level*: Applications reference one or more ontologies and use standard OMG/ISO URIs.
- Maintain modularity and reuse across deployments by referencing ISO standards for naming conventions, versioning, and documentation.

## Practical Strategies to Improve Search and Relevancy

- Encode ontological data as dense embeddings, mapping classes, relations, and instances into vector representations using IEEE-endorsed neural models (such as semantic encoders or graph neural networks), ensuring compatibility with OWL2/JSON-LD.
- Use the OMG Data Products Ontology (DPROD) spec to describe data products, metadata, entitlements, and data lineage for search and discovery, leveraging controls for compliance and distributed access.
- Apply RDF and DCAT vocabularies to structure entity metadata, ground semantic search, and facilitate cross-platform discovery and interoperability.
- Enable hybrid search and reranking pipelines that combine symbolic reasoning (OWL2/DOL-based inference) and vector similarity (cosine distance, reranked by ontological rules), guided by scenario-specific ontologies.
- Implement continuous integration and quality assurance for ontological objects following ISO standards for data quality, error reduction, and extensibility.

## Implementation Outline

| Component | Specification | Standard Ref |
|---|---|---|
| Ontology Design | ODM/OWL/RDF/JSON-LD for domain model and semantic mappings | OMG ODM, ISO/IEC/IEEE 24765, W3C RDF/OWL |
| Data Product Metadata | DCAT extension, DPROD schema for data products and lineage | OMG DPROD, W3C DCAT, ISO naming/versioning |
| Embedding | IEEE-endorsed deep graph encoders for semantic | IEEE ML/AI Guidance, OWL-API, |

| Component | Specification | Standard Ref |
|---|---|---|
| Procedure | vectorization | ODM |
| Vector DB Integration | Schema alignment, rich metadata indexing, linkage to ontological URIs | OMG ODM, ISO/IEC JTC 1/SC 7, IEEE interoperability |
| Hybrid Retrieval System | Pipeline for vector search reranked by ontological inference | OMG DOL, ISO error reduction standards |
| Quality & Compliance | Automated tests, versioning, data quality monitoring | ISO documentation, OBO Foundry practices |
| Scenario Mapping | Participatory ontology engineering, use-case embedding, modular workflows | OMG PSIG, domain-specific ontologies, W3C standards |

# Inbound Microservices, Agents, and Search

This section defines a reusable set of microservices and agents that implement the inbound semantic services in Section 1.4 for any digital asset, whether held in-house or by a third party. It supersedes the tool choices in the Section 1.4 microservices breakdown (for example, Celery and Redis for queuing) with one event backbone, Apache Kafka, and one set of naming conventions.

The design rests on a strict division of labor. **Microservices do the work**: simple, single-purpose, deterministic steps that access, extract, transform, and load an asset and log what happened. **Agents make the decisions**: which route an asset takes, how a failure is handled, when a source is paused, and when a person must decide. Search is a utility built into the Semantic Knowledge Layer: a facade of microservices that agents extend with federated graph queries.

## Division of Responsibilities

| | Microservices | Agents |
|---|---|---|
| Purpose | Perform one step the same way every time | Decide what to do when the path is not obvious |
| Logic | Deterministic; no business rules in code; behavior set by configuration and the request | Reason over catalog metadata, policies, history, and SKL context |
| State | Stateless; idempotent | Read state from events and the SKL; keep a decision log |
| Failure | Retry transient errors a bounded number of times, then publish a failure event | Classify the failure and choose a recovery action |
| How they act | Consume one command topic; publish success or failure events | Publish command events to microservices; never write to the SKL directly |
| Change control | Released through the software pipeline | Prompts, policies, and allowed actions versioned as knowledge artifacts (Section 3.3) |

## End-to-End Flow

```
receive-request → inbound-orchestration-agent (route plan)
  → locate-asset → access-asset → extract-asset → profile-asset
  → transform-asset → validate-graph → package-graph → load-graph
  → register-asset → catalog event → search-index, search-vector
  → send-notification;  track-request follows every event

on any failed event: exception-agent → retry | reroute | quarantine | escalate
on source health:    resilience-agent → pause | throttle | resume | replay
```

## Naming Conventions

Names follow the Company standards in Section 3.2: full words, no acronyms, lower case with hyphens for services, and dot-separated segments for topics.

| Element | Pattern | Example |
|---|---|---|
| Microservice | {verb}-{object} | extract-asset, load-graph |
| Agent | {responsibility}-agent | exception-agent, federation-agent |
| Command topic | knowledge.inbound.{step}.requested.v{major} | knowledge.inbound.extract.requested.v1 |
| Event topic | knowledge.inbound.{step}.{completed \| failed}.v{major} | knowledge.inbound.extract.completed.v1 |
| Retry topic | knowledge.inbound.{step}.retry.v{major} | knowledge.inbound.transform.retry.v1 |
| Dead-letter topic | knowledge.inbound.{step}.dead-letter.v{major} | knowledge.inbound.load.dead-letter.v1 |
| Exception topics | knowledge.inbound.exception.{raised \| resolved}.v{major} | knowledge.inbound.exception.raised.v1 |
| Catalog events | knowledge.catalog.asset.{registered \| updated \| withdrawn}.v{major} | knowledge.catalog.asset.registered.v1 |
| Consumer group | {service-or-agent-name} | transform-asset |
| Event type | com.insperity.knowledge.{area}.{step}.{outcome} | com.insperity.knowledge.inbound.extract.completed |
| Schema subject | {topic}-value | knowledge.inbound.extract.completed.v1-value |
| Asset graph | https://www.insperity.com/graph/asset/{source}/{asset-identifier} | https://www.insperity.com/graph/asset/benefits-portal/plan-guide-2026 |

- **Versions apply to contracts, not names:** the topic suffix is the major version of the event schema. A breaking schema change creates a new topic version and both run in parallel until consumers move, consistent with the versioning rules in Section 3.3.
- **One envelope for every event:** events use the CloudEvents specification (id, source, type, subject, time, data schema) with three Company extensions: correlationid, assetiri, and traceparent (W3C Trace Context).
- **Partition key is the source:** events are keyed by source system, which keeps ordering per source and isolates a misbehaving source from the others.
- **Claim check for content:** asset content never travels on Kafka. Extracted content is written to a staging object store; the event carries its location and checksum.

## Microservice Catalog

Twelve microservices cover the inbound flow in Section 1.4. Each consumes its own command topic and publishes a completed or failed event; the step name in each topic matches the service.

| Microservice | Does | Section 1.4 steps covered |
|---|---|---|
| receive-request | Accepts requests from an API, a schedule, or a source change notice; assigns the correlation identifier | Request; Scheduler |
| locate-asset | Resolves the asset's identifier, location, and access point from the source registry and catalog | Search for Digital Asset; Search Catalog; Obtain URL/URI; Obtain Access Point |
| access-asset | Obtains a short-lived access grant from the secrets vault; passes a reference, never the credential | Obtain Authorization; Obtain Credentials |

| Microservice | Does | Section 1.4 steps covered |
|---|---|---|
| extract-asset | Retrieves content and technical metadata through a connector adapter; writes it to staging with a checksum | Extract Digital Asset |
| profile-asset | Detects media type, schema, version, timestamps, and the attribute list | Obtain Schema, Version, & Datetime Stamp; Obtain Attributes |
| transform-asset | Applies the approved mapping (RML for structured sources; metadata extraction for documents) and adds the metadata template (DCAT, Dublin Core, PROV-O) | Assemble Inputs; Generate RDF Structure; Add Metadata for Ontology Template |
| validate-graph | Runs the SHACL shapes for the asset type and publishes the validation report | Validate RDF Structure |
| package-graph | Serializes the asset's named graph as TriG with its manifest | Create .TRIG file |
| load-graph | Replaces the asset's named graph in the SKL in one transaction; records the asset graph version | Load Digital Asset; Validate (SPARQL) |
| register-asset | Updates the asset catalog entry and publishes a catalog event for search and vector indexing | Close out Request |
| send-notification | Sends notices to people and systems through approved channels | Notify SG Operations |
| track-request | Maintains request status from the event stream for dashboards and the workflow queue | Update SG Workflow; Inbound Queue |

**Inbound loads are not ontology releases.** The inbound flow loads descriptions of assets (instance data) into asset graphs, each with its own version. Changes to ontologies, taxonomies, or shapes discovered during ingestion are raised as change requests and follow the release pipeline in Section 3.3; the inbound flow never modifies them directly. This replaces the "increment ontology version" step in Section 1.4.

### Standard Service Contract

- **Single purpose:** one command topic in; one completed and one failed topic out.
- **Idempotent:** the idempotency key is the correlation identifier plus the step plus the content checksum, so a repeated event produces the same result once.
- **Bounded retries:** transient errors are retried up to three times with exponential back-off and jitter; any other failure is published immediately as a failed event.
- **Classified failures:** every failed event carries an error category (transient, access, data, contract, or unknown), an error code, and a reference to the input.
- **Same image everywhere:** one container image per service version, promoted through development, test, stage, and production with configuration-only differences.
- **Observable:** health and readiness endpoints, metrics, and OpenTelemetry traces linked by the traceparent value in each event.

### Standard Logging

Every service and agent writes structured JSON logs with the same fields, provided by a shared logging library:

| Field | Content |
|---|---|
| timestamp, level | Time in UTC (ISO 8601) and severity |
| service, serviceVersion, environment | Who wrote the entry, which release, and where |
| correlationId, requestId, assetIri | Links the entry to the request and the asset |
| step, eventType, outcome | What was attempted and whether it completed, failed, or was skipped |
| durationMilliseconds | Time taken by the step |
| errorCategory, errorCode, errorMessage | Present on failures only |
| traceId, spanId | Links to the distributed trace |
| decisionId, rationale | Agents only: the decision taken and why |

- **Never logged:** asset content, credentials or tokens, and personal data. Logs reference the asset by IRI and checksum.
- **Provenance, not just logs:** completed loads also produce a PROV-O record in the provenance graph, linking the asset graph to the source, the services, and the mapping version used, so lineage is queryable alongside the change log (Section 3.3).

## Agent Catalog

| Agent | Decides | Acts by |
|---|---|---|
| inbound-orchestration-agent | The route for each asset: which steps, which connector, which mapping and shapes, based on source, asset type, and policy | Publishing the first command with a route plan; reuses an approved plan for known asset types without reasoning again |
| exception-agent | How to recover from a failed event | Choosing one action from the allowed list below and publishing the matching command or exception event |
| mapping-agent | How an unfamiliar schema maps to SKL classes and properties, using labels and definitions (Section 3.8) | Proposing a mapping for human review; approved mappings are stored in the mapping registry and reused |
| validation-agent | Whether a SHACL violation is a correctable data issue (for example, a date format) or a real defect | Requesting a corrected transform, or raising a data defect to the source owner |
| governance-agent | Data classification, access restrictions, retention, and licensing terms for each asset, especially third-party assets | Setting the asset graph's access labels and retention; blocking assets that fail policy |
| resilience-agent | When a source or downstream system is unhealthy, and how much load each source can take | Pausing and resuming a source's consumers (circuit breaker), throttling to third-party rate limits, and scheduling replays |

### Agent Guardrails

- **Act through events only:** agents publish commands to microservices; they never call source systems or write to the SKL themselves.

- **Allowed actions are a closed list,** versioned under change control. An agent cannot invent a new kind of action.
- **Confidence thresholds:** below a set confidence an agent escalates to a person instead of acting; thresholds are set per action.
- **Every decision is logged** with its inputs, the action taken, the rationale, and the model and prompt versions, so it can be audited and replayed.
- **Feedback improves future decisions:** outcomes of agent decisions, and human overrides, are fed back into the approved route plans, mappings, and thresholds (the recursive learning loop in Section 3.6).

## Exception Handling and Resiliency

1. A microservice retries a transient error up to three times, then publishes a failed event.
2. The exception-agent classifies the failure using the error category, the source's recent history, and the asset's route plan.
3. It chooses one allowed action: retry later, retry with alternate parameters, reroute (alternate connector or mapping), quarantine the asset, skip a non-critical attribute and continue, or escalate.
4. Escalations create an issue in run-time issue management (Section 3.6) with the full event history attached.
5. Events still failing after the allowed actions go to the step's dead-letter topic; nothing is silently dropped.
6. When the cause is fixed, the resilience-agent replays affected events from Kafka retention or the dead-letter topic.

| Pattern | Provided by | Purpose |
|---|---|---|
| Idempotent consumers | Every microservice | Safe retries and replays |
| Retry with back-off and jitter | Every microservice | Absorb brief outages without overloading the source |
| Circuit breaker per source | resilience-agent | Stop calling a failing source; resume after a health check |
| Bulkhead by source | Partitioning and consumer configuration | One slow or failing source cannot block the others |
| Rate limiting | resilience-agent with access-asset | Respect third-party limits and contractual terms |
| Claim check | extract-asset | Large content stays off the event backbone |
| Quarantine and dead-letter | exception-agent | Isolate bad assets for review without stopping the flow |
| Replay | resilience-agent | Reprocess from retained events after a fix |

## Search as an SKL Utility

Content stays where it lives, in Company repositories and third-party systems. The SKL holds an RDF description of each asset: what it is, which concepts, process steps, products, and clients it is

about, who owns it, how it is classified, and where it can be accessed (DCAT access and download links). Search finds assets by querying these descriptions and then resolves each result to its source location, applying the user's entitlements.

```
@prefix asset: <https://www.insperity.com/resource/domain-data-engineering-services/digital-asset/> .

asset:benefits-portal-plan-guide-2026 a dcat:Dataset, core:digital-asset ;
    dcterms:title        "2026 Medical Plan Guide"@en ;
    dcterms:subject      benefits:medical-plan, benefits:open-enrollment ;
    core:supportsStep    benefits:enroll-in-benefits-step ;
    core:hasClassification core:Internal ;
    core:sourceSystem    source:benefits-portal ;
    dcat:distribution    [ a dcat:Distribution ;
        dcat:accessURL   <https://benefits-portal.example/documents/plan-guide-2026> ;
        dcat:mediaType   <https://www.iana.org/assignments/media-types/application/pdf> ] ;
    prov:wasGeneratedBy  provenance:load-2026-10-13-0042 .
```

### Search Facade Microservices

| Microservice | Does |
|---|---|
| search-gateway | The single search interface for applications and agents: keyword, faceted, semantic, and graph queries; applies the caller's entitlements before anything else |
| search-index | Full-text and faceted search over asset descriptions, kept current from catalog events |
| search-vector | Similarity search over the domain vector partitions (Section 2.5) |
| query-graph | Runs SPARQL against the SKL, including federated queries (SPARQL SERVICE) to other graphs and virtual graphs over relational sources |
| query-external | Calls third-party search interfaces through adapters that translate results into the SKL vocabulary |
| merge-results | Combines and de-duplicates results from all sources by asset IRI and ranks them |
| resolve-asset | Turns a result into a usable link to the source, after checking the user's access at the source |

### Search Agents

| Agent | Extends search by |
|---|---|
| query-planning-agent | Turning a question into a graph query using concept labels, persona vocabulary (Section 3.8), and process-step context |
| federation-agent | Choosing which graphs, endpoints, and third-party sources to include, based on the catalog, source health, cost, and the user's entitlements; plans the federated query |
| result-explanation-agent | Explaining why each result matched, with the path through the graph and its provenance, for people and for AI answers |

- **Filter first, then rank:** entitlement filtering happens in search-gateway before any query is planned, so agents only ever see what the user may see (Section 3.8).
- **The facade is the only entry point:** applications and agents use search-gateway rather than calling the index, vector store, or graph directly, so security, logging, and ranking stay consistent.

- **Extensible by configuration:** adding a new repository means registering a source, a connector adapter, and a mapping; the facade, agents, and conventions do not change.

# Processing and Outbound Services, and Orchestration

Sections 1.4 and 2.6 give the inbound tier event contracts, a microservice catalog and decision-making agents. This section gives the processing tier (mapping, inferencing, learning) and the outbound tier (search, contextual relevancy, conversational AI, publishing, APIs) the same treatment, names who orchestrates each tier, and places every decision behind the AI circuit breaker gateway.

- **Upstream orchestration**: from a source change to a governed asset description in the Semantic Knowledge Layer (SKL). Defined in Sections 1.4 and 2.6.
- **Processing orchestration**: from a registered asset or an approved release to mapped, validated, materialized and indexed knowledge.
- **Downstream orchestration**: from governed knowledge to the people, agents and applications that consume it, and back again as feedback.

The rules of Section 2.6 apply without change: microservices do the work, agents make the decisions, agents act only through events, and nothing learned at run time becomes authoritative without change control (Section 3.3). Where this section and Section 2.1 differ on tooling, this section applies.

**Independent of any one AI tool.** Semantic models are built in the KE Workbench and do not depend on any model, agent framework or vendor. They are augmented by an evolving AI fabric organized around domain holons: each domain holon carries its own ontology, mappings, vector partition and agents, which insulates domains from each other's changes and keeps concerns separate. The model and agent framework named in Section 2.1 are one implementation of this fabric, not part of the contract; the services and agents in this section are defined without them.

**Every decision passes the AI circuit breaker.** Decisions made by agents and by people (route plans, mappings, releases, answers, learning updates, review dispositions) are evaluated by the AI circuit breaker gateway before they take effect. Ground truth is the released knowledge layer plus the measured data of the sources.

## End-to-End Flow

Every hand-off between tiers is an event, and every decision in every tier passes the AI circuit breaker gateway before it takes effect, so upstream and downstream orchestration share one contract, one gate and one trace.

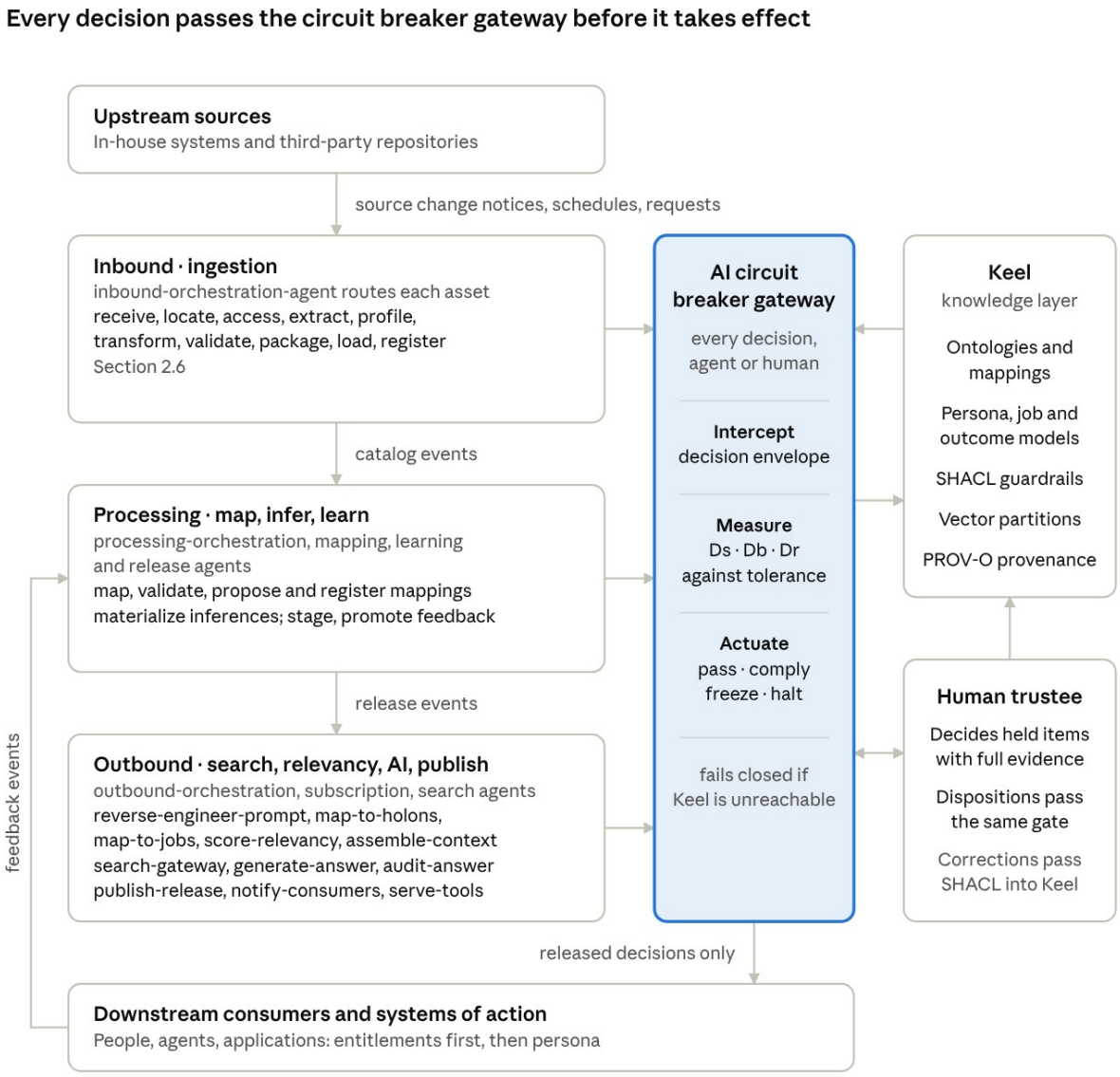



*Figure 2.7. Every decision passes the AI circuit breaker gateway before it takes effect*

The gateway measures each decision against Keel, the released knowledge layer, and releases, holds or blocks it. Only released decisions reach Keel, consumers and systems of action; held ones go to the human trustee, whose dispositions pass the same gate. Corrections reach Keel only as a new release through the SHACL gate, and feedback from consumers returns to processing, never straight into Keel.

## Naming Conventions

Processing and outbound reuse the Section 2.6 patterns with their own area segment: inbound, processing or outbound. The event envelope (CloudEvents plus correlationid, assetiri and traceparent), the claim check and the partitioning rules are unchanged.

| Element | Pattern | Example |
|---|---|---|
| Command topic | knowledge.{area}.{step}.requested.v{major} | knowledge.processing.map.requested.v1 |
| Event topic | knowledge.{area}.{step}.{completed \| failed}.v{major} | knowledge.outbound.index.completed.v1 |
| Retry topic | knowledge.{area}.{step}.retry.v{major} | knowledge.processing.materialize.retry.v1 |
| Dead-letter topic | knowledge.{area}.{step}.dead-letter.v{major} | knowledge.outbound.notify.dead-letter.v1 |
| Exception topics | knowledge.{area}.exception.{raised \| resolved}.v{major} | knowledge.processing.exception.raised.v1 |
| Release events | knowledge.release.{artifact}.{published \| withdrawn \| rolled-back}.v{major} | knowledge.release.ontology.published.v1 |
| Feedback events | knowledge.feedback.{signal}.recorded.v{major} | knowledge.feedback.answer.recorded.v1 |
| Decision events | knowledge.decision.{evaluate.requested \| pass \| comply \| freeze \| halt}.v{major} | knowledge.decision.freeze.v1 |
| Consumer subscription | knowledge.subscription.{consumer}.{event}.v{major} | knowledge.subscription.hr-advisory-agent.release.v1 |

- **Partition key by tier.** Inbound and processing events are keyed by source system, as in Section 2.6. Outbound events are keyed by consumer, so one slow consumer cannot hold up the others.
- **Release events are the downstream contract.** Consumers learn about new ontologies, taxonomies, shapes and mappings from knowledge.release events, never by polling GraphDB.
- **Feedback events never write knowledge.** They are inputs to the learning services, which raise change requests (Section 3.3).

## Processing Microservice Catalog

Processing turns registered assets and approved releases into knowledge that outbound services can serve: mapped, validated, inferred and learned from. It starts on a knowledge.catalog.asset event from inbound or a knowledge.release event from the release pipeline.

### Mapping

| Microservice | Does | Source |
|---|---|---|
| map-schema | Applies an approved mapping ontology to a registered asset's schema and writes the mapping links to the asset's mapping graph | 2.1 Mapping Layer; 2.6 transform-asset |
| propose-mapping | Packages an agent-proposed mapping, with labels, definitions and confidence, as a review item; never writes to the mapping registry | 2.1 Domain-Specific AI Agents; 2.6 mapping-agent |
| register-mapping | Stores a mapping a person approved in the mapping registry with its version, and publishes a release event | 2.6 mapping registry; 3.3 |
| validate-mapping | Runs the domain shapes against the mapped graph and publishes the validation report | 2.1 Feedback Validation Controls |

### Inferencing

| Microservice | Does | Source |
|---|---|---|
| materialize-inferences | Computes the approved closure (subclass, subproperty, inverse, named transitive properties) for a release and writes it into the release, kept apart from asserted statements | 3.1 Use of OWL 2; 3.10 |
| check-inferred-types | Reports inferred types that no one asserted, such as types produced by a property used on the wrong kind of subject | 3.10 |
| validate-release | Runs SHACL shapes and competency questions against a release candidate before it is published | 3.3; 3.6 Measurement |

### Learning

| Microservice | Does | Source |
|---|---|---|
| collect-feedback | Consumes knowledge.feedback events from agents, microservices, search and users, and records each with its trace | 3.6 Recursive Learning |
| trace-feedback | Rejects feedback that cannot be traced to the ontology IRIs and mappings it used | 2.1 Feedback Validation Controls |
| stage-feedback | Writes traced, validated feedback that passed the gateway to the staging partition of the domain vector store; queues the rest for review | 2.1 Feedback Validation Controls |
| promote-feedback | Moves staged feedback into the live partition after one release cycle if retrieval quality has not degraded; rolls back by version otherwise | 2.1 Feedback Validation Controls |
| raise-change-request | Turns reviewed signals (new synonyms, mappings, constraints, terms) into change requests for Section 3.3 | 3.6 Recursive Learning |

Conformance Review dispositions in the KE Workbench are the design-time form of this loop (Section 3.6); collect-feedback through raise-change-request are its run-time form.

## Outbound Microservice Catalog

Outbound serves governed knowledge to people, agents and applications through one front door per kind of use. Every request is filtered by the caller's entitlements first and tailored by persona second (Section 3.8).

**Search** (Section 2.6): search-gateway, search-index, search-vector, query-graph, query-external, merge-results, resolve-asset. search-index and search-vector also consume knowledge.release events, so a new release is indexed the same way a new asset is.

### Contextual Relevancy

A prompt is ambiguous until it is anchored. Contextual relevancy reverse engineers each prompt and maps it back to domain holons and to persona, job, job step and desired outcome instances (Section 3.13); each mapping narrows what the prompt can mean before any context is retrieved. This realizes Sense › Reverse Engineering and Understand › Relevancy (Domains), Relevancy (Persona) in Section 2.4, and the Relevancy Mapping Engine of the AI circuit breaker's context engine.

| Microservice | Does | Narrows the prompt by |
|---|---|---|
| resolve-persona | Sets the caller's persona from the authenticated identity, never from the prompt (Section 3.8) | Who is asking: the persona's domains, process steps and jobs |
| reverse-engineer-prompt | Decomposes the prompt into its intent, the entities and terms it names, its constraints and the task it implies | What is being asked, in the prompt's own words |
| map-to-holons | Resolves the terms and entities to concepts in the core and domain ontologies, and so to the domain holons that own them | Which domains the prompt is about |
| map-to-jobs | Matches the intent to the persona's job, job map step and desired outcome instances | Which job and step the person is in, and how they judge success |
| score-relevancy | Computes the relevancy distance (Dr) the gateway uses, across five dimensions weighted by the persona and outcome models, and the ambiguity left: how many holons, jobs or steps still fit | Whether the prompt is anchored well enough to answer |
| assemble-context | Builds the context from the narrowed holons and job step: concepts, definitions, mappings, provenance, persona labels, within a token budget (Section 1.8) | Only what the anchored meaning needs |
| rank-context | Scores and filters candidate context by similarity, keywords, recency and fit to the job step; removes duplicates | Order and noise |

When Dr exceeds its tolerance or more than one holon or job still fits, the outbound-orchestration-agent asks a clarifying question naming the candidates instead of answering. The prompt, its mappings and its Dr are logged with the answer, so the trace runs from words to holons to jobs to the governed terms cited.

### Conversational AI

| Microservice | Does | Source |
|---|---|---|
| route-query | Decomposes a question and picks the relevant domain holons | 2.1 Federated RAG 2.0 |
| retrieve-context | Retrieves from the domain vector partitions and the graph through search-gateway | 2.1 Federated RAG 2.0 |
| generate-answer | Calls the model with the assembled context and returns the answer with the IRIs and mappings it used | 2.1 Federated RAG 2.0 |
| audit-answer | Rejects or flags answers that cite no governed terms; records each answer for the answer audit log | 2.1 Feedback Validation Controls; 3.6 |

### Publishing and APIs

| Microservice | Does | Source |
|---|---|---|
| publish-release | Promotes an approved, validated release with its materialized closure and release notes, and publishes the release event | 2.4 Publishing and Release Workflow; 3.3 |
| notify-consumers | Delivers release and asset events to each subscribed consumer with the change summary and the breaking-change flag | This section |

| Microservice | Does | Source |
|---|---|---|
| serve-graph | The read API over released graphs (SPARQL and REST), behind the entitlement filter | 2.4; A.13 |
| serve-tools | The Model Context Protocol (MCP) server: exposes graph access, policy validation and workflow triggers to agents as governed tools | A.13 |
| export-release | Writes release files (Turtle, TriG, CSV crosswalks) for consumers that load files instead of calling the API | This section |

Publishing has two meanings, kept apart: publish-release releases knowledge artifacts under Section 3.3; notify-consumers and export-release deliver them. The search facade's rule extends to every outbound family: applications and agents call the front door (search-gateway, generate-answer, serve-graph, serve-tools), never an index, vector store or graph directly.

## Agent Catalog and Orchestration

Each tier has one orchestrating agent that publishes the first command of a route plan; the other agents decide exceptions within it. The Section 2.6 orchestration-agent is named inbound-orchestration-agent from v2.1 to make its scope plain.

| Agent | Tier | Decides | Acts by |
|---|---|---|---|
| inbound-orchestration-agent | Inbound | The route for each asset (Section 2.6) | Publishing the first inbound command |
| processing-orchestration-agent | Processing | Which processing steps a registered asset or release needs: map, validate, materialize, index; reuses an approved plan for known asset types | Publishing processing commands; handing indexing to outbound through catalog and release events |
| mapping-agent | Processing | How an unfamiliar schema maps to SKL classes (Section 2.6) | Requesting propose-mapping; a person approves |
| learning-agent | Processing | Which feedback is a correctable signal, which needs review, and which becomes a change request | Requesting stage-feedback, promote-feedback or raise-change-request |
| release-agent | Processing and outbound | Whether a release candidate is ready and whether it breaks consumers | Requesting publish-release; flagging breaking changes to notify-consumers |
| outbound-orchestration-agent | Outbound | The route for each request: which front door, which domain holons, which persona, job and outcome context, and whether to answer or ask a clarifying question | Publishing outbound commands for search, context assembly and answer generation |
| query-planning-agent, federation-agent, result-explanation-agent | Outbound | As in Section 2.6 | As in Section 2.6 |
| subscription-agent | Outbound | Which consumers each release or asset event concerns, and when a consumer must be told before a breaking change | Requesting notify-consumers; scheduling replays for a consumer that was down |
| exception-agent, governance-agent, | All tiers | As in Section 2.6, now also for processing and outbound events | As in Section 2.6 |

| Agent | Tier | Decides | Acts by |
|---|---|---|---|
| resilience-agent | | | |

Every agent decision is a decision envelope evaluated by the AI circuit breaker gateway before the agent publishes the command that gives it effect.

**How the tiers hand off**

1. Inbound ends with register-asset publishing knowledge.catalog.asset.registered.
2. The processing-orchestration-agent picks it up, routes the asset through mapping and validation, and asks for indexing.
3. Approved changes to knowledge artifacts go through Section 3.3 and leave as knowledge.release events from publish-release.
4. The outbound-orchestration-agent serves requests from released knowledge only; the subscription-agent tells consumers what changed.
5. Answers, searches and agent outcomes return as knowledge.feedback events, which the learning-agent turns into staged feedback or change requests. This closes the recursive learning loop of Section 3.6.

## Decision Evaluation: the AI Circuit Breaker

The AI circuit breaker evaluates every decision before it takes effect, whether an agent or a person made it. Its deterministic veto is absolute: governed knowledge has authority over a probabilistic suggestion, and the human veto has final authority over both.

| Decision | Made by | Evaluated before |
|---|---|---|
| Route plan for an asset, release or request | Orchestration agents | The first command is published |
| Recovery action (retry, reroute, quarantine) | exception-agent, resilience-agent | The action command is published |
| Proposed or approved mapping | mapping-agent; knowledge engineer | register-mapping stores it |
| Release go or no-go | release-agent; release approver | publish-release promotes it |
| Answer or tool result | generate-answer, serve-tools | It is returned to the consumer |
| Learning update (staged feedback, ontology enhancement) | learning-agent; trustee | promote-feedback or raise-change-request acts on it |
| Review disposition | Reviewer (Conformance Review) | It is replayed on re-ingestion |

The gate is an AI circuit breaker gateway that sits between every decision and its effect, following the Sales Advisor Circuit Breaker architecture. No agent or service acts on the knowledge layer, a consumer or a system of action directly; every decision leaves as a decision envelope (output, persona, evidence, model or reviewer, and the pinned release), and the gateway decides what happens to it. Ground truth is the released knowledge layer (Keel) plus the measured data of the sources; the gateway keeps no copy of its own.

| Stage | Does |
|---|---|
| Intercept | Captures the decision envelope at the boundary where it would take effect, and pins the Keel release it is measured against |
| Measure | Checks the SHACL guardrails and deontic norms first, then computes three distances with uncertainty budgets: structural (Ds, the share of asserted facts that disagree with the ontology), behavioral (Db, how far the output is from the stated intent) and relevancy (Dr, across five dimensions, weighted by the persona and outcome models) |
| Actuate | Returns a verdict and routes the decision: release it, hold it for the human trustee, or block it |

| Verdict | When | Effect |
|---|---|---|
| HALT | A guardrail is breached | Blocked; deterministic, cannot be approved |
| FREEZE | Any distance at 1.5 times its tolerance or more | Held for the trustee, with a manual fallback |
| COMPLY | Any distance above its tolerance | Held for review or corrected before release |
| PASS | All distances within tolerance | Released to its effect |

- **Consequence tightens tolerances.** Tolerances shrink as the decision's consequence weight rises. High-consequence decisions and any decision carrying a guardrail are evaluated synchronously; the rest asynchronously, by sample.
- **Fail closed.** If the gateway or Keel is unreachable, guardrail-bearing and high-consequence decisions stop; low-consequence ones continue in advisory mode and are flagged for review.
- **Agent state follows verdicts.** An agent is RESTRICTED after one FREEZE or two COMPLY verdicts, and DROPPED after a HALT, two FREEZE verdicts or a trust score below 0.50. Fidelity compounds along a chain of decisions.
- **People are measured too.** A reviewer's disposition goes through the same gateway; corrections reduce drift, recalibration restores an agent, and overrides count against trust.
- **Calibration loop.** Corrections pass the SHACL gate and then ship to Keel as a new standards release; they never change Keel directly. Data tied to a HALT or FREEZE is excluded from learning.
- **Provenance.** Every evaluation and disposition is recorded in PROV-O; distance, fidelity and MTBH time series feed statistical process control baselines.

| Circuit breaker subsystem | Where it sits |
|---|---|
| 1 Semantic knowledge layer | The knowledge layer: released ontologies, mappings, vector partitions, approved inferences |
| 2 Context engine | Contextual relevancy: reverse-engineer-prompt, map-to-holons, map-to-jobs, score-relevancy (Dr); the persona, job and outcome instances supply the weights |
| 3 Veto gate | The AI circuit breaker gateway (evaluate-decision): intercept, measure Ds, Db and Dr against consequence-weighted tolerances, actuate |
| 4 Recursive learning and model governance | collect-feedback through raise-change-request; tripped data goes to the exclusion filter |
| 5 Human interaction | The trustee decides held decisions and signs off learning updates; MTBH and the trust score are reported |
| 6 Cognitive immunity | Reviewers and knowledge engineers are evaluated too, so human decisions |

| Circuit breaker subsystem | Where it sits |
|---|---|
| | stay independent of the agents |
| 7 Predator-prey validation | Red-team cycles against agents and dynamically randomized tolerances, hidden from the agents |

evaluate-decision is the microservice behind the gateway. It consumes knowledge.decision.evaluate.requested.v1, measures the envelope against the pinned Keel release, and publishes one of knowledge.decision.{pass | comply | freeze | halt}.v1 with the distances, tolerances, fidelity and the rule that decided it.

## Resilience and Guardrails

The Section 2.6 service contract, logging standard, exception flow and agent guardrails apply to every processing and outbound service and agent. This section adds what is specific to the downstream side.

| Pattern | Provided by | Purpose |
|---|---|---|
| Circuit breaker per consumer | resilience-agent | Stop delivering to a failing consumer; resume after a health check; replay what it missed |
| Bulkhead by consumer | Outbound partitioning | One slow consumer cannot hold up deliveries to the others |
| Fall back to the previous release | serve-graph, release-agent | Restore service first (Section 3.6): a failed release is rolled back by version, and consumers keep reading the last good one |
| Degrade, don't guess | generate-answer, outbound-orchestration-agent | When retrieval or the graph is unavailable, return search results or decline; never answer without citations |
| Breaking-change notice | subscription-agent | A release that changes or removes a term a consumer uses is announced before it is published, with the crosswalk |
| Staging and quarantine | stage-feedback, promote-feedback | Feedback stays out of the live vector partition for one release cycle and can be rolled back |

- **Released knowledge only.** Outbound services read released graphs and live vector partitions, never drafts, staging partitions or unreviewed mappings.
- **Filter first, tailor second, everywhere.** Entitlements are applied before persona tailoring in search, context assembly, conversational AI and the APIs, not only in search-gateway.
- **No silent learning.** No processing service writes a term, mapping, shape or rule to the SKL. Learned changes leave processing only as change requests.
- **Answers carry their trace.** Every answer, search result and tool result carries the correlation identifier and the IRIs and release versions it used, so an issue downstream can be traced to the artifact and source upstream.

## Service Levels and Traceability

Each service family is held to a service level objective from Section 3.6 or the AI circuit breaker. Answer traceability is 100%: an answer that cannot cite governed terms is not returned as an

answer; generate-answer returns the search results it found or a clarifying question, and records a coverage gap for the learning services.

| Tier | Service family | Service level objective | Measured from |
|---|---|---|---|
| Inbound | Ingestion (Section 2.6) | New or changed schemas mapped within one business day | Inbound events to the first completed map step |
| Processing | Mapping, inferencing, learning | 99.5% of pipeline runs complete without manual intervention | Completed processing events versus escalations |
| Processing | Mapping | Mapping coverage 90% for in-scope domains; mapping accuracy 80% or higher | Mapping registry; review outcomes of propose-mapping |
| Processing | Release validation | Competency question pass rate 100% before release | validate-release reports |
| Outbound | Search and APIs | 99.9% availability; 95% of queries under 2 seconds | search-gateway, serve-graph, serve-tools telemetry |
| Outbound | Search | Search success 85% or higher | Search feedback events |
| Outbound | Conversational AI | Answer traceability 100% | audit-answer log |
| Outbound | Contextual relevancy | Dr within its persona-weighted tolerance before answering; otherwise a clarifying question | score-relevancy log |
| Outbound | Publishing | Releases published on schedule with release notes | Release events against the release calendar |
| All | Decision evaluation | No HALT or FREEZE verdict takes effect; Ds, Db and Dr within consequence-weighted tolerances; Constraint Evasion Index below 0.25 per mechanism; System Robustness Index between 0.70 and 0.85; MTBH tracked with statistical process control | Decision events |

Every event carries the correlation identifier and trace context, and every completed load, release, answer and decision produces a provenance record, which extends the Section 3.6 chain across all three tiers: source change → inbound request → asset graph → processing run → release → outbound request → decision → answer or result → feedback → change request → next release.

Related documents: AI Circuit Breaker: Design Specification (Subsystems 1–7); Operational Framework: AI Circuit Breaker and Trustworthiness Engineering; Sales Advisor Circuit Breaker architecture (plm.enablingvalue.com).

# Design

Knowledge engineering at the Company follows five design principles. A small **core ontology**, built on Basic Formal Ontology from primitive terms, gives every domain the same foundation; a **unifying ontology** makes the process step the anchor for every fact; **domain ontologies** add detail inside a domain; **operational areas** group domains and hold what the domains of an area share; and the **enterprise service catalog** is the only place where content from different areas is combined. OWL 2 is used in a deliberately simple way: deep subclass hierarchies carry the meaning, so little inference is needed at run time.

## Core Ontology

The core domain ontology (https://www.insperity.com/ontology/core/) is small, stable, and owned by Architecture. Every one of its classes is a subclass of a BFO 2020 class (ISO/IEC 21838-2), and every relation is a subproperty of a BFO, RO, or IAO relation, an inverse of one, or a property chain (Section 3.10). Asset is the genus: product, service, offering, and platform are assets assembled from other assets, and every asset has the same five-stage lifecycle (Discover, Define, Design, Develop, Deliver). The core also holds the Jobs model (job, job step, desired and delivered outcome; Section 3.13), and it imports the core taxonomies, whose concepts reach their classes through core:formalizedBy. Core vocabularies hold primitives (Section 3.7). The core never imports domain terms; domains extend the core, never the reverse.

## Unifying Ontology: the Process Step as First-Class Citizen

The unifying ontology makes the **process step** the anchor for every other concept. Instead of modeling data around tables or systems, each fact is attached to the step in which it is created, used, or governed:

| Relationship | Meaning | Example |
|---|---|---|
| performedBy | The party or role that carries out the step | Approve payroll run is performed by Payroll Manager |
| consumes / produces | Assets the step reads or creates | Calculate gross pay consumes Timesheet, produces Pay Statement |
| triggers / triggeredBy | Events that start or result from the step | Employee termination triggers Final pay calculation |
| governedBy | Rules, constraints, and legal norms that apply to the step | Final pay calculation is governed by State final pay law |
| measuredBy | Metrics that evaluate the step | Payroll run is measured by On-time payment rate |
| precedes / follows | Order of steps within a process | Validate timesheet precedes Calculate gross pay |

Because every system, document, rule, and metric is linked to a process step, the graph can answer impact questions directly: what breaks if this rule changes, which reports use this field, which steps a regulation touches.

## Domain Specificity: Intra-Domain Edges and Properties

A domain ontology (for example, Payroll, Benefits, or Human Resources Compliance) specializes core classes and defines the relationships that stay **inside** the domain. Domain subject-matter experts own these definitions.

- **Subclasses:** Payroll Run is a subclass of Process Step; Pay Statement is a subclass of Asset.
- **Intra-domain properties:** relationships whose domain and range are both in the same domain, such as Payroll Run hasPayPeriod Pay Period.
- **Domain vocabulary and taxonomy:** preferred terms, synonyms, and category hierarchies for the domain, linked to the ontology classes they describe.
- **Imports:** a domain ontology imports the core ontology and its area container by their ontology IRIs, so tools resolve them, and pins the versions it was built against with dcterms:requires (for example …/ontology/core/0.1.0/). A core release reaches a domain only when the domain is rebuilt against it and re-released.
- **Specialize, do not redefine:** a domain concept that narrows a core or container concept links to it with skos:broadMatch; the core concept is never copied into the domain.

## Operational Areas, Containers, and Insulation

Domains are grouped into four operational areas. Each area has a **container**, a set of assets named after the area (for example the Commercial Operations Vocabulary, Taxonomy, and Ontology), which holds the primitives, concepts, and classes that two or more of its domains use. Insulation keeps one domain's change from reaching another: classes, relations, concepts (skos:broader, skos:broadMatch, skos:exactMatch, core:formalizedBy), imports, and instances refer only to:

- the core;
- the domain's own operational area (its container);
- its parent domains (an L2 domain may refer to its L1);
- itself; and
- the APQC reference registry (Section 3.12).

Primitives are the exception: each primitive has one owner, the domain or area whose subject it is, and any domain in any area may build its concepts from it with dcterms:subject or core:hasPrimaryTerm, citing it and never copying it. The builders, the load script, and the Service Assembly page enforce these rules, and a reference to a sibling domain fails the build.

| Operational area | Domains |
|---|---|
| Commercial Operations | Sales (with Sales.Partner & Channel at L2), Marketing, Pricing, Customer Care |
| HR Service Operations | The external service domains of the HR Services & PEO Enterprise Business Model: Workforce Administration & Payroll; Employee Benefits & Retirement Sponsorship; Government Compliance & Employer Risk Management; Talent Management & Organizational Development; Client Relationship & Business Development; HR Technology Platform & Employee Experience; Workforce Development, Coaching & Performance Measurement; Organizational Values, Culture & Ethics Stewardship |
| Business Operations | Finance, Accounting, Human Resources, Billing & Accounts Receivable, Compliance, Legal, Security |
| Technical Operations | Business Systems Services, AI Strategy & Enterprise Analytic Services, Platform |

| Operational area | Domains |
|---|---|
| | Services, Data Engineering Services, Knowledge Engineering Services, Cybersecurity Services |

### Cross-Area Composition: the Enterprise Service Catalog

Real work crosses areas: a benefits enrollment changes a payroll deduction, and a sale creates an onboarding. These connections are not modeled as edges between domains of different areas. Every area publishes its services to the enterprise service catalog as interfaces, and a domain consumes another area's service only through its published catalog entry, never by referring to that area's classes, processes, or resources. Classes and instances from different areas meet only in enterprise services, which are composed from core terms and the domain terms they need. There is no enterprise-level vocabulary: core terms are the DNA, domain terms are the molecules, and enterprise services are composed from both.

- **Within an area:** what several domains share moves up to the container (for example, census moved from Pricing to the Commercial Operations container because Sales collects it and Pricing underwrites from it).
- **Across areas:** the catalog entry is the interface; the service that publishes it is owned by one area, and changes to it need that area's owner and the owners of the areas that consume it (Section 3.3, Human in the Loop).
- **Concept alignment:** where two domains use similar terms, each maps its term to the core or its container through a reviewed SKOS mapping (Section 3.5); domains never map directly to a domain in another area.
- **Replaces the operational module:** v1.9 placed inter-domain properties in one jointly owned operational module. That module is retired; "operational" now names the areas.

## Use of OWL 2: Maximize Subclasses, Minimize Inferencing

Meaning is carried by an explicit, deep subclass hierarchy. Properties, constraints, and documentation are attached as high in the hierarchy as they are true, so every subclass inherits them. This keeps the ontology predictable, fast to query, and easy for people to read, and it avoids reasoning that is expensive or hard to explain.

| Prefer | Avoid | Why |
|---|---|---|
| Asserted rdfs:subClassOf hierarchies | Defined classes (owl:equivalentClass with complex restrictions) | Asserted classification is transparent and needs no reasoner to compute |
| Properties with simple domain and range | Long property chains and inverse-functional tricks | Simple properties are inherited cleanly and query quickly |
| SHACL shapes for data constraints | OWL restrictions used as data validation | OWL is open-world; SHACL checks data the way the business expects |
| OWL 2 RL- or EL-friendly constructs | Full OWL 2 DL expressivity and SWRL rules | Keeps reasoning tractable inside GraphDB rulesets |
| Materialized hierarchy at publish time | Run-time inference for every query | Answers stay fast and repeatable |

# Standards

All knowledge artifacts follow one set of naming and identifier conventions. Every identifier lives under the base URL https://www.insperity.com, and names are spelled out in full: acronyms are not used in identifiers or preferred labels. Version 2.0 makes one identifier rule for classes, taxonomy concepts, and vocabulary terms: their primitives, in lower case, joined by hyphens.

## Naming Conventions

| Artifact | Element | Convention | Example |
|---|---|---|---|
| Vocabulary | Term (primitive) | Lower-case words joined by hyphens; preferred label in sentence case; synonyms and acronyms as alternative labels | pay-period, label "Pay period" (alternative: "Payroll cycle") |
| Taxonomy | Concept | Its primitives in lower case, joined by hyphens; preferred label a spaced phrase in sentence case; codes in skos:notation | wages-and-hours, label "Wages and hours" |
| Ontology | Class | Singular noun phrase in lower case, words joined by hyphens | process-step, payroll-run, worksite-employee |
| Ontology | Object property | lowerCamelCase verb phrase | performedBy, hasStep, deliveryOf |
| Ontology | Data property | lowerCamelCase noun phrase with has- or is- prefix | hasEffectiveDate, isExempt |
| Ontology | Controlled value | UpperCamelCase individual of a core value set (kept for stability) | core:Draft, core:FullForm |
| Instance data | Individual | Class segment and stable identifier, in lower case joined by hyphens (Instance Identifiers, below) | …/resource/domain-sales/service/proposal-and-quote |
| Shapes | Shape | Identifier of the target class plus -shape | process-step-shape, payroll-run-shape |

**Why hyphens:** a taxonomy concept or class built from several primitives is a phrase, not a new word. Concatenating primitives (ProcessStep, ServiceDelivery) coins words that search and display do not recognize; joining them with hyphens (process-step) keeps each primitive visible. The rule applies to single-primitive identifiers too, so every identifier has the same shape. Properties keep lowerCamelCase because they are verbs and nouns of the modeling language, not business terms.

## Asset Titles

An asset's title is a first qualifier followed by a second qualifier.

- **First qualifier:** Core; Domain L1[.L2[.L3]] (levels joined by dots, written with the domain names); or the name of the operational area for its container.
- **Second qualifier:** Vocabulary, Taxonomy, Ontology, or Shapes.

- **Examples:** "Core Process Vocabulary", "Domain Sales Taxonomy", "Domain Sales.Partner & Channel Ontology", "Commercial Operations Ontology", "Domain Finance.Capital Budgeting & Investment Analysis Vocabulary".
- **Acronyms in titles:** spelled out in the preferred label, with the short form as an alternative label: "Human Resources Service Operations Ontology" (alternative: "HR Service Operations Ontology").
- **HR Service Operations levels:** L1 is the business model's domain, L2 its bounded context, L3 its domain service.

## Avoiding Acronyms

- Identifiers and preferred labels use full words: professional-employer-organization, not PEO; family-and-medical-leave-act, not FMLA.
- Acronyms are recorded only as alternative labels (skos:altLabel), so search still finds them.
- Appendix F lists approved expansions, and the ingestion configuration holds the same list (acronyms.approved and acronyms.expansions); a new acronym is added to both before it is used in any label.
- In a preferred label, "&" is written "and", and in an identifier it becomes the word and.

## URI Conventions

URIs are persistent, readable, and never reused. Slash-based namespaces are used so each term can be resolved on its own. The segment after the artifact kind names the scope: a core facet, an operational area, a domain, or a reference set.

| Scope | Pattern | Example |
|---|---|---|
| Core ontology | https://www.insperity.com/ontology/core/{term} | https://www.insperity.com/ontology/core/process-step |
| Core vocabulary or taxonomy (one per facet) | https://www.insperity.com/{vocabulary\|taxonomy}/core-{facet}/{term} | https://www.insperity.com/taxonomy/core-process/approval-step |
| Area container | https://www.insperity.com/{kind}/{area}/{term} | https://www.insperity.com/ontology/commercial-operations/worksite-employee |
| Domain (L1) | https://www.insperity.com/{kind}/domain-{L1}/{term} | https://www.insperity.com/ontology/domain-sales/selling-step |
| Nested domain (L2, L3) | https://www.insperity.com/{kind}/domain-{L1}.{L2}[.{L3}]/{term} | https://www.insperity.com/taxonomy/domain-sales.partner-and-channel/referral |
| Shapes | https://www.insperity.com/shapes/{scope}/{shape} | https://www.insperity.com/shapes/core/process-step-shape |
| Reference registry | https://www.insperity.com/taxonomy/apqc-pcf/{element} | https://www.insperity.com/taxonomy/apqc-pcf/ |
| Mapping (bridge) | https://www.insperity.com/mapping/{scope}/{term} | https://www.insperity.com/mapping/domain-sales/benefits-plan-comparison |
| Instance data | https://www.insperity.com/resource/{scope}/{class}/{id} | https://www.insperity.com/resource/domain-sales/role/sales-representative |
| Version | https://www.insperity.com/{kind}/{scope}/{version}/ | https://www.insperity.com/ontology/domain-sales/0.4.0/ |

| Scope | Pattern | Example |
|---|---|---|
| Standard edition | https://www.insperity.com/standard/semantic-governance/{edition}/ | https://www.insperity.com/standard/semantic-governance/2.0/ |

- **{kind}** is vocabulary, taxonomy, ontology, or shapes. **{area}** is the area name in lower case joined by hyphens, with no "domain-" prefix (commercial-operations, hr-service-operations).
- **Nested domains** join their levels with dots in one segment, each level in lower case joined by hyphens; "&" is written "and". The Part III checks accept the dots.
- **Prefixes:** {scope}voc for a vocabulary, {scope}tax for a taxonomy, and the scope alone for an ontology (salesvoc:, salestax:, sales:; commercialopsvoc:); core facets use the facet (processvoc:, processtax:) and the core ontology core:; shapes add shp (coreshp:).

```
@prefix core:  <https://www.insperity.com/ontology/core/> .
@prefix sales: <https://www.insperity.com/ontology/domain-sales/> .

sales:selling-step a owl:Class ;
    rdfs:subClassOf core:process-step ;
    rdfs:label "Selling step"@en ;
    skos:definition "A process step of a selling process that advances an
                     opportunity from one selling stage to the next."@en .
```

## Named Graphs

The named graph of a knowledge artifact is the asset IRI itself: the Domain Sales Ontology is held in the graph https://www.insperity.com/ontology/domain-sales/. Every loader, editor, and check relies on this, and a deployment replaces exactly that graph (Section 3.3). Other graphs belong to reserved families:

| Graph family | Pattern | Holds |
|---|---|---|
| Knowledge artifact | The asset IRI | One vocabulary, taxonomy, ontology, or shapes file |
| Change log | https://www.insperity.com/graph/change-log | Append-only change records (Section 3.3) |
| Deployment log | https://www.insperity.com/graph/deployment-log | Deployment records, one log per environment |
| Assembly component | https://www.insperity.com/resource/{scope}/{tier}/{id}/ | Process, service, persona, value-map, and manifest graphs (Section 3.12) |
| Records and parties | https://www.insperity.com/resource/{area}/record-graph/{name}/ and …/party-graph/{name}/ | Records and parties shared by the domains of an area |
| Customer 360 | https://www.insperity.com/graph/customer-360/… | Persona research, study results, reviews (one graph per reviewer and per decision), demand (Section 3.13) |
| Reference ontologies | Their own IRIs (for example http://purl.obolibrary.org/obo/bfo.owl) | BFO 2020, RO Core, and the RO extract, never ingested |

## Instance Identifiers

**Rule (proposed, decision pending):** an individual is minted as …/resource/{scope}/{class}/{stable-id}, with the class segment and the identifier in lower case joined by hyphens, and the identifier taken from a stable business key, never a row number or a name that can change. Assembly components already follow it (…/resource/domain-sales/service/proposal-and-quote). Two established patterns are kept for stability and recorded as exceptions:

- **Identifier-coded instances** whose codes are the business key, such as Customer 360 outcomes (ob:Outcome-O001) and jobs (ob:Job-J01).
- **Change records** at …/resource/change-log/ChangeRecord/{timestamp}-{tool}, and the publisher at …/resource/core/Organization/insperity, which every asset header cites.

Until the rule is decided, new instance data follows it and the exceptions are not extended.

# Change Control

Every change to a knowledge artifact is recorded as a change record in a dedicated change log in the same knowledge graph, so history can be queried and joined to the artifacts it describes. Each change is traceable to a requirement, tested before release, and published with a version. The target is that releases are built once from source control and promoted unchanged through development, test, stage, and production, so no environment is ever edited by hand; today there is one environment, and all edits are made in GraphDB through the workbench pages, which export the Turtle and TriG files as releases of the graph.

## Versioning

- **Semantic versioning:** MAJOR for changes that break consumers (removing or redefining a class or property), MINOR for additions, PATCH for label and documentation fixes.
- **Diff-based classification:** a schema-diff check in the build pipeline determines the required version bump, and the build fails if the declared version does not match.
- **Version identifiers:** each release carries owl:versionIRI and owl:priorVersion, and consumers declare the version range they depend on.
- **Version in identifiers:** only ontology and vocabulary releases receive versioned IRIs (owl:versionIRI), because consumers must be able to import or cite an exact release. Class, property, and concept IRIs never contain a version: their meaning is stable across releases, and a versioned term IRI would break every mapping, stored fact, and link at each release. When a term's meaning changes, it is deprecated and replaced by a new term.
- **Before 1.0.0:** every asset starts at 0.1.0 as Draft. Until 1.0.0, a change that would be MAJOR (removing terms, new identifiers) bumps MINOR; additions bump MINOR; edits bump PATCH. From 1.0.0 the rules above apply as written.
- **Bump by tool:** Conformance Review bumps PATCH for edits and MINOR (before 1.0.0) or MAJOR (after) for removals and new identifiers; a metadata save bumps PATCH; builders set the version from their specification and record it in skos:changeNote.

## Change Records and the Change Log

Each change set is recorded once, by the tool that makes it, as a PROV-O activity in the change log. The record says what changed and why, who made it, what it used and generated, and, as required extensions, who approved it and which requirement it implements. Three placements for these records were considered:

| Option | Strengths | Weaknesses | Decision |
|---|---|---|---|
| Separate RDF store | Isolates volume and retention; separate access control | Every lineage question needs a cross-store join; two stores to synchronize | Only for high-volume run-time events (Section 3.6) |
| Same store, dedicated named graphs | One SPARQL query joins a change to the artifacts it touched; append-only; simple reporting | Needs graph naming and access discipline | Adopted for design-time change records |
| Extensions on | Current state is easy to read | History accumulates on every | Limited to a light |

| Option | Strengths | Weaknesses | Decision |
|---|---|---|---|
| existing elements | | term; current and past values mix; records become editable | pointer on each element |

### Change Log Structure

- **Append-only graph:** change records are resources of their own in https://www.insperity.com/graph/change-log. A record is never edited; a correction is a new record that references the one it corrects. Reloading a repository saves the log first and puts it back afterwards.
- **One record per change set:** a record describes one reviewed unit of work, such as an ingestion run, a Conformance Review save, an editor save, or a build. Each edit inside it is a part (dcterms:hasPart) with its own label, comment, subject term, and type.
- **Written by every tool:** Ingestion, Conformance Review, the Model Editors, the builders, and the migration scripts write records in the same profile; a change made outside the workbench gets a record written afterwards, which says so.
- **On disk and in the graph:** a record is written as change-record-{timestamp}.ttl and .trig next to the files it describes and loaded into the change log; saves made only in GraphDB are exported with the next release. The Operations Dashboard lists records found in only one of the two places.
- **Links, not copies:** a record points to the artifacts it used, generated, and invalidated by their stable IRIs; generated artifacts point back with prov:wasGeneratedBy and prov:wasDerivedFrom.
- **A light pointer on each element:** artifacts carry dcterms:modified, owl:versionInfo, and skos:changeNote. The full history stays in the change log.
- **Automatic tracking is a safety net only:** triple-level history kept by the triple store does not replace governed change records, because it cannot record why a change was made or who approved it.

### Change Record Profile

| Field | Property | Status |
|---|---|---|
| Record type | rdf:type prov:Activity, dcterms:ProvenanceStatement | Required, written |
| Identifier | https://www.insperity.com/resource/change-log/ ChangeRecord/{timestamp}-{tool} | Required, written |
| Label and description | rdfs:label, dcterms:description | Required, written |
| Who made it | dcterms:creator; dcterms:contributor for assistants and agents | Required, written |
| When | prov:startedAtTime, prov:endedAtTime (or dcterms:date) | Required, written; endedAtTime missing on some records |
| How | dcterms:source (the tool or script) | Required, written |
| Inputs and outputs | prov:used, prov:generated, prov:invalidated | Required, written |
| Edits | dcterms:hasPart → prov:Entity with rdfs:label, rdfs:comment, dcterms:subject, dcterms:type | Required for edits, written |

| Field | Property | Status |
|---|---|---|
| Approver | core:approvedBy | Required extension, not yet written |
| Requirement | core:implementsFeature (a feature or a Customer 360 demand request) | Required extension, not yet written |
| Version bump | core:versionBump (MAJOR, MINOR, PATCH) | Required extension, not yet written |
| Change type | core:changeType (KGCL class), core:changeCommand | Recommended extension |
| Release | core:releasedIn | Required once releases are packaged |

The required extensions are the governance fields v1.9 specified. Until every tool writes them, the Operations Dashboard reports their absence as process policy failures P03 and P04 (Section 3.4), and the corresponding actions remain open.

```
@prefix prov:   <http://www.w3.org/ns/prov#> .
@prefix dct:    <http://purl.org/dc/terms/> .
@prefix cr:     <https://www.insperity.com/resource/change-log/ChangeRecord/> .
@prefix person: <https://www.insperity.com/resource/core/person/> .
@prefix req:    <https://www.insperity.com/resource/core/requirement/> .

GRAPH <https://www.insperity.com/graph/change-log> {
  cr:2026-10-20T121500Z-core-ontology a prov:Activity , dct:ProvenanceStatement ;
    rdfs:label         "Core domain ontology imports the core taxonomies"@en ;
    dct:description    "The core domain ontology imports the 23 core taxonomies."@en ;
    dct:creator        "Jeffrey Wallk" ;
    dct:contributor    "Claude (AI assistant), at the direction of Jeffrey Wallk" ;
    dct:source         "ontology-core.source.ttl" ;
    prov:startedAtTime "2026-10-20T12:15:00Z"^^xsd:dateTime ;
    prov:endedAtTime   "2026-10-20T12:15:00Z"^^xsd:dateTime ;
    prov:used          <https://www.insperity.com/taxonomy/core-process/> ,
                       <https://www.insperity.com/ontology/core/> ;
    prov:generated     <https://www.insperity.com/ontology/core/> ;
    # required extensions
    core:approvedBy        person:architecture-owner ;
    core:implementsFeature req:core-taxonomy-imports ;
    core:versionBump       "PATCH" .

  <https://www.insperity.com/ontology/core/> prov:wasGeneratedBy cr:2026-10-20T121500Z-core-
ontology .
}
```

### Reporting, Traceability, and Lineage

- **Traceability and lineage** are answered in the graph with SPARQL: from a requirement to its changes, releases, and deployments, or from a term back to every change and approval that shaped it.
- **Operational reporting** (change volume, cycle time, approval lead time, defects per release) is fed on a schedule from the change log to the enterprise analytics warehouse, so dashboards use the standard business intelligence tools.

- **Explicit statements only:** change review and lineage queries read authored statements, never inferred ones (Section 3.9).
- **Operations Dashboard:** the workbench reads the change log (files and graph), joins it to the current inventory of models and their conformance, and shows activity by tool, the causes and effects of each change set (prov:used, prov:generated, prov:wasDerivedFrom), the change set that last produced an asset with findings, domain growth, policy compliance, and the actions that close each gap (Section 3.6).

## Lifecycle Management

Artifacts move through explicit states: **Draft** → **Candidate** → **Released** → **Deprecated** → **Retired**. Released versions are immutable. Deprecated versions carry a sunset date, and terms are deprecated rather than deleted so history is kept. These states run inside the ten-stage governance lifecycle in Section 3.4, which refines the six Semantic Knowledge Lifecycle stages used elsewhere in this document.

Lifecycle states map to environments: **Draft** artifacts exist only in source control and development; **Candidate** releases are deployed to test and stage; **Released** versions run in production.

### Candidate Entry Criteria (proposed, decision pending)

An asset moves from Draft to Candidate when all of the following hold, and the Operations Dashboard reports each asset's readiness against them:

1. No conformance errors under the Part III checks, and every warning dispositioned (Section 3.4).
2. Mandatory metadata complete (Section 3.11), and dcterms:conformsTo names the current edition of this guide.
3. Its shapes executed against instance data with no violations.
4. The open questions recorded against it closed or deferred with a rationale.
5. A subject-matter expert review recorded, and the change record names the approver (core:approvedBy).
6. Exactly one current copy on disk and in the graph, at the same version.

## Testing and Impact Assessment

- **Competency questions:** each requirement's competency questions run as SPARQL tests and must return the expected answers.
- **Consistency and quality:** a reasoner checks logical consistency, and SHACL shapes check data quality and naming conventions.
- **Impact analysis:** dependency queries list every mapping, rule, service, report, and agent that uses the changed terms, using the process-step links from Section 3.1.
- **Consumer compatibility:** the build checks every consumer's declared version range against the new release.

## Release Pipeline and Environment Promotion

Knowledge artifacts are released the way software is: from version-controlled source, built once into a fixed package, and promoted through environments by an automated pipeline. Changes are never

copied or re-applied from one environment to another, which removes replication and drift problems at the source.

### Principles

- **Source control is the source of truth.** Ontologies, taxonomies, vocabularies, SHACL shapes, rules, persona definitions, and change records are kept as Turtle and KGCL files in source control. Every change enters through a reviewed pull request.
- **Build once.** A tagged commit is built into one immutable release package: a TriG file with one named graph per artifact, the version IRIs, the change records, and a manifest with a checksum for each graph.
- **Promote the same package.** Development, test, stage, and production receive the identical package; only the configuration (endpoint, credentials, data sources) differs.
- **Replace, do not patch.** A deployment makes each target graph equal to the release ("make these graphs equal release 2.4.0"), never "apply these forty changes". Deployments can be repeated safely, and a failed deployment is simply rerun.
- **Same IRIs everywhere.** Identifiers are identical in every environment; an IRI names a thing, not a server. Only the endpoint an application connects to changes.
- **KGCL is for review, not transport.** Change commands explain and classify changes for reviewers and release notes; they are not the mechanism that moves changes between environments.

### Pipeline Stages

| Stage | Environment | What happens | Gate to proceed |
|---|---|---|---|
| 1. Author | Development | Knowledge engineers change artifacts in the workbench, export to source control, and open a pull request with its change record | Pull request complete; change record linked to an approved feature |
| 2. Build | Pipeline | Validate syntax, naming, and SHACL; run competency questions; classify the version bump; assemble the release package | All automated checks pass; declared version matches the diff |
| 3. Test | Test | Deploy the package; run consumer contract tests, impact queries, and search and AI evaluation sets on synthetic or masked data | No failed tests; impact assessment reviewed |
| 4. Stage | Stage | Deploy the identical package against production-like data volumes; rehearse the rollback | Human approval by the owners in the Human in the Loop table |
| 5. Release | Production | Deploy the identical package; switch over; publish; send release notes | Post-deployment checks pass; deployment record written |

### What Is Promoted and What Is Not

| Content | Promoted? | How each environment gets it |
|---|---|---|
| Ontologies, taxonomies, vocabularies, shapes, rules, personas | Yes | From the release package |
| Change records | Yes | In the release package; identical in every environment |
| Instance data | No | Loaded from each environment's own sources: synthetic or masked data in development and test, systems of record in production |

| Content | Promoted? | How each environment gets it |
|---|---|---|
| Inferred statements | No | Recomputed by each environment's ruleset after loading; never copied |
| Deployment records | No | Written by the pipeline in each environment |

### Current Practice: Stage 1 of the Pipeline

Today there is one environment. Loading is done by the workbench's Load into GraphDB, which replaces each asset's named graph and appends the change records, and by the load scripts (graphdb-load.command for the domains and graphdb-rebuild.command for Baseline Core v1.0), which back up the repository, replace the graphs in dependency order, put the change log back, and check lineage. These scripts are stage 1 of the target pipeline below. Until the pipeline exists, they are to write a deployment record for each load (policy P05).

### Deployment Records

History is split into two kinds. **Change records** say what changed and why; they are authored once and travel with the release. **Deployment records** say what arrived where and whether its checks passed; each environment writes its own in a local deployment log graph. Lineage then runs from requirement to feature, change record, release, and deployment, and a single query answers what is running in production and why.

```
@prefix deploy: <https://www.insperity.com/deployment/> .

GRAPH <https://www.insperity.com/graph/deployment-log> {
  deploy:stage-2026-10-13-01 a core:deployment-record, prov:Activity ;
    core:deployedRelease   finance:2.4.0 ;
    core:targetEnvironment core:StageEnvironment ;
    core:packageChecksum   "sha256:9f2c…41ab" ;
    core:validationResult  core:Passed ;
    core:pipelineRun       <https://pipeline.example/knowledge/runs/1187> ;
    prov:startedAtTime     "2026-10-13T14:05:00-05:00"^^xsd:dateTime ;
    prov:endedAtTime       "2026-10-13T14:10:00-05:00"^^xsd:dateTime .
}
```

### Safe Loading and Rollback

1. Load the release into new graphs, or a standby repository, alongside the live content.
2. Run SHACL validation and smoke-test queries against the newly loaded content.
3. Switch over in one step: replace the live graphs in a single transaction, or point the service to the standby repository. If any check fails, the live version is untouched.
4. Recompute inferred statements, refresh search indexes and domain vector partitions, and write the deployment record.
5. Roll back by redeploying the previous release package; no reverse changes are written by hand.

### Preventing Drift

- **No direct edits:** people have read-only access to stage and production knowledge graphs; only the pipeline writes to them.

- **Urgent fixes use the same path:** a hotfix follows the same pipeline on an expedited approval track, with its own change record and release.
- **Nightly drift check:** a scheduled job compares each environment's graph checksums with its release manifest and opens an issue on any mismatch (Section 3.6).

### Release Cadence

Changes are bundled into scheduled releases (weekly for MINOR and PATCH; planned and announced for MAJOR). Each release has release notes generated from its change records, a rollback plan, and a sign-off record.

## Human in the Loop

| Change touches | Required approvals |
|---|---|
| A domain ontology, vocabulary, or taxonomy | Domain subject-matter expert, plus automated checks |
| The core ontology, vocabularies, or taxonomies | Architecture, plus every affected domain owner; a primitive is promoted to the core only by a deliberate core release |
| An area container | Every domain owner in the area, plus Architecture |
| An enterprise service catalog entry | The owner of the publishing area, the owners of the areas that consume it, and Architecture |
| A mapping (bridge) | Each connected owner and Architecture |
| The APQC reference registry | Architecture (read-only reference; changes only by a new APQC release and crosswalk) |
| Personas, jobs, and desired outcomes (Customer 360) | Business and compliance subject-matter experts in PEO Outcome Review; a final decision by a designated decider |
| Legal or regulatory norms | Legal or Compliance reviewer |
| Agent-proposed mappings, definitions, or extensions | Knowledge Engineer review and a recorded disposition before any change becomes authoritative |

## Publishing

After the production deployment succeeds, released artifacts are registered in the metadata registry (ISO/IEC 11179) and published at their persistent URIs, with human-readable pages and machine-readable downloads (Turtle, JSON-LD), a SPARQL endpoint, and the Semantic Modeling Workbench. Search indexes and domain vector partitions are refreshed from the same release.

## Communications

- **Release notes** to all subscribers of the affected artifacts, generated from change records.
- **Deprecation notices** with sunset dates and replacement terms, sent at least one release before retirement.
- **Change calendar** showing upcoming MAJOR releases and their impact assessments.

# Governance

Knowledge engineering is governed against Company policy and against the international standards catalogued in the **PEO Standards & Compliance Map** and its guide. No single ISO standard covers the whole knowledge lifecycle; ISO/IEC 5392:2024 (reference architecture for knowledge engineering) is the umbrella, and each lifecycle stage is guided by the standards below.

## Adherence to Policies

- **Company policies:** data classification, privacy, records retention, information security, and responsible use of AI apply to every knowledge artifact and service.
- **Quality gates:** SHACL release gates enforce naming, provenance, and verification rules; an artifact that fails a gate cannot be released.
- **Verification status:** every reference fact (for example, a law or wage rate) records whether it has been checked against a published source, and unverified facts are reported as warnings, not hidden.
- **Audit:** change records, approvals, and release sign-offs are kept in the graph and reviewed each quarter.

## Governance Lifecycle and Guiding Standards

| Stage | What happens | Guiding standards |
|---|---|---|
| 1. Scope and competency questions | Agree what the knowledge must answer, for whom, and who owns it | ISO/IEC 5392, ISO 30401, ISO/IEC 42001 |
| 2. Acquire knowledge and provenance | Collect authoritative sources and record where each fact came from | ISO/IEC 5392, ISO 30401, W3C PROV |
| 3. Define terms | Preferred terms, intensional definitions, term metadata | ISO 704, ISO 1087, ISO 12620-1, ISO 30042, ISO 26162-1, ISO 30400 |
| 4. Organize concepts | Hierarchies, associative relations, mappings to external vocabularies | ISO 25964-1 and -2, W3C SKOS |
| 5. Formalize the ontology | Classes, relations, and axioms aligned to a top-level ontology | ISO/IEC 21838-1, -2 (BFO), -5, ISO/IEC 5392, W3C OWL 2 |
| 6. Validate | Reasoner consistency, SHACL data quality, expert and legal review | ISO/IEC 5259, W3C SHACL |
| 7. Register and publish | Record each artifact, version, and status in a metadata registry | ISO/IEC 11179-3, -32, -35; ISO/IEC 19763-1, -3 |
| 8. Deploy as persistent or virtual graphs | Load into a store, or expose existing databases as a graph | ISO/IEC 39075 (GQL), ISO/IEC 9075-16 (SQL/PGQ), W3C SPARQL, W3C R2RML |
| 9. Operate, protect and measure | Access control, AI impact assessment, quality monitoring, reporting | ISO/IEC 27701, ISO/IEC 42001, ISO/IEC 42005, ISO/IEC 5259, ISO 30414 |
| 10. Change, version and retire | Track changes, issue versions, deprecate rather than delete, retain history | ISO 30300, ISO/IEC 11179-3, ISO 30401 |

## Six-Stage and Ten-Stage Lifecycles

The ten stages refine the six Semantic Knowledge Lifecycle stages used in Section 2.4: Ideation = stages 1–2; Development = stages 3–5; Testing = stage 6; Deployment = stages 7–8; Maintenance = stage 9; Retirement = stage 10.

## Engineering, Requirements, and Quality Standards

| Standard | Applies to |
|---|---|
| ISO/IEC/IEEE 29148 (requirements engineering) | Ontology requirements and competency questions (Appendix A.4) |
| ISO/IEC/IEEE 24765 (systems and software engineering vocabulary) | Engineering terminology in specifications |
| ISO/IEC/IEEE 42010 (architecture description) | Architecture views of the knowledge fabric and services |
| ISO 30401 (knowledge management systems) | The knowledge management system as a whole |
| ISO/IEC 42001 and ISO/IEC 42005 (AI management and impact assessment) | Agents, search, and AI services that use the graph |
| ISO/IEC 27701 (privacy information management) | Worker and client data exposed through the graph |

The Standards & Compliance Map records the edition and status of each standard (current, under review, draft, or withdrawn); it is the reference for which edition applies.

## Implemented Quality Gates

The quality gates in this section are implemented in the KE Workbench as follows.

| Gate | How it works | Where |
|---|---|---|
| Part III checks | Every file and graph is checked against the naming, URI, acronym, label, definition, taxonomy, metadata, and reference rules; findings are errors, warnings, or information | Ingestion, Conformance Review, Standards Conformance, builders |
| Load gate | Loading stops while a file has a blocking finding (terms outside the base URL, another organization's content, an invalid date; configured as loadBlockingRules); Load anyway is a recorded exception | Ingestion, Conformance Review |
| Dispositions | Every finding is fixed, accepted as is, marked not an issue, deferred, or escalated for a modeling decision, with a rationale; decisions are kept in an append-only file and replayed by later ingestions | Conformance Review |
| Build checks | Typing, classification, insulation, closure, lineage to BFO, primitives unique across core and domains, and shapes | build_core.py, build_domains.py, build_assembly.py, load scripts |
| Subject-matter expert review | Business and compliance reviewers record a verdict, wording, risk flags, and approved uses for each persona, job, and outcome; a decider records the final decision | PEO Outcome Review |
| Demand admission | A demand request is admitted only when it is traced, reviewed, evidenced, measurable, owned, and unflagged | Demand Gateway |

## Process Policies

Process policies apply to the change log itself. The Operations Dashboard measures each one on every snapshot and lists the actions to close each failure.

| Policy | Measured as | Standard |
|---|---|---|
| P01 Every change set has a change record | Assets no change record generated, revised, or used | 3.3 Change Log Structure |
| P02 Records name who made the change | Records with dcterms:creator | 3.3; W3C PROV |
| P03 Records name who approved the change | Records with core:approvedBy | 3.3 Human in the Loop |
| P04 Records trace to a requirement | Records with core:implementsFeature or a demand request | 3.3; A.4 |
| P05 Deployments are recorded | Deployment records in the deployment log | 3.3 Deployment Records |
| P06 Records are closed | Records with prov:endedAtTime | 3.3 |
| P07 Assets declare the current edition | Assets whose dcterms:conformsTo is the current edition of this guide | 3.11 |
| P08 Assets have one current copy | An asset IRI in more than one current file at different versions | 3.3 Release Pipeline |
| P09 Assets move through the lifecycle | Assets beyond Draft | 3.3 Lifecycle Management |

# Single Source of Truth

Business logic is often scattered across code, database triggers, rules-engine screens, and ontologies, and the same rule drifts apart in each place. The Single Source of Truth (SSOT) defines every rule, constraint, logic component, and calculation **once**, in a declarative specification under version control, and generates what each runtime needs from it.

## The Right Home for Each Kind of Logic

| Category | Governing question | Authored as | Runs in | Example |
|---|---|---|---|---|
| Constraints | What must always be true about the data? | Ontology terms and SHACL shapes, or JSON Schema | Graph store, API validation | A payroll run must have exactly one pay period |
| Rules | What decision follows from the current state? | DMN decision tables | Rules engine behind the Decision API | Overtime applies above 40 hours unless the employee is exempt |
| Logic | How is a multi-step process carried out? | Code, referencing SSOT terms and versions | Microservices | Orchestrating the inbound pipeline |
| Calculations | How is a value computed? | Declarative formula specification with test vectors | Generated code in services; documented in the graph | Gross pay = hours × rate + overtime premium |

## Governance Pipeline

- **Nothing is edited in production:** every change is committed to the SSOT repository and reaches production only through continuous integration.
- **Automated checks:** a reasoner checks ontology consistency, DMN tables are checked for gaps and overlaps, and calculations run against their test vectors.
- **Generated artifacts:** each merge generates SHACL/OWL files for the graph store, DMN files for the rules engine, and typed interfaces for microservices, so they cannot drift apart.
- **Decision API:** services call one Decision API, which resolves the policy version they declare and routes to the backend that owns the decision, including wrapped legacy engines during migration.

## Core and Domain Ownership

The repository is split into a core namespace and one namespace per domain, mirroring the core and domain ontologies in Section 3.1. Domains extend the core by composition and cannot redefine it. Ownership is enforced by review routing: domain changes need the domain owner's approval; core and mapping changes also need Architecture.

## Interfaces for Subject-Matter Experts

- **Constraints:** a data-dictionary form (field, type, required, allowed values), never raw SHACL or JSON Schema.
- **Rules:** DMN decision tables, edited as grids.
- **Logic and calculations:** owned by engineers; experts see a read-only trace of which services use each rule or constraint, and approve calculation test vectors.

## Choosing Storage

Structural constraints and taxonomies can be stored as versioned rows in a relational registry. A graph store is used where cross-domain semantic reasoning or process-step traceability is required, which is the case for the core, unifying, and area container ontologies and the service assemblies. SWRL is not used; inference-style business rules belong in DMN.

# Target Operating Model

The target operating model describes how knowledge engineering runs day to day: how knowledge artifacts and the services that use them (inbound, processing, outbound) are monitored, measured, traced, corrected, and improved.

## Traceability

Every link in the chain below is stored as triples, so any item can be traced forward to its effects or back to its origin:

```
Requirement → Feature → Change record → Release → Deployment (dev → test → stage → prod)
            → Service (inbound / processing / outbound) → Monitored outcome
            → Defect or issue → New requirement
```

- **Knowledge artifacts:** vocabularies, taxonomies, ontologies, mappings, shapes, rules, and calculations.
- **Services:** inbound ingestion and mapping (Section 1.4), processing and enrichment, and outbound search, conversational AI, and APIs.
- **As implemented:** change records link requirements (Customer 360 demand requests) to the change sets that implement them, and change sets to the assets they used and generated; the Operations Dashboard follows these links in both directions.

## Monitoring

- **Artifact health:** SHACL violations, unverified facts, orphan terms with no path to the core, and deprecated terms still in use.
- **Service health:** availability, latency, error rates, queue depth, and freshness of each inbound source.
- **AI and search behavior:** answer traceability, hallucination flags, low-confidence mappings, and zero-result searches.
- **Change and conformance:** change sets by day and tool, conformance findings by rule and domain, error density per hundred terms, records missing from the graph or from disk, and possible untracked changes (commits that changed model files with no change record).

## Measurement

| Measure | What it shows | Initial target |
|---|---|---|
| Mapping coverage | Share of inbound schema elements mapped to ontology terms | 90% for in-scope domains |
| Mapping accuracy | Share of agent-proposed mappings accepted on review | 80% or higher |
| Competency question pass rate | Requirement tests passing in each release | 100% before release |
| Change lead time | Approved requirement to released artifact | Two weeks or less for MINOR changes |

| Measure | What it shows | Initial target |
|---|---|---|
| Defect escape rate | Design-time defects found after release | Below 5% of changes |
| Search success | Searches returning a result the user accepts | 85% or higher |
| Answer traceability | AI answers that cite governed terms and mappings | 100% |

Targets are starting values to be calibrated during the first release.

## Human in the Loop

- **Design time:** approvals defined in Section 3.3 for every change to core, domain, mapping, and legal artifacts.
- **Run time:** review queues for low-confidence mappings, agent-proposed extensions, and AI answers flagged by users.
- **Escalation:** any item affecting legal norms or worker data is routed to Legal, Compliance, or Privacy.

## Defect Management (Design Time)

A **defect** is a fault in a knowledge artifact found before or after release: a wrong definition, a broken mapping, a missing constraint, or a failed competency question.

1. **Log** the defect against the artifact and version, with the failing test or review note.
2. **Triage** severity (blocking, major, minor) and assign the artifact owner.
3. **Fix** through a normal change record, tied to the defect and the original requirement.
4. **Verify** with a new or updated test so the defect cannot recur, then release.

## Issue Management (Run Time)

An **issue** is a problem observed in operation: a failed ingestion, a wrong search result, an agent error, or a breached service level.

1. **Detect** through monitoring or a user report, and record the affected service and request.
2. **Restore** service first (retry, roll back, or fall back to the previous release).
3. **Find the root cause** using traceability: which artifact version, rule, or mapping produced the result.
4. **Raise a defect** when the cause is a knowledge artifact, so design-time correction follows.

## Service Level Monitoring and Management

| Service | Service level objective | Measured by |
|---|---|---|
| Inbound ingestion | New or changed schemas mapped within one business day | Time from change event to published mapping |
| Processing and enrichment | 99.5% of pipeline runs complete without manual intervention | Pipeline run outcomes |
| Search and APIs | 99.9% availability; 95% of queries under 2 seconds | Service monitoring |

| Service | Service level objective | Measured by |
|---|---|---|
| Conversational AI | 100% of answers traceable to governed terms and mappings (Section 2.7) | Answer audit logs |
| Publishing | Releases published on schedule with release notes | Release calendar |

Breaches open an issue automatically, are reviewed in a monthly service review, and feed capacity and backlog planning.

## Recursive Learning

The operating model learns from its own use. Signals from AI agents, microservices, and search are collected, reviewed, and turned into improvements through the feedback controls in Section 2.1:

| Source | Signal | Improvement |
|---|---|---|
| AI agents | Low-confidence or rejected mappings; flagged answers | New mappings, synonyms, or constraints |
| Microservices | Validation failures, retries, and schema drift | Updated shapes, mappings, or inbound contracts |
| Search | Zero-result and abandoned queries; reformulations | New vocabulary terms and alternative labels |
| Users | Feedback on answers and search results | Prioritized requirements and competency questions |

Nothing learned this way becomes authoritative until it passes the same change control as any other change.

Conformance Review dispositions are the first implemented loop: fixes are replayed when the same term is ingested again, accepted findings are reported as information with their rationale, and the definitions reviewers approved or corrected are given to the definition writer as examples.

## Operations Dashboard and Continuous Improvement

The Operations Dashboard is the run-time view of this operating model for knowledge artifacts. A snapshot builder reads every change record, the current model files, the decisions and open questions, the release history, and the source control history; with GraphDB connected, the page merges the change log graph by record IRI. Each rebuild adds a line to a history file, which gives trends.

| View | Answers |
|---|---|
| Overview | How much changed, by which tool; how many findings, policies met, actions open, guide items open |
| Change log | What each change set did: description, creator, times, sources, used, generated, invalidated, and its parts |
| Causality and root cause | What caused a change set and what it led to; the history of an asset or term; the change set that introduced a finding; which tools defects enter through |
| Domains | Domain expansion over time by area; continuous improvement by domain (change sets, edits, review cycles, findings, density, trend) |

| View | Answers |
|---|---|
| Compliance | Findings by rule and domain; process policies P01–P09; the guide edition each asset declares |
| Standards guide | Where this guide and practice differ, and the proposed change |
| Actions | One list of actions by category (conformance, policy, guide, decision, lifecycle, portfolio), priority, scope, and status |
| Workbench | Workbench releases, commits, decisions, and open questions |

Root cause analysis follows the change log: a finding on an asset is traced to the change set that last generated the asset, then up the chain of change sets that produced its inputs. When one tool keeps producing findings, the fix belongs in the tool or its configuration, not in each artifact.

## Measured Agent Assurance (proposed)

Agents that answer from the knowledge layer are measured the way instruments are. The proposal, to be decided with the PEO Service Map expansion, is:

- **Reference question sets:** the questions personas ask while doing a job (Section 3.13) become the test sets for each agent and service.
- **Measured accuracy and groundedness with uncertainty:** each release of an agent records its results against the reference set as measurement records, with the sample size and confidence interval.
- **Thresholds that act:** a measured value below its threshold trips the circuit breaker (Sections 1.5 and 2.6) and opens an issue, so a degraded agent is withdrawn before users find it.

# Term Definition Standard

Every class, concept, and property is defined with the same set of fields, so that a term means the same thing to people, applications, and AI agents. The fields below match the class editor in the Semantic Modeling Workbench (implemented as the KE Workbench Model Editors: Annotations, OWL Axioms, SKOS Annotations, Referenced By, and Data Properties). Required fields must be complete before a term can move from Draft to Candidate (Section 3.3).

## Class Definition Fields

| Editor field | RDF / OWL property | Required | Rule |
|---|---|---|---|
| IRI | (the class identifier) | Yes | https://www.insperity.com/ontology/{scope}/{class}; lower-case primitives joined by hyphens, no acronyms, no hierarchy in the path |
| Label | rdfs:label, skos:prefLabel | Yes | One per language, tagged @en; sentence case; full words |
| Is defined by | rdfs:isDefinedBy | Yes | The IRI of the ontology that owns the term |
| Definition | skos:definition | Yes | One sentence, genus plus differentia (ISO 704): "A [parent] that [distinguishing feature]" |
| Alternate labels | skos:altLabel, skosxl:altLabel | As needed | Synonyms, acronyms, legacy names, persona terms (Section 3.8) |
| Hidden labels | skos:hiddenLabel | Optional | Misspellings and search variants; never displayed |
| Scope note | skos:scopeNote | Recommended | What is included and excluded; points to neighboring terms |
| Example | skos:example | Recommended | One or two real instances |
| Notation | skos:notation | Optional | Codes from source systems; the only place a code or acronym may stand alone |
| Sub-Class Of | rdfs:subClassOf (named class) | Yes | At least one named parent; the hierarchy carries the meaning (Section 3.1) |
| Restrictions | rdfs:subClassOf (owl:Restriction) | Optional | Existential (someValuesFrom) restrictions only, to document necessary links |
| Equivalent class | owl:equivalentClass | Avoid | Only with Architecture approval; defined classes force run-time inference |
| Disjoint with | owl:disjointWith | When true | Assert only where membership truly excludes; prefer at the highest level |
| Key properties | owl:hasKey | Optional | Data properties that uniquely identify an instance (for example, a document identifier) |
| All disjoint groups | owl:AllDisjointClasses | When true | For sibling sets that are mutually exclusive |
| Enumeration | owl:oneOf | Closed sets only | Only for fixed value sets such as a lifecycle status; never for open business classes |
| Disjoint union of | owl:disjointUnionOf | Avoid | Use subclasses plus AllDisjointClasses instead |
| Referenced by | (computed) | Review | Axioms that use the term; generated by the |

| Editor field | RDF / OWL property | Required | Rule |
|---|---|---|---|
| | | | workbench and checked in impact assessment |
| Data properties | owl:DatatypeProperty with this domain | As needed | Drawn from the attribute taxonomy (Section 3.10) |
| Governance | core:hasLifecycleStatus, owl:versionInfo, dcterms:created, dcterms:modified, dcterms:creator | Yes | Status, version, dates, and owner; set by change control (fields in Section 3.11) |

## Primitives and Polymorphic Terms

Vocabularies hold **primitives**: unique, irreducible terms. Taxonomy concepts and ontology classes are built from them. A concept built from more than one primitive is **polymorphic**: "Approval step" is built from approval and step.

- **Unique:** a primitive is unique across the core and every domain. A build rejects a duplicate label unless it is declared a homonym with the difference in sense stated.
- **Multiword lexical units:** a primitive may have several words when its meaning is not the sum of its words (circuit breaker, segregation of duties, life event, waiting period). Named statutes are multiword lexical units.
- **Adjectives:** modifier adjectives are not primitives; they become their noun primitive, kept as its alternative label (strategic → strategy, missing → completeness). The RACI terms and hybrid are exceptions.
- **Links:** every concept carries dcterms:subject to each primitive it is built from and core:hasPrimaryTerm to the head primitive. A concept that is itself a primitive has just that one.
- **Polymorphic terms are phrases:** labels are spaced phrases and identifiers join primitives with hyphens (Section 3.2); primitives are never concatenated into new words.
- **Where polymorphic classes live:** the core ontology keeps a polymorphic class only where it differs in BFO (a different BFO parent or its own axioms); other polymorphic terms live in the taxonomies, formalized by the class of their head primitive.
- **Owners and promotion:** each primitive has one owner, the core or the domain whose subject it is (Section 3.1). A general primitive the core lacks is added to the domain that first needs it, or to its area container if several do, and flagged as a promotion candidate; the core changes only through a deliberate core release.
- **Domain-specific terms stay out of the core:** revenue belongs to Domain Finance, not to the core.

The builders check these rules: every label word is covered by a primitive or reported, every concept has a primary term, and no primitive is defined twice.

## Worked Example: Documentation Standards

The example below completes the class editor entry for **Documentation Standards**. In the source entry most fields were empty and the identifier carried its hierarchy and an acronym in the path (sm:artifact:Develop:SE:Documentation_Standards). Under the Company standard, the hierarchy

moves into rdfs:subClassOf, "SE" is spelled out, and the identifier moves under the Company base URL.

| Field | Value |
|---|---|
| IRI | https://www.insperity.com/ontology/domain-platform-services/documentation-standards |
| Label | "Documentation standards"@en |
| Is defined by | https://www.insperity.com/ontology/domain-platform-services/ |
| Definition | "A software engineering artifact that specifies the required content, structure, format, review, and retention of the documentation produced during development."@en |
| Alternate labels | "Documentation guidelines"@en; "Documentation rules"@en; persona labels in Section 3.8 |
| Scope note | "Includes templates, style rules, and review checklists for documentation. Excludes coding standards and testing standards, which are separate classes."@en |
| Example | "Application programming interface documentation standard; release note template."@en |
| Sub-Class Of | Software Engineering Artifact (which is a subclass of Development Artifact, a subclass of Artifact) |
| Restrictions | appliesTo some Software Engineering Work Product; ownedBy some Role |
| Equivalent class | None (asserted hierarchy preferred) |
| Disjoint with | None asserted at this level; inherits Artifact disjoint with Process Step |
| Key properties | hasDocumentIdentifier |
| All disjoint groups | None |
| Enumeration | None (open class) |
| Disjoint union of | None |
| Referenced by | Software Engineering Process Step governedBy some Documentation Standards (computed) |
| Data properties | hasDocumentIdentifier, hasEffectiveDate, hasReviewCycleInMonths |
| Governance | Status Draft; version 0.1.0; owner Software Engineering Practice Lead |

The same entry in Turtle:

```
@prefix sm:      <https://www.insperity.com/ontology/domain-platform-services/> .
@prefix core:    <https://www.insperity.com/ontology/core/> .
@prefix owl:     <http://www.w3.org/2002/07/owl#> .
@prefix rdfs:    <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos:    <http://www.w3.org/2004/02/skos/core#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix xsd:     <http://www.w3.org/2001/XMLSchema#> .

sm:documentation-standards a owl:Class ;
    rdfs:label        "Documentation standards"@en ;
    skos:prefLabel    "Documentation standards"@en ;
    skos:altLabel     "Documentation guidelines"@en , "Documentation rules"@en ;
    skos:definition   "A software engineering artifact that specifies the required
                       content, structure, format, review, and retention of the
```

```
                      documentation produced during development."@en ;
    skos:scopeNote    "Excludes coding standards and testing standards."@en ;
    rdfs:isDefinedBy  <https://www.insperity.com/ontology/domain-platform-services/> ;
    rdfs:subClassOf   sm:software-engineering-artifact ,
        [ a owl:Restriction ; owl:onProperty sm:appliesTo ;
          owl:someValuesFrom sm:software-engineering-work-product ] ,
        [ a owl:Restriction ; owl:onProperty core:ownedBy ; owl:someValuesFrom core:role ] ;
    owl:hasKey        ( sm:hasDocumentIdentifier ) ;
    core:hasLifecycleStatus core:Draft ;
    owl:versionInfo   "0.1.0" ;
    dcterms:created   "2026-10-13"^^xsd:date .

sm:software-engineering-artifact rdfs:subClassOf sm:development-artifact .
sm:development-artifact         rdfs:subClassOf core:artifact .
```

Identifiers join lower-case words with hyphens (Section 3.2), and so do multi-word scope segments (domain-platform-services, commercial-operations).

# Labels and Personas

One concept is called different things by different people: a payroll specialist, a worksite employee, a developer, and an auditor rarely use the same words. Labels capture those words without creating duplicate concepts, and **personas** record who uses which words and what each group needs. Together they drive search, user-interface text, AI answers, and access decisions.

## Use of Alternate Labels

| Label type | Property | Use for | Rules |
|---|---|---|---|
| Preferred | skos:prefLabel | The single display name | Exactly one per language; unique within the concept scheme; full words, no acronyms |
| Alternate | skos:altLabel | Synonyms, acronyms, abbreviations, legacy system names, persona terms | Any number; must not repeat the concept's own preferred label; each has a recorded source |
| Hidden | skos:hiddenLabel | Misspellings and search variants | Used by search only; never shown to users or used in AI answers |
| Notation | skos:notation | Codes from systems of record | Typed with a datatype for the source system; not a label |

- **One concept, many labels:** if two terms mean the same thing, they become labels of one concept; if they differ in meaning, they become two concepts linked by a relation.
- **No collisions:** an alternate label that is the preferred label of another concept in the same scheme is rejected by validation, because it makes search ambiguous.
- **Acronyms:** every acronym in Appendix F is recorded as an alternate label of its spelled-out concept, so "FMLA" finds Family and Medical Leave Act.
- **Language tags:** every label carries a language tag (@en, @es), so labels can be served per language.
- **Provenance:** each alternate label records where it came from: search logs, a source system, a subject-matter expert, or a regulation.

## Associating Alternate Labels with Personas

A plain skos:altLabel is a text value, so nothing can be said about it, including which persona uses it. Where a label belongs to a persona, it is recorded as a **label resource** using the SKOS eXtension for Labels (SKOS-XL), and the label resource is linked to one or more personas. At publish time the plain skos:altLabel is also written, so tools that only understand SKOS still find the label.

```
@prefix persona: <https://www.insperity.com/vocabulary/core-persona/> .
@prefix label:   <https://www.insperity.com/vocabulary/domain-platform-services/label/> .
@prefix skosxl:  <http://www.w3.org/2008/05/skos-xl#> .

sm:documentation-standards
    skosxl:altLabel label:documentation-standards-software-developer ,
                    label:documentation-standards-compliance-auditor .
```

```
label:documentation-standards-software-developer a skosxl:Label ;
    skosxl:literalForm        "Docs guidelines"@en ;
    core:intendedForPersona   persona:software-developer ;
    dcterms:source            "Developer portal search logs" .

label:documentation-standards-compliance-auditor a skosxl:Label ;
    skosxl:literalForm        "Documentation controls"@en ;
    core:intendedForPersona   persona:compliance-auditor .

# written at publish time for SKOS-only consumers
sm:documentation-standards skos:altLabel "Docs guidelines"@en , "Documentation controls"@en .
```

- **Selection rule:** an application shows the preferred label unless the current persona has its own label for the concept, in which case it may show that label with the preferred label available on hover or in the definition.
- **Search rule:** all alternate labels are searchable by everyone; persona labels raise ranking for that persona only.

## Definition and Maintenance of Personas

A persona is a governed description of a group of users who share a role, goals, vocabulary, and information needs. Personas are individuals of the class core:persona, classified by the core persona taxonomy and labeled from the persona vocabulary (https://www.insperity.com/vocabulary/core-persona/). A persona describes a typical user; it is not a person and not a security role.

| Attribute | Property | Example (Payroll Specialist) |
|---|---|---|
| Name and definition | skos:prefLabel, skos:definition | "Payroll specialist": processes payroll for client worksites |
| Roles | core:correspondsToRole | Payroll Specialist role in the organization model |
| Process steps | core:performsStep | Validate timesheet; Calculate gross pay; Approve payroll run |
| Domains | core:hasDomainInterest | Payroll; Tax; Wages and hours |
| Expertise level | core:hasExpertiseLevel | Practitioner (from a controlled list: Novice, Practitioner, Expert) |
| Vocabulary | (persona labels, above) | "Gross-to-net", "Off-cycle run" |
| Information needs | core:hasInformationNeed | Deadlines, tax rule changes, exception reasons |
| Communication style | core:prefersReadingLevel, core:prefersTone | Concise, procedural, technical terms allowed |
| Channels | core:usesChannel | Payroll application, internal search, AI assistant |
| Owner and review | core:ownedBy, core:nextReviewDate | Payroll Operations lead; reviewed every six months |
| Status and version | core:hasLifecycleStatus, owl:versionInfo | Released, 1.2.0 |

- **Keep the set small:** a persona is added only when its vocabulary or needs differ enough to change what is shown; most organizations need ten to twenty.

- **Evidence-based:** personas are created and revised from evidence: search and assistant logs, support tickets, user research, and subject-matter experts.
- **Review cycle:** every persona is reviewed at least every six months and after organizational or product changes; unused labels are retired.
- **Change control:** persona changes follow Section 3.3 like any other knowledge artifact, with a version, an approver, and a change record.
- **Deprecate, never delete:** retired personas are marked deprecated and replaced, so logs and past AI answers remain explainable.
- **No protected attributes:** personas never encode age, gender, ethnicity, health, or other protected or sensitive characteristics; they describe work, not people.

## The Persona Model

There is one persona model, and persona instances are many. Personas are not typed by role: typing them by role bleeds into the job being performed and adds ambiguity. A persona is not mapped to a job title or position; where scenarios involve employees, persona-to-role correspondences are curated as instance mappings (core:correspondsToRole). Personas are assembled with the jobs they perform, the steps of those jobs, and the outcomes they want, which is the subject of Section 3.13. The labels-for-personas rules above are unchanged.

## Use of Personas for Tailoring, Prompt and Context Engineering, and Security

| Use | What the persona drives | Control |
|---|---|---|
| Tailoring | Labels shown, search ranking and query expansion, depth of definitions, examples chosen, reading level | Same concept and same facts for every persona; only presentation changes |
| Prompt engineering | Persona-specific prompt templates: role statement, tone, vocabulary, allowed tasks, answer format | Templates are versioned knowledge artifacts under change control |
| Context engineering | Which part of the graph is retrieved: concepts linked to the persona's process steps and domains, persona labels, definitions, provenance | Retrieval is limited to a token budget and always includes definitions and sources |
| Security | Nothing on its own: a persona selects relevance, never permission | Access is decided by the user's authenticated entitlements, applied before persona tailoring |

- **Persona from identity, not from the prompt:** the persona is set from the authenticated user's role, so a user cannot gain different context by claiming to be another persona in a prompt.
- **Filter first, tailor second:** retrieval first removes everything the user is not entitled to see (by named graph and data classification), then applies persona ranking.
- **Traceability:** every AI answer logs the persona, prompt template version, and context retrieved, supporting the traceability requirements in Section 3.6 and ISO/IEC 42001.
- **Feedback:** corrections and low-rated answers are grouped by persona, so missing labels and information needs are found and fed back into persona maintenance.

# Attributes, Properties and Relations

This section sets out which attributes are recorded for taxonomies and ontologies, how ambiguous relations such as "includes" are modeled, and how asserted, subclass, and inferred relations are kept apart. The complete metadata field set for each artifact, as captured by the workbench forms, is in Section 3.11.

## Attributes and Properties for Taxonomies

Taxonomies are published as SKOS concept schemes, following ISO 25964 for thesauri and the label rules in Section 3.8.

| Level | Property | Required | Notes |
|---|---|---|---|
| Scheme | dcterms:title, dcterms:description | Yes | Name and purpose of the taxonomy |
| Scheme | owl:versionInfo, dcterms:modified, dcterms:publisher | Yes | Version, last change, and owning team |
| Scheme | skos:hasTopConcept | Yes | The first level of the hierarchy |
| Concept | skos:prefLabel, skos:definition | Yes | As in the term definition standard (Section 3.7) |
| Concept | skos:inScheme | Yes | The scheme the concept belongs to |
| Concept | skos:broader / skos:narrower | Yes (except top) | Classification hierarchy; top concepts use skos:topConceptOf |
| Concept | skos:related | Optional | Associative link to a concept that is not broader or narrower |
| Concept | skos:altLabel, skos:hiddenLabel, skos:notation | As needed | Synonyms, search variants, and codes |
| Concept | skos:scopeNote, skos:example, skos:changeNote, skos:historyNote | Recommended | Usage guidance and change history |
| Concept | skos:exactMatch, closeMatch, broadMatch, narrowMatch, relatedMatch | As needed | Mappings to other schemes, always through the core (Section 3.5) |
| Concept | owl:deprecated, dcterms:isReplacedBy | On retirement | Retired concepts point to their replacement |

- **Hierarchy means classification:** skos:broader is used only for "is a kind of" or "is a category of"; part-of and membership use ontology relations instead.
- **Limited polyhierarchy:** a concept may have more than one broader concept only when both are true classifications, and validation reports more than two.
- **Transitivity:** skos:broader is not transitive; queries that need all ancestors use skos:broaderTransitive or a property path (skos:broader+).

## Attributes and Properties for Ontologies

| Element | Required properties | Notes |
|---|---|---|
| Ontology header | owl:Ontology, owl:versionIRI, owl:versionInfo, | owl:backwardCompatibleWith or |

| Element | Required properties | Notes |
|---|---|---|
| | owl:priorVersion, owl:imports, dcterms:title, dcterms:description, dcterms:creator, dcterms:license, dcterms:modified | owl:incompatibleWith on every release |
| Class | The fields in Section 3.7 | At least one named parent class |
| Object property | Labels and definition, rdfs:subPropertyOf (from the relation taxonomy), rdfs:domain, rdfs:range, owl:inverseOf where needed | Characteristics (functional, transitive, symmetric) declared only when always true |
| Data property | Labels and definition, rdfs:subPropertyOf (from the attribute taxonomy), rdfs:domain, rdfs:range (an XML Schema datatype) | Unit, data classification, and system of record recorded as annotations |
| Annotation property | Labels and definition | Used for documentation and governance only; never carries business facts |

### Use of "includes"

"Includes" is one of the most common words in business definitions and one of the most ambiguous. It can mean at least six different things, each with different consequences for inheritance and queries. A generic property named includes is therefore **not** created. The modeler chooses the specific relation, and "includes" may be recorded as an alternate label of that relation so business users still recognize it.

| Statement | Actual meaning | Model as |
|---|---|---|
| "Benefits include medical, dental, and vision" | Kinds of (subclass) | rdfs:subClassOf: Medical Benefit subclass of Benefit |
| "The payroll process includes calculating gross pay" | Part of a process | Has occurrent part (BFO), specialized as core:hasStep |
| "The service bundle includes the payroll service" | Component of a package | core:hasComponent (has continuant part, BFO) |
| "The Wages and Hours category includes Overtime" | Taxonomy narrower concept | skos:narrower |
| "The client list includes Acme Corporation" | Membership of a collection | core:hasMember, or rdf:type when the list is a class |
| "The payroll ontology includes the core ontology" | Reuse of another ontology | owl:imports |

## Use of Relations: Asserted, Subclass, and Inferred

| Kind | Where it comes from | Where it is stored | Example | Rule |
|---|---|---|---|---|
| Asserted | Stated by an author, a mapping, or an ingestion pipeline | Authored named graph, with provenance | Payroll Run hasStep Calculate gross pay; Payroll Run performedBy Payroll Specialist | The only kind people edit; every assertion has a source |
| Subclass | Asserted rdfs:subClassOf and rdfs:subPropertyOf | Ontology graph | Payroll Run subclass of Process Step | Carries inheritance; kept explicit and deep (Section 3.1) |

| Kind | Where it comes from | Where it is stored | Example | Rule |
|---|---|---|---|---|
| Inferred | Computed by the GraphDB ruleset (RDFS plus selected OWL 2 RL rules) | Implicit statements, kept separate from explicit ones | A Payroll Run is also typed as Process Step and Process | Never edited by hand; fix the asserted source instead |

- **Approved inferences only:** the ruleset is limited to subclass and subproperty closure, inverse properties, and transitivity for named properties (for example, part-of). Anything else needs Architecture approval.
- **Domain and range are not validation:** in RDFS, using a property on the wrong kind of subject silently infers a new type. Data quality is therefore checked with SHACL shapes, and unexpected inferred types are reported.
- **Explicit versus implicit:** queries for change review and impact assessment read explicit statements only; GraphDB exposes them separately from implicit statements, so reviewers see what was actually authored.
- **Materialize at publish:** inferred hierarchy statements that applications rely on are written into the release, so consumers get the same answers without running a reasoner.

# BFO and RDF Foundations

Basic Formal Ontology (BFO 2020, ISO/IEC 21838-2) gives the core ontology its top-level categories and its fundamental relations. Company relations and attributes are placed under those categories, so every new property has a defined meaning and a parent, and the ontologies stay compatible with other BFO-based ontologies such as the Information Artifact Ontology (IAO) and the OBO Relations Ontology (RO).

## Use of BFO for Relations and Attributes

| BFO category | Meaning | Company examples |
|---|---|---|
| Material entity | A physical thing | Worksite building, laptop |
| Process | Something that happens over time | Payroll run, enrollment, onboarding step |
| Quality | A measurable characteristic of a thing | Hours worked, pay rate, headcount |
| Role | Something a bearer has because of its situation, realized in processes | Payroll Manager, client administrator, co-employer |
| Disposition or function | A built-in tendency or purpose | Purpose of a service, capability of a system |
| Generically dependent continuant (information) | Information that can be copied across carriers | Policy, regulation text, document, rule, identifier, record |

**Relations.** Company object properties are declared as subproperties of BFO or RO relations where the meaning fits, so that their behavior (for example, transitivity of parthood) is inherited rather than re-invented.

| Company relation | BFO / RO parent | Note |
|---|---|---|
| hasStep / isStepOf | has occurrent part / occurrent part of | Transitive; supports roll-up of steps into processes |
| hasComponent | has continuant part | For packages, bundles, and assemblies |
| performedBy | has participant (RO: has active participant) | Shortcut for: an agent bearing a role participates in the step |
| consumes / produces | RO: has input / has output | Inputs and outputs of a process step |
| precedes / follows | precedes / preceded by | Order of process steps |
| hasRole | bearer of | Links a party to a role; the role is realized in process steps |
| locatedAt | located in | Worksites, offices, jurisdictions |
| governs / governedBy | IAO: is about (via governs) | A rule or policy is information about a process step |
| measures / measuredBy | IAO: is about (via measures) | A metric is information about a process or quality |

**Attributes.** An attribute is modeled as a simple data property when its value is a plain fact about one thing. It is modeled as a BFO quality, role, or information entity with its own class when it changes over time, has units or currency, has its own provenance, or is shared. For example, hasEffectiveDate on a policy is a data property, while a pay rate (amount, currency, effective dates, source) is a Pay Rate entity linked to the employee.

## Taxonomy of Relations

Every object property sits in one hierarchy rooted in the BFO relation families. A new relation cannot be released without a parent in this taxonomy.

| Family (top property) | Company relations beneath it |
|---|---|
| Parthood (has part / part of) | hasStep, hasComponent, hasSubprocess |
| Participation (has participant) | performedBy, approvedBy, consumes, produces, notifies |
| Dependence and inherence (bearer of / inheres in) | hasRole, hasQuality, hasCapability |
| Realization (realizes) | fulfillsRole, deliversService |
| Temporal order (precedes) | precedes, triggers, supersedes |
| Location (located in) | locatedAt, hasJurisdiction, hasWorksite |
| Aboutness (is about) | governs, measures, documents, describes |
| Social and contractual (core:socialRelation; not in BFO) | employs, coEmploys, isClientOf, contractsWith, reportsTo |

- **Leaf properties carry domain and range;** top properties carry definitions and characteristics only.
- **Inverses are declared once** with owl:inverseOf; only one direction is asserted in data.
- **Mappings are not relations:** SKOS mapping properties (exactMatch and others) are kept out of this taxonomy.

## Taxonomy of Attributes and Properties

| Family (top data property) | Examples | Rules |
|---|---|---|
| Identifier | hasDocumentIdentifier, hasEmployeeNumber, hasClientNumber | Usually functional; candidates for owl:hasKey; never reused |
| Temporal | hasEffectiveDate, hasEndDate, hasCreatedDate | xsd:date or xsd:dateTime with time zone |
| Quantitative | hasAmount, hasRate, hasHoursWorked | xsd:decimal; unit and currency always recorded |
| Descriptive | hasName, hasDescription | rdf:langString where text needs a language |
| Status and flag | isExempt, isActive | xsd:boolean; if a status has a lifecycle, use a controlled concept instead |
| Contact and personal | hasEmailAddress, hasPhoneNumber | Classified Confidential or Restricted; access controlled by named graph |

Annotation properties follow the same discipline in four families: labels (Section 3.8), documentation (definition, scope note, example), provenance (source, creator, dates), and governance (status, owner, version, data classification, intended persona).

## RDF 1.1 and Properties

RDF 1.1 (W3C Recommendation, 2014) is the base data model for every artifact in this document. Its treatment of properties is what makes the relation and attribute taxonomies above possible.

- **Properties are resources:** any IRI used as a predicate is an rdf:Property, so a property can have its own label, definition, parent, owner, and version, and can be governed like a class.
- **RDF Schema 1.1** adds rdfs:subPropertyOf, rdfs:domain, and rdfs:range, which carry the property hierarchy; OWL 2 adds object, data, and annotation properties and their characteristics.
- **Internationalized identifiers:** RDF 1.1 uses IRIs, which allow characters beyond ASCII; Company identifiers still use plain ASCII for portability.
- **Every literal has a datatype:** plain text is xsd:string and language-tagged text is rdf:langString, so data property ranges are always explicit.
- **Datasets and named graphs:** RDF 1.1 standardized datasets of named graphs and the TriG and N-Quads formats; Company artifacts use named graphs for ownership, provenance, release, and access control.

RDF 1.1 has no compact way to attach properties to a single statement (for example, the source or confidence of one fact). Until that is needed, two RDF 1.1 patterns are used: a **named graph** for provenance shared by a batch of statements, and a **qualified relation** (an n-ary pattern) when the relationship itself has business attributes. RDF 1.2, being standardized by W3C, adds triple terms and a Turtle annotation syntax for statement-level properties; GraphDB supports the earlier RDF-star form. It may be adopted for metadata such as source and confidence once the standard is final, but business facts stay in qualified relations.

```
# RDF 1.1: qualified relation for an employment with its own attributes
@prefix employment: <https://www.insperity.com/resource/domain-human-resources/employment/> .
@prefix employee:   <https://www.insperity.com/resource/domain-human-resources/worksite-
employee/> .
@prefix client:     <https://www.insperity.com/resource/commercial-operations/client/> .

employment:000123 a core:employment ;
    core:hasEmployee   employee:004512 ;
    core:hasEmployer   client:000987 ;
    core:hasStartDate  "2024-01-15"^^xsd:date .
```

## Reference Ontology Stack and Load Order

The ontologies form one stack, loaded in dependency order:

BFO 2020 ← RO Core + RO extract ← core domain ontology ← area container ontology ← domain ontology (L1 ← L2 ← L3)

- **Reference ontologies keep their own IRIs:** BFO 2020, RO Core, and the RO extract (the BFO 2020 relations the OBO file lacks, has input and has output from RO, and is about from IAO) are loaded into their own named graphs and never ingested or renamed.

- **Lineage:** every core class reaches BFO, every relation reaches BFO, RO, or IAO, and every domain class reaches BFO through a core class. The build and the reload script check lineage.
- **Taxonomies reach BFO through classes:** vocabularies and taxonomies carry no BFO links of their own; a concept reaches BFO through core:formalizedBy to a class. A domain class that formalizes an existing core concept states the core:formalizedBy triple in the domain's graph (proposed), so unloading the domain removes it.
- **Pinned imports:** a domain ontology imports its parents by ontology IRI and pins their versions with dcterms:requires (Section 3.1).

# Semantic Asset Metadata

Every vocabulary, taxonomy, and ontology, and every term, concept, class, and property within them, carries the same governed metadata. Each metadata field maps to exactly one RDF property, and each property is tied to the ISO or IEEE standard that governs it. This section is the field specification for the Semantic Modeling Workbench metadata forms and the metadata registry; Sections 3.7 to 3.9 remain the rules for how definitions, labels, and relations are written.

## Standards Basis

ISO standards govern content: what a label, definition, or status must be. IEEE standards govern the lifecycle: versioning, contribution roles, and change records. The short codes below are used in every table in this section; the Standards & Compliance Map (Section 3.4) records which edition applies.

| Code | Standard | What it governs here |
|---|---|---|
| ISO 15836 | ISO 15836-1:2017 and ISO 15836-2:2019, Dublin Core metadata element set and properties | Title, description, creator, contributor, publisher, subject, audience, language, dates, source, license, conformance |
| ISO 25964-1, -2 | ISO 25964-1:2011 and ISO 25964-2:2013, Thesauri and interoperability with other vocabularies | Preferred and non-preferred terms, hierarchy, scope and history notes, mappings |
| ISO 704, ISO 1087 | ISO 704:2022 Terminology work: principles and methods; ISO 1087:2019 Terminology vocabulary | Concepts, terms, intensional definitions, acronyms |
| ISO 30042, ISO 12620-1 | ISO 30042:2019 TermBase eXchange (TBX); ISO 12620-1:2022 terminology data categories | Term types (full form, acronym, abbreviation) and usage by audience |
| ISO/IEC 11179-4, -5, -6 | ISO/IEC 11179-4:2004 data definitions; -5:2015 naming; -6:2023 registration | Definition rules, naming rules, registration status, registration authority |
| ISO/IEC 19763-3 | ISO/IEC 19763-3:2020, Metamodel for ontology registration | Registering ontologies, their versions, imports, and locations |
| ISO/IEC 21838-1, -2 | ISO/IEC 21838-1:2021 and ISO/IEC 21838-2:2021 (BFO), Top-level ontologies | Top-level alignment, named parents, relation families |
| ISO 24619 | ISO 24619:2011, Persistent identification of language resources | Persistent, resolvable, version-aware identifiers |
| ISO 8601, ISO 639 | ISO 8601-1:2019 dates and times; ISO 639 language codes, applied through BCP 47 | Date values; language tags on every label and definition |
| IEEE LOM | IEEE 1484.12.1-2020, Learning Object Metadata | Title (1.2), language (1.3), description (1.4), version (2.1), status (2.2), contribution roles (2.3), classification (9) |
| IEEE 828 | IEEE 828-2012, Configuration management in systems and software engineering | Baselines, version identification, change records |
| ISO/IEC/IEEE 15288 | ISO/IEC/IEEE 15288:2023, System life cycle processes | Configuration and information management of deployed artifacts |

| Code | Standard | What it governs here |
|---|---|---|
| ISO/IEC/IEEE 42010 | ISO/IEC/IEEE 42010:2022, Architecture description | Stakeholders and concerns, the basis of personas and audiences |
| ISO/IEC/IEEE 24765 | ISO/IEC/IEEE 24765:2017, Systems and software engineering vocabulary | Term and definition practice for engineering terms |

W3C recommendations (SKOS, SKOS-XL, OWL 2, RDF Schema, DCAT, SPARQL Service Description) and the community vocabularies BIBO, VANN, and VAEM are the RDF bindings that carry these requirements. Each field cites the ISO or IEEE standard its binding implements; Dublin Core Terms (dcterms:) is the RDF form of ISO 15836.

## Conventions

- **Header and members:** each artifact has a header record (the vocabulary, taxonomy, or ontology itself) and member records (its terms, concepts, classes, and properties). Header fields describe the asset; definitions belong only to members, so an asset header carries dcterms:description, never skos:definition.
- **Table columns:** Card. is cardinality (1, 0..1, 1..*, 0..*; "per lang" means one per language tag). Obl. is M mandatory, R recommended, O optional, or C conditional. Entry is U entered by the user, P picked from a controlled list, or S set by the workbench or the release pipeline and shown read-only.
- **Release gate:** mandatory fields must be complete before an artifact moves from Draft to Candidate (Section 3.3), and the SHACL release gates enforce them.
- **Prefixes:** as in Sections 3.2 and 3.7; core: is the core ontology and dcterms: is Dublin Core Terms.
- **Status recorded once, published twice:** the workbench records core:hasLifecycleStatus. At publish time the pipeline also writes bibo:status for consumers outside the Company, the same way plain skos:altLabel values are written alongside SKOS-XL labels (Section 3.8).
- **Versions set by the build:** owl:versionInfo and owl:versionIRI are set by the release pipeline from the schema diff (Section 3.3); the form shows them read-only.
- **Location is not content:** Named graph and Repository are entered on the same form but written to the artifact's entry in the metadata registry, not into the artifact. Identifiers are the same in every environment; only the endpoint changes (Section 3.3).

## Identifiers, Versions, and Named Graphs

Asset URIs follow the patterns in Section 3.2 and never contain a version. Only the release receives a versioned IRI, and each artifact's current release is held in one named graph, whose name is the asset URI itself (Section 3.2).

| Artifact | Asset URI | Version IRI | Named graph |
|---|---|---|---|
| Vocabulary | https://www.insperity.com/vocabulary/{scope}/ | https://www.insperity.com/vocabulary/{scope}/{version}/ | The asset URI |
| Taxonomy | https://www.insperity.com/taxonomy/{scope}/ | https://www.insperity.com/taxonomy/{scope}/{version}/ | The asset URI |
| Ontology | https://www.insperity.com/ | https://www.insperity.com/ | The asset URI |

| Artifact | Asset URI | Version IRI | Named graph |
|---|---|---|---|
| | ontology/{scope}/ | ontology/{scope}/{version}/ | |
| Shapes | https://www.insperity.com/shapes/{scope}/ | https://www.insperity.com/shapes/{scope}/{version}/ | The asset URI |

- **One graph per artifact:** a deployment replaces the artifact's named graph with the new release (Section 3.3). Earlier releases are retrieved from their release packages, not kept as archived graphs.
- **Repository:** an environment-neutral repository identifier (for example, skl-knowledge); the endpoint for each environment comes from configuration.
- **Member IRIs:** terms, concepts, classes, and properties are minted under the artifact's namespace with the naming rules in Section 3.2, and never contain a version.

## Term Metadata

A term is one skos:Concept in a vocabulary. Meaning lives in the term; taxonomies and ontologies point to it.

### Term Form Fields

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Term IRI | (the subject IRI) | 1 | M | S | ISO 24619; ISO/IEC 11179-5 | IEEE 828 |
| Vocabulary | skos:inScheme | 1..* | M | S | ISO 25964-1 | — |
| Preferred label | skos:prefLabel | 1 per lang | M | U | ISO 25964-1; ISO 704; ISO 30042 | ISO/IEC/IEEE 24765 |
| Label | rdfs:label | 1 per lang | M | S | ISO/IEC 11179-5 | — |
| Alternate label | skos:altLabel | 0..* | R | U | ISO 25964-1; ISO 30042 | — |
| Acronym | skosxl:altLabel with core:hasTermType | 0..* | C | U | ISO 30042; ISO 1087 | — |
| Persona label | skosxl:altLabel with core:intendedForPersona | 0..* | O | U, P | ISO 30042; ISO 12620-1 | ISO/IEC/IEEE 42010 |
| Hidden label | skos:hiddenLabel | 0..* | O | U | ISO 25964-1 | — |
| Definition | skos:definition | 1 per lang | M | U | ISO 704; ISO/IEC 11179-4 | ISO/IEC/IEEE 24765 |
| Definition source | dcterms:source | 1..* | R | U | ISO 15836; ISO 704 | — |
| Scope note | skos:scopeNote | 0..* | R | U | ISO 25964-1 | — |
| Example | skos:example | 0..* | R | U | ISO 704 | — |
| Notation | skos:notation | 0..* | O | U | ISO 25964-1 | — |
| Mapping | skos:exactMatch, skos:closeMatch | 0..* | O | P | ISO 25964-2 | — |
| Creator | dcterms:creator | 1 | M | S | ISO 15836 | IEEE LOM 2.3 |
| Contributor | dcterms:contributor | 0..* | R | S | ISO 15836 | IEEE LOM 2.3 |

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Created | dcterms:created | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Modified | dcterms:modified | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Version | owl:versionInfo | 1 | M | S | ISO/IEC 19763-3 | IEEE LOM 2.1; IEEE 828 |
| Lifecycle status | core:hasLifecycleStatus | 1 | M | P | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Deprecated | owl:deprecated | 0..1 | C | S | ISO/IEC 11179-6; ISO 30042 | IEEE LOM 2.2 |
| Replaced by | dcterms:isReplacedBy | 0..1 | C | P | ISO 15836; ISO 25964-1 | IEEE 828 |
| Change note | skos:changeNote | 0..* | R | U | ISO 25964-1 | IEEE 828 |
| Editorial note | skos:editorialNote | 0..* | O | U | ISO 25964-1 | — |

Conditional fields: Acronym is mandatory when the term has an acronym listed in Appendix F; Deprecated is set when the status becomes Deprecated, and Replaced by is then mandatory. The Acronym and Persona label fields create SKOS-XL label resources; the pipeline also writes their text as plain skos:altLabel at publish time.

### Term Cross-Reference: Terms and Acronyms

| Term | Acronym | Description | Property |
|---|---|---|---|
| Term identifier | IRI | Persistent identifier of the term; never contains a version or an acronym | (the subject IRI) |
| Vocabulary membership | — | The vocabulary that owns the term | skos:inScheme |
| Preferred term | PT | The one authoritative name per language: full words, sentence case, no acronyms | skos:prefLabel, rdfs:label |
| Non-preferred term | NPT | Synonym or variant that leads to the preferred term | skos:altLabel |
| Acronym | — | Short form built from initial letters; recorded only as an alternate label | skosxl:altLabel, core:hasTermType |
| Abbreviation | — | Shortened form that is not an acronym | skosxl:altLabel, core:hasTermType |
| Persona label | — | The word a persona uses for the concept (Section 3.8) | skosxl:altLabel, core:intendedForPersona |
| Search variant | — | Misspelling or legacy term matched by search and never displayed | skos:hiddenLabel |
| Intensional definition | — | Genus-plus-differentia statement of what the concept is (Section 3.7) | skos:definition |
| Authoritative source | — | Policy, regulation, or standard the definition was taken from | dcterms:source |
| Scope note | SN | What the term includes and excludes, and when to use it | skos:scopeNote |
| Notation | — | Code from a system of record; the only place a code may stand alone | skos:notation |

| Term | Acronym | Description | Property |
|---|---|---|---|
| Mapping | — | Equivalent or near-equivalent concept in another scheme | skos:exactMatch, skos:closeMatch |
| Lifecycle status | — | Draft, Candidate, Released, Deprecated, or Retired | core:hasLifecycleStatus |
| Successor term | — | The term to use instead of a deprecated one | dcterms:isReplacedBy |
| History note | — | Dated record of a change: what, why, and who | skos:changeNote |
| Editorial note | — | Internal note for stewards; not shown to consumers | skos:editorialNote |

The same term in Turtle:

```
@prefix voc:     <https://www.insperity.com/vocabulary/domain-employee-benefits-and-
retirement-sponsorship/> .
@prefix label:   <https://www.insperity.com/vocabulary/domain-employee-benefits-and-
retirement-sponsorship/label/> .
@prefix persona: <https://www.insperity.com/vocabulary/core-persona/> .

voc:retirement-plan a skos:Concept ;
    skos:inScheme   <https://www.insperity.com/vocabulary/domain-employee-benefits-and-
retirement-sponsorship/> ;
    skos:prefLabel  "Retirement plan"@en ;
    rdfs:label      "Retirement plan"@en ;
    skos:altLabel   "Retirement savings plan"@en , "401(k) plan"@en ;
    skos:definition "An employer-sponsored benefit plan that provides income
                     to employees after they stop working."@en ;
    dcterms:source  "Employee Benefits Policy, Section 4" ;
    skosxl:altLabel label:retirement-plan-worksite-employee ;
    core:hasLifecycleStatus core:Draft ;
    owl:versionInfo "0.1.0" ;
    dcterms:created "2026-10-15"^^xsd:date .

label:retirement-plan-worksite-employee a skosxl:Label ;
    skosxl:literalForm      "My 401(k)"@en ;
    core:intendedForPersona persona:worksite-employee .
```

## Vocabulary Metadata

A vocabulary is a skos:ConceptScheme that owns a flat list of terms. Its header carries identity, description, provenance, version, and status.

### Vocabulary Form Fields

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Asset URI | (the subject IRI); rdf:type skos:ConceptScheme | 1 | M | S | ISO 24619; ISO/IEC 11179-5 | IEEE 828 |
| Namespace | vaem:namespace | 1 | M | U | ISO 24619 | — |
| Namespace prefix | vann:preferredNamespacePrefix | 1 | R | U | ISO/IEC 11179-5 | — |

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Named graph | sd:name (registry entry) | 1 | M | S | ISO/IEC 19763-3 | IEEE 828 |
| Repository | dcat:DataService (registry entry) | 1 | M | P | ISO/IEC 19763-3; ISO/IEC 11179-6 | ISO/IEC/IEEE 15288 |
| Title | dcterms:title | 1 per lang | M | U | ISO 15836 | IEEE LOM 1.2 |
| Label | rdfs:label | 1 per lang | M | S | ISO/IEC 11179-5 | — |
| Preferred label | skos:prefLabel | 1 per lang | M | S | ISO 25964-1 | — |
| Alternate label | skos:altLabel | 0..* | O | U | ISO 25964-1; ISO 30042 | — |
| Persona label | skosxl:altLabel with core:intendedForPersona | 0..* | O | U, P | ISO 30042 | ISO/IEC/IEEE 42010 |
| Description | dcterms:description | 1 per lang | M | U | ISO 15836 | IEEE LOM 1.4 |
| Subject (topic or domain) | dcterms:subject | 1..* | M | P | ISO 15836; ISO 25964-1 | IEEE LOM 9 |
| Audience | dcterms:audience | 0..* | O | P | ISO 15836 | ISO/IEC/IEEE 42010 |
| Language | dcterms:language | 1..* | M | P | ISO 639; ISO 15836 | IEEE LOM 1.3 |
| Creator | dcterms:creator | 1..* | M | S | ISO 15836 | IEEE LOM 2.3 |
| Contributor | dcterms:contributor | 0..* | R | S | ISO 15836 | IEEE LOM 2.3 |
| Publisher | dcterms:publisher | 1 | M | P | ISO/IEC 11179-6; ISO 15836 | IEEE LOM 2.3 |
| License | dcterms:license | 0..1 | R | P | ISO 15836 | — |
| Created | dcterms:created | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Modified | dcterms:modified | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Issued | dcterms:issued | 0..1 | C | S | ISO 15836; ISO 8601 | IEEE 828 |
| Version | owl:versionInfo | 1 | M | S | ISO/IEC 19763-3 | IEEE LOM 2.1; IEEE 828 |
| Version IRI | owl:versionIRI | 1 | M | S | ISO 24619 | IEEE 828 |
| Prior version | owl:priorVersion | 0..1 | C | S | ISO/IEC 19763-3 | IEEE 828 |
| Lifecycle status | core:hasLifecycleStatus (bibo:status at publish) | 1 | M | P | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Conforms to | dcterms:conformsTo | 1 | M | S | ISO 15836 | IEEE 828 |
| Change note | skos:changeNote | 0..* | R | U | ISO 25964-1 | IEEE 828 |

Conditional fields: Issued is set when the status becomes Released; Prior version is set on every release after the first. Subject values are concepts from the Company domain taxonomy, never free text.

### Vocabulary Cross-Reference: Terms and Acronyms

| Term | Acronym | Description | Property |
|---|---|---|---|
| Asset identifier | IRI | Persistent, version-free identifier of the artifact | (the subject IRI) |
| Namespace | — | IRI prefix under which member IRIs are minted | vaem:namespace |
| Namespace prefix | — | Short prefix used for the namespace in Turtle and SPARQL | vann:preferredNamespacePrefix |
| Named graph | — | Graph in the knowledge graph that holds the artifact's current release | sd:name (registry entry) |
| Repository | — | Triplestore repository that serves the artifact | dcat:DataService (registry entry) |
| Title | — | Human name of the artifact; also written as its label and preferred label | dcterms:title, rdfs:label, skos:prefLabel |
| Description | — | Purpose, coverage, exclusions, and intended users | dcterms:description |
| Subject | — | Topic or domain, picked from the domain taxonomy | dcterms:subject |
| Intended audience | — | Personas the vocabulary is written for (Section 3.8) | dcterms:audience |
| Language | — | Languages its labels and definitions are written in | dcterms:language |
| Registration authority | RA | Organization accountable for the artifact | dcterms:publisher |
| Release date | — | Date the current version was released | dcterms:issued |
| Semantic version | SemVer | MAJOR.MINOR.PATCH release number (Section 3.3) | owl:versionInfo |
| Version identifier | — | IRI of one specific release, and of the release it replaces | owl:versionIRI, owl:priorVersion |
| Lifecycle status | — | Draft, Candidate, Released, Deprecated, or Retired | core:hasLifecycleStatus, bibo:status |
| Conformance | — | Edition of this Guide the artifact was validated against | dcterms:conformsTo |
| History note | — | Dated record of a release: what changed, why, and who approved it | skos:changeNote |

A vocabulary header in Turtle:

```
<https://www.insperity.com/vocabulary/domain-employee-benefits-and-retirement-sponsorship/> a
skos:ConceptScheme ;
    vaem:namespace    "https://www.insperity.com/vocabulary/domain-employee-benefits-and-
retirement-sponsorship/"^^xsd:anyURI ;
    vann:preferredNamespacePrefix "voc" ;
    dcterms:title     "Benefits vocabulary"@en ;
    rdfs:label        "Benefits vocabulary"@en ;
    skos:prefLabel    "Benefits vocabulary"@en ;
    dcterms:description "Terms for employee benefit plans, eligibility, and
```

```
                      enrollment. Excludes payroll tax terms."@en ;
    dcterms:subject   <https://www.insperity.com/vocabulary/core-domain/benefit> ;
    dcterms:language  "en" ;
    dcterms:creator   person:benefits-data-steward ;
    dcterms:publisher <https://www.insperity.com/resource/core/Organization/insperity> ;
    dcterms:created   "2026-09-01"^^xsd:date ;
    dcterms:modified  "2026-10-15"^^xsd:date ;
    owl:versionInfo   "1.2.0" ;
    owl:versionIRI    <https://www.insperity.com/vocabulary/domain-employee-benefits-and-
retirement-sponsorship/1.2.0/> ;
    owl:priorVersion  <https://www.insperity.com/vocabulary/domain-employee-benefits-and-
retirement-sponsorship/1.1.0/> ;
    core:hasLifecycleStatus core:Candidate ;
    skos:changeNote   "2026-10-15, 1.2.0: added retirement plan terms (feature
                      Benefits-Feature-021); approved by the benefits domain owner."@en .
```

## Taxonomy Metadata

A taxonomy is a skos:ConceptScheme that arranges concepts into a classification. Each concept takes its labels and meaning from one or more vocabulary terms. Taxonomies stay in SKOS; inherited meaning comes from ontology classes (Section 3.10).

### Taxonomy Header Form Fields

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Asset URI | (the subject IRI); rdf:type skos:ConceptScheme | 1 | M | S | ISO 24619; ISO/IEC 11179-5 | IEEE 828 |
| Namespace | vaem:namespace | 1 | M | U | ISO 24619 | — |
| Namespace prefix | vann:preferredNamespacePrefix | 1 | R | U | ISO/IEC 11179-5 | — |
| Named graph | sd:name (registry entry) | 1 | M | S | ISO/IEC 19763-3 | IEEE 828 |
| Repository | dcat:DataService (registry entry) | 1 | M | P | ISO/IEC 19763-3; ISO/IEC 11179-6 | ISO/IEC/IEEE 15288 |
| Title | dcterms:title | 1 per lang | M | U | ISO 15836 | IEEE LOM 1.2 |
| Label | rdfs:label | 1 per lang | M | S | ISO/IEC 11179-5 | — |
| Preferred label | skos:prefLabel | 1 per lang | M | S | ISO 25964-1 | — |
| Alternate label | skos:altLabel | 0..* | O | U | ISO 25964-1; ISO 30042 | — |
| Persona label | skosxl:altLabel with core:intendedForPersona | 0..* | O | U, P | ISO 30042 | ISO/IEC/IEEE 42010 |
| Description | dcterms:description | 1 per lang | M | U | ISO 15836 | IEEE LOM 1.4 |
| Source vocabularies | dcterms:source | 1..* | M | P | ISO 15836; ISO 25964-2 | — |
| Subject (topic or domain) | dcterms:subject | 1..* | M | P | ISO 15836; ISO 25964-1 | IEEE LOM 9 |
| Top concepts | skos:hasTopConcept | 1..* | M | S | ISO 25964-1 | — |

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Language | dcterms:language | 1..* | M | P | ISO 639; ISO 15836 | IEEE LOM 1.3 |
| Creator | dcterms:creator | 1..* | M | S | ISO 15836 | IEEE LOM 2.3 |
| Contributor | dcterms:contributor | 0..* | R | S | ISO 15836 | IEEE LOM 2.3 |
| Publisher | dcterms:publisher | 1 | M | P | ISO/IEC 11179-6; ISO 15836 | IEEE LOM 2.3 |
| Created | dcterms:created | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Modified | dcterms:modified | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Issued | dcterms:issued | 0..1 | C | S | ISO 15836; ISO 8601 | IEEE 828 |
| Version | owl:versionInfo | 1 | M | S | ISO/IEC 19763-3 | IEEE LOM 2.1; IEEE 828 |
| Version IRI | owl:versionIRI | 1 | M | S | ISO 24619 | IEEE 828 |
| Prior version | owl:priorVersion | 0..1 | C | S | ISO/IEC 19763-3 | IEEE 828 |
| Lifecycle status | core:hasLifecycleStatus (bibo:status at publish) | 1 | M | P | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Conforms to | dcterms:conformsTo | 1 | M | S | ISO 15836 | IEEE 828 |
| Change note | skos:changeNote | 0..* | R | U | ISO 25964-1 | IEEE 828 |

## Taxonomy Concept Form Fields

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Concept IRI | (the subject IRI) | 1 | M | S | ISO 24619; ISO/IEC 11179-5 | IEEE 828 |
| Taxonomy | skos:inScheme | 1 | M | S | ISO 25964-1 | — |
| Primary term | core:hasPrimaryTerm | 1 | M | P | ISO 704; ISO 25964-1 | — |
| Concepts (source terms) | dcterms:subject | 1..* | M | P | ISO 15836; ISO 25964-2 | — |
| Preferred label | skos:prefLabel, rdfs:label | 1 per lang | M | S | ISO 25964-1 | — |
| Definition | skos:definition | 1 per lang | M | S | ISO 704; ISO/IEC 11179-4 | ISO/IEC/IEEE 24765 |
| Broader | skos:broader | 0..2 | C | P | ISO 25964-1 | — |
| Narrower | skos:narrower | 0..* | O | S | ISO 25964-1 | — |
| Top concept of | skos:topConceptOf | 0..1 | C | S | ISO 25964-1 | — |
| Related | skos:related | 0..* | O | P | ISO 25964-1 | — |
| Formalizing class | core:formalizedBy | 0..1 | R | P | ISO/IEC 21838-1; ISO 704 | — |
| Mapping | skos:exactMatch, closeMatch, broadMatch, narrowMatch, relatedMatch | 0..* | O | P | ISO 25964-2 | — |
| Notation | skos:notation | 0..* | O | U | ISO 25964-1 | — |
| Lifecycle status | core:hasLifecycleStatus | 1 | M | P | ISO/IEC 11179-6 | IEEE LOM 2.2 |

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Deprecated | owl:deprecated | 0..1 | C | S | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Replaced by | dcterms:isReplacedBy | 0..1 | C | P | ISO 15836; ISO 25964-1 | IEEE 828 |
| Change note | skos:changeNote | 0..* | R | U | ISO 25964-1 | IEEE 828 |

Conditional fields: Broader is mandatory for every concept except top concepts, which carry Top concept of instead. Concept IRIs join their primitives in lower case with hyphens (Section 3.2).

### Polymorphism: Meaning from Several Terms

The Concepts field is a multi-select list of terms from the source vocabularies, so one concept can draw meaning from several terms. Polymorphism works as follows:

- **Several source terms:** every selected term is written as dcterms:subject. The term that names the concept is also written as core:hasPrimaryTerm, a subproperty of dcterms:subject, because RDF does not keep the order of values.
- **Several parents:** a concept may have up to two broader concepts when both are true classifications; validation reports more than two (Section 3.9).
- **Inherited meaning:** skos:broader classifies but implies nothing about meaning. Where applications need inherited properties and constraints, the concept is linked with core:formalizedBy to the ontology class that formalizes it, and inheritance comes from that class's rdfs:subClassOf hierarchy (Section 3.1). This is the path to a core class that the release gates check.
- **Pick-list contents:** the list offers only Candidate and Released terms from the source vocabularies; deprecated terms are hidden and their successors shown.

```
@prefix tax: <https://www.insperity.com/taxonomy/domain-employee-benefits-and-retirement-
sponsorship/> .
@prefix voc: <https://www.insperity.com/vocabulary/domain-employee-benefits-and-retirement-
sponsorship/> .

tax:retirement-benefits a skos:Concept ;
    skos:inScheme       tax:benefit-topics ;
    skos:prefLabel      "Retirement benefits"@en ;
    core:hasPrimaryTerm voc:retirement-plan ;
    dcterms:subject     voc:retirement-plan , voc:deferred-compensation ,
                        voc:employer-matching-contribution ;
    skos:broader        tax:employee-benefits , tax:compensation-plans ;
    core:formalizedBy   benefits:retirement-plan .

# declared once in the core ontology
core:hasPrimaryTerm rdfs:subPropertyOf dcterms:subject .
```

### Taxonomy Cross-Reference: Terms and Acronyms

| Term | Acronym | Description | Property |
|---|---|---|---|
| Concept scheme | — | The taxonomy as a whole; one per file (Appendix E, Step 3) | skos:ConceptScheme |

| Term | Acronym | Description | Property |
|---|---|---|---|
| Source vocabulary | — | Vocabulary whose terms populate the Concepts list | dcterms:source |
| Top concept | TT | Concept with no broader concept; the first level of the hierarchy | skos:hasTopConcept, skos:topConceptOf |
| Primary term | — | Vocabulary term that names the concept; its label and definition are copied | core:hasPrimaryTerm |
| Source term | — | Any vocabulary term the concept draws meaning from, including the primary term | dcterms:subject |
| Broader term | BT | More general concept in the classification | skos:broader |
| Narrower term | NT | More specific concept in the classification | skos:narrower |
| Related term | RT | Associative, non-hierarchical link | skos:related |
| Polyhierarchy | — | Concept with more than one broader concept; at most two (Section 3.9) | skos:broader |
| Polymorphism | — | Concept that draws meaning from several vocabulary terms or parents | dcterms:subject, core:hasPrimaryTerm, skos:broader |
| Formalizing class | — | Ontology class whose subclass hierarchy carries the concept's inherited meaning | core:formalizedBy |
| Mapping | — | Link to a concept in another scheme, always through the core (Section 3.5) | skos:exactMatch, skos:closeMatch, skos:broadMatch |
| Header terms | — | Asset identifier through History note, as for vocabularies | See the vocabulary cross-reference |

## Ontology Metadata

An ontology header adds imports, top-level alignment, and compatibility to the shared header fields. Its classes and properties carry their own definitions and link back to the vocabulary terms they formalize.

### Ontology Header Form Fields

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Ontology IRI | (the subject IRI); rdf:type owl:Ontology | 1 | M | S | ISO 24619; ISO/IEC 11179-5 | IEEE 828 |
| Namespace | vaem:namespace | 1 | M | U | ISO 24619 | — |
| Namespace prefix | vann:preferredNamespacePrefix | 1 | R | U | ISO/IEC 11179-5 | — |
| Named graph | sd:name (registry entry) | 1 | M | S | ISO/IEC 19763-3 | IEEE 828 |
| Repository | dcat:DataService (registry entry) | 1 | M | P | ISO/IEC 19763-3; ISO/IEC 11179-6 | ISO/IEC/IEEE 15288 |
| Title | dcterms:title | 1 per lang | M | U | ISO 15836 | IEEE LOM 1.2 |
| Label | rdfs:label | 1 per lang | M | S | ISO/IEC 11179-5 | — |

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Alternate label | skos:altLabel | 0..* | O | U | ISO 30042 | — |
| Description | dcterms:description | 1 per lang | M | U | ISO 15836 | IEEE LOM 1.4 |
| Subject (topic or domain) | dcterms:subject | 1..* | M | P | ISO 15836 | IEEE LOM 9 |
| Imports | owl:imports | 0..* | C | S | ISO/IEC 19763-3 | IEEE 828 |
| Top-level alignment | dcterms:conformsTo (BFO 2020) | 1 | M | P | ISO/IEC 21838-1; ISO/IEC 21838-2 | — |
| Language | dcterms:language | 1..* | M | P | ISO 639; ISO 15836 | IEEE LOM 1.3 |
| Creator | dcterms:creator | 1..* | M | S | ISO 15836 | IEEE LOM 2.3 |
| Contributor | dcterms:contributor | 0..* | R | S | ISO 15836 | IEEE LOM 2.3 |
| Publisher | dcterms:publisher | 1 | M | P | ISO/IEC 11179-6; ISO 15836 | IEEE LOM 2.3 |
| License | dcterms:license | 1 | M | P | ISO 15836 | — |
| Created | dcterms:created | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Modified | dcterms:modified | 1 | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Issued | dcterms:issued | 0..1 | C | S | ISO 15836; ISO 8601 | IEEE 828 |
| Version | owl:versionInfo | 1 | M | S | ISO/IEC 19763-3 | IEEE LOM 2.1; IEEE 828 |
| Version IRI | owl:versionIRI | 1 | M | S | ISO 24619 | IEEE 828 |
| Prior version | owl:priorVersion | 0..1 | C | S | ISO/IEC 19763-3 | IEEE 828 |
| Compatibility | owl:backwardCompatibleWith or owl:incompatibleWith | 1..* | C | P | ISO/IEC 19763-3 | IEEE 828 |
| Lifecycle status | core:hasLifecycleStatus (bibo:status at publish) | 1 | M | P | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Conforms to | dcterms:conformsTo | 1 | M | S | ISO 15836 | IEEE 828 |
| Change note | skos:changeNote | 0..* | R | U | ISO 25964-1 | IEEE 828 |

Conditional fields: Imports is set by the workbench whenever terms from another ontology are used; Compatibility is mandatory on every release after the first (Section 3.9).

### Class and Property Form Fields

The rows below are the metadata the form captures; the OWL axiom fields (restrictions, disjointness, keys) are specified in Section 3.7. Rows marked (class) or (property) apply only to that kind of member.

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| IRI | (the class or property IRI) | 1 | M | S | ISO 24619; ISO/IEC 11179-5 | IEEE 828 |
| Is defined by | rdfs:isDefinedBy | 1 | M | S | ISO/IEC 19763-3 | — |
| Label | rdfs:label, skos:prefLabel | 1 per lang | M | U | ISO/IEC 11179-5; ISO 704 | — |

| Form field | Property | Card. | Obl. | Entry | ISO | IEEE |
|---|---|---|---|---|---|---|
| Alternate label | skos:altLabel, skosxl:altLabel | 0..* | O | U | ISO 30042 | — |
| Definition | skos:definition | 1 per lang | M | U | ISO 704; ISO/IEC 11179-4 | ISO/IEC/IEEE 24765 |
| Definition source | dcterms:source | 0..* | R | U | ISO 15836 | — |
| Vocabulary term | dcterms:subject | 0..* | R | P | ISO 25964-2 | — |
| Parent class (class) | rdfs:subClassOf (named class) | 1..* | M | P | ISO/IEC 21838-1; ISO 704 | — |
| Parent property (property) | rdfs:subPropertyOf | 1 | M | P | ISO/IEC 21838-2 | — |
| Domain (property) | rdfs:domain | 1 | M | P | ISO/IEC 19763-3 | — |
| Range (property) | rdfs:range | 1 | M | P | ISO/IEC 19763-3 | — |
| Inverse (object property) | owl:inverseOf | 0..1 | O | P | ISO/IEC 21838-2 | — |
| Example | skos:example | 0..* | R | U | ISO 704 | — |
| Creator | dcterms:creator | 1 | M | S | ISO 15836 | IEEE LOM 2.3 |
| Created, Modified | dcterms:created, dcterms:modified | 1 each | M | S | ISO 15836; ISO 8601 | IEEE 828 |
| Version | owl:versionInfo | 1 | M | S | ISO/IEC 19763-3 | IEEE LOM 2.1 |
| Lifecycle status | core:hasLifecycleStatus | 1 | M | P | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Deprecated | owl:deprecated | 0..1 | C | S | ISO/IEC 11179-6 | IEEE LOM 2.2 |
| Replaced by | dcterms:isReplacedBy | 0..1 | C | P | ISO 15836 | IEEE 828 |
| Change note | skos:changeNote | 0..* | R | U | ISO 25964-1 | IEEE 828 |

## Ontology Cross-Reference: Terms and Acronyms

| Term | Acronym | Description | Property |
|---|---|---|---|
| Ontology | — | Formal model of classes, relations, and axioms | owl:Ontology |
| Import | — | Ontology this one reuses | owl:imports |
| Top-level ontology | TLO | Domain-neutral ontology the classes descend from (ISO/IEC 21838-1) | dcterms:conformsTo |
| Basic Formal Ontology | BFO | The top-level ontology used by the core (ISO/IEC 21838-2) | dcterms:conformsTo, rdfs:subClassOf |
| Compatibility | — | Prior releases this one does or does not keep working with | owl:backwardCompatibleWith, owl:incompatibleWith |
| Defining ontology | — | The ontology that owns a class or property | rdfs:isDefinedBy |
| Named parent | — | Parent class that carries inherited meaning (Section 3.1) | rdfs:subClassOf |
| Parent relation | — | Family in the taxonomy of relations or attributes (Section 3.10) | rdfs:subPropertyOf |
| Domain | — | Class of the subject of a property | rdfs:domain |

| Term | Acronym | Description | Property |
|---|---|---|---|
| Range | — | Class or datatype of a property's value | rdfs:range |
| Inverse relation | — | The same relation read in the other direction | owl:inverseOf |
| Source term | — | Vocabulary term the class or property formalizes | dcterms:subject |
| Deprecated term | — | Retired class or property that stays resolvable | owl:deprecated |
| Successor term | — | The class or property to use instead | dcterms:isReplacedBy |
| Header terms | — | Asset identifier through History note, as for vocabularies | See the vocabulary cross-reference |

## Controlled Values

### Lifecycle Status

Status values are the lifecycle states in Section 3.3. The table shows the value recorded, the bibo:status value written at publish time, and the equivalent registration status (ISO/IEC 11179-6) and LOM status (IEEE 1484.12.1).

| State | Recorded value | bibo:status at publish | ISO/IEC 11179-6 | IEEE LOM 2.2 |
|---|---|---|---|---|
| Draft | core:Draft | bibo:status/draft | Incomplete | draft |
| Candidate | core:Candidate | bibo:status/forthcoming | Qualified | draft |
| Released | core:Released | bibo:status/published | Standard | final (first release); revised (later releases) |
| Deprecated | core:Deprecated | bibo:status/published, with owl:deprecated true | Superseded | final |
| Retired | core:Retired | None written; owl:deprecated true | Retired | unavailable |

bibo:status values are IRIs in the namespace http://purl.org/ontology/bibo/status/. Deprecated and Retired have no BIBO equivalent, so owl:deprecated carries the retirement.

### Term Type

| Value | Meaning | ISO 30042 / ISO 12620-1 data category |
|---|---|---|
| core:FullForm | Complete written form; the only type allowed as a preferred label | fullForm |
| core:Acronym | Initial letters read as a word or as letters (PEO, FMLA) | acronym |
| core:Abbreviation | Shortened form that is not an acronym (Dept.) | abbreviation |
| core:ShortForm | Truncated form of a longer term (Benefits for Employee benefits) | shortForm |

Personas, audiences, and persona labels take their values from the persona vocabulary in Section 3.8.

### Edition Conformance

Every asset header declares dcterms:conformsTo https://www.insperity.com/standard/semantic-governance/2.0/ (and BFO 2020 for ontologies). The ingestion configuration (conformsTo) sets it, and a rebuild after an edition changes updates every asset. An asset conforms to the edition it names only when it passes that edition's checks; the Operations Dashboard reports assets that declare an earlier edition (policy P07).

## Validation

- **One shape per form:** a SHACL shape for each form above (named as in Section 3.2) checks cardinality, obligation, datatypes (xsd:date for every date), language tags, and that status, persona, subject, and term-type values come from their controlled lists.
- **Conditional rules:** Deprecated requires Replaced by; Released requires Issued; an acronym listed in Appendix F requires an Acronym label; more than two broader concepts is reported.
- **Forms and gates agree:** the workbench forms apply the same rules at entry, and the shapes in the build pipeline are the authoritative gate (Appendix E, Steps 3 and 9).

# Services, Processes, and the Reference Registry

Services are what the enterprise delivers; processes are how it delivers them. Both are modeled as components that are built once and assembled, so a process shared by several services is one component, and the reach of a change to it is visible before it is made. This section sets the service and process model, the process levels, the reference registry, and the measures computed from an assembly.

## Service › Process › Process Step

- **A service is enabled by one or more processes** (core:supports). A process is built from process steps (core:hasStep), in order (core:precedes). The step that hands the service over is its core:deliveryOf. Processes, not only steps, are reusable: census collection is one process that enables two services.
- **Every process step carries its SIPOCM**, in terms of the core ontology: Supplier, Input (core:consumes), the Process step and who performs it (core:performedBy, core:fulfillsRole), Output (core:produces), Customer, and Measure (core:measuredBy).
- **Suppliers and customers inside an assembly are derived** from the data flow: the supplier of an input is whoever performs the step that produces it, and the customer of an output is whoever performs the step that consumes it. Parties outside the assembly are stated with has supplier and has customer, which specialize RO has participant.
- **Performers:** core:performedBy names a party (manual work) or a digital agent (automated work); a step is manual, automated, or hybrid.
- **Standard instances:** a step, record, or service in a component graph stands for the organization's standard step, record, or service, not one occurrence of it. Occurrences that realize them are modeled when execution data is captured (open decision).
- **Records are information assets** about the parties and offerings they concern (core:concerns). The steps that create and advance them consume and produce them, so their history stays anchored on the process step.

## Process Levels and the APQC Reference Registry

- **Levels:** the process levels of a domain are APQC's: process group (L2) › process (L3) › activity (L4), and an activity's tasks (L5) are its steps. A domain instantiates only the activities it owns and their steps; L2 and L3 are read from the registry, so one APQC process can hold activities owned by different domains without a cross-domain part-of link.
- **One registry:** the APQC Process Classification Framework (PCF 7.4) is loaded once, read-only, as the APQC PCF Taxonomy (https://www.insperity.com/taxonomy/apqc-pcf/). Each element has exactly one owner domain. Domain process kinds and PEO extensions link to it with skos:exactMatch or skos:broadMatch; other domains refer to the owner's process and never re-create it.

- **Classification:** an activity or step is classified (dcterms:type) by the APQC element it is, with its hierarchy number as skos:notation. A PEO extension is classified by the element it extends and numbered PX-….
- **Leaves and applicability:** an activity with no tasks is performed as one step. Elements a PEO does not perform (warranty, recall, packaging) are not instantiated; the registry records their applicability in a note.
- **Upgrades:** element identifiers are stable across APQC releases. A new release replaces the registry with a crosswalk; only renumbered or removed elements need attention in the assemblies. APQC names are kept as published on the registry, with attribution; domain resources carry labels that follow Part III.
- **Scope of APQC:** the PCF is used only for internal domain services. External services follow the business model (below).

## External Services and the Business Model

External services, which clients and their worksite employees receive, follow the HR Services & PEO Enterprise Business Model: Service line › Domain (L1) › Bounded context (L2) › Domain service (L3) › Business process. Its L1 domains are the HR Service Operations domains (Section 3.1). The business processes behind an external domain service are performed by internal domains, so an external service is delivered through the internal services and processes of the departments that support it. The PEO Service Map joins these services to the personas, jobs, and desired outcomes of Section 3.13, so every question a persona asks is traced to the service that answers it, and the gaps where no service does are listed.

## Assembly Components

The vocabulary, taxonomy, ontology, and shapes of a domain are its parts catalog and joining rules, one each. The components are instance graphs, each in its own named graph under …/resource/{scope}/{tier}/{id}/:

| Component | Holds |
|---|---|
| Process graph | One APQC process (L3) of a domain: its activities, their steps in order, the SIPOCM of each step, and the step measures |
| Service graph | One service: the processes that enable it, its delivery step, and what it delivers today |
| Record and party graphs | Records the steps consume and produce, and parties and roles outside the processes, shared by the domains of an area; a record one domain produces and another consumes is the hand-off between them |
| Persona graph and value map | Personas, jobs, job steps, and desired outcomes, and the bridge from process steps to job steps; a separate assembly (below) |
| Manifest | The release: the component graphs it contains and the asset versions they require (core concept Release manifest) |

- **A component refers, never copies.** Kinds are defined once, in the taxonomy; every resource is typed by a core or domain class and classified by a taxonomy concept with dcterms:type.
- **Service and persona assemblies are separate.** The service assembly never refers to the persona graph or the value map, and the build fails if it does. The value map holds the bridge from process steps to job steps (core:addresses) and the delivered outcomes.

- **Measures belong to their side:** step measures are the provider's and belong to the service assembly; metric cards of desired outcomes are the buyer's and belong to the value map, which relates the two.
- **One graph for every edit:** components are edited in GraphDB; the specification files that build them today are retired once the workbench has editing controls, and the graph is exported instead.

## Assembly Measures

| Measure | Definition | Use |
|---|---|---|
| Joins | A node's parts less one; primitives are atoms | Building block of the index |
| Assembly index (modeled pathway) | Joins in a service's closure, shared parts counted once; an upper bound on the true assembly index | Complexity of a service |
| Copy number | How many services contain a part | Shared steps and records with a high copy number are platform and automation candidates |
| Value gap | Desired outcome (target) compared with the delivered outcome (measured baseline) on the same job step and metric card; a job step without a baseline is "not yet measured", not "met" | Where to improve a service |

# Personas, Jobs, and Demand

Personas say who the Company serves; jobs say what they are trying to get done; desired outcomes say how they judge success. Together they are the measure services are held to, and the source of demand for new work. This section sets the model and the governed path from research to demand planning, which the Customer 360 pages implement.

## The Jobs Model

- **The persona model is supported by the jobs model.** Jobs are a type of activity, kept in their own taxonomy because they are not process models. A job is unique to the job executor; it gains contextual relevance when combined with the persona executing it.
- **Job kinds** are functional and emotional (social jobs are recorded with them), evaluated in the context of the job. A job that involves money is associated with finance as a subject, not a kind. Core and related jobs are a second facet.
- **Job steps are akin to process steps but intentionally not one-to-one:** job steps describe what the customer is trying to get done; process steps describe how the organization does its work. The gap between them is where value gaps arise.
- **Job map stages:** the eight universal stages of a job map are concepts (jtbd:job-map-stage); steps people describe in interviews are job map steps (jtbd:job-map-step, a core:job-step).
- **Desired outcomes** are mapped to job steps and written as a direction (minimize or maximize), a metric, an object, and a clarifier, with a metric card, a target, and a baseline.
- **Buying stages are the provider's view** (Discover, Qualify, Evaluate, Propose, Decide, Onboard): steps of the selling process. They may correspond to one or more of a persona's jobs or job steps, and must not be confused with them.

| Customer 360 class | Aligned to |
|---|---|
| Client persona | core:persona |
| Job to be done | core:job |
| Job map step | core:job-step |
| Desired outcome | core:desired-outcome |
| Outcome target | core:measure |
| Demand request | core:requirement |
| Interview notes and excerpts | core:information-asset |
| Reviewer, item review, review decision | core:person, core:information-asset, core:decision |

## From Research to Demand

1. **Research:** persona interviews follow the appreciative inquiry cycle (Discover, Dream, Design, Destiny). A passage is coded as a functional, emotional, or social job, a job step, or a desired outcome, as a new candidate or as evidence for an existing item, and published to its own named graph (Appreciative Inquiry).

2. **Measure:** an outcome-driven survey scores the importance and satisfaction of each desired outcome; every Customer 360 page uses the same scores (PEO Job Study).
3. **Review:** business and compliance subject-matter experts review every persona, job, and outcome (verdict, wording, risk flags, approved uses), and a decider records the final decision. Reviews are shared through GraphDB, one named graph per reviewer and per decision (PEO Outcome Review).
4. **Act:** reviewed outcomes drive battle cards, the Fix / Innovate / Protect / Simplify action matrix, and the sales advisor agent (PEO Outcome Playbook), and the PEO Service Map shows which service answers each question.
5. **Demand:** under-served outcomes are bundled into demand requests and admitted to demand planning only when they are traced, reviewed, evidenced, measurable, owned, and unflagged. Requests are written back to the graph and become the requirements that change records implement (Demand Gateway; core:implementsFeature, Section 3.3).

## Rules

- **No protected attributes:** personas, jobs, and outcomes describe work, never age, gender, ethnicity, health, or other protected or sensitive characteristics (Section 3.8).
- **Evidence before authority:** a candidate job or outcome becomes authoritative only after review; risk flags and approved uses travel with it to every consumer.
- **Standards apply:** Customer 360 ontologies and schemes carry the Section 3.11 header fields and pass the Part III checks.

APPENDIX A

# Finance (FP&A) Domain

An illustrative, end-to-end example of the framework applied to Financial Planning & Analysis: domain model and ontology, requirements and change management, LKIF mappings, vector databases, sequence diagrams, use cases, and MCP integration.


A.1 Finance Domain
A.2 FP&A Ontology
A.3 FP&A Ontology: Process-Centric Model
A.4 Ontology Requirements Specification (ORSD)
A.5 KGCL-to-ORSD Mapping
A.6 KGCL-to-ORSD Mapping (Detailed)
A.7 Mapping FP&A Metrics to LKIF Statements
A.8 Mapping FP&A Classes to LKIF Statements
A.9 Scenario Mapping in Vector Databases
A.10 Vector Database Integration with Ontologies
A.11 FP&A Sequence Diagrams
A.12 FP&A Use Cases
A.13 MCP and Microservices Integration

# Finance Domain

A functional breakdown for the finance domain involves separating it into its core functions: accounting, financial planning and analysis (FP&A), treasury, tax, and risk and compliance. These functions work together to ensure a company's financial health, support strategic decisions, and maintain compliance.

## Accounting and reporting

This function is responsible for the systematic recording, summarizing, and reporting of an organization's financial transactions to create a verifiable record of its financial position.

- General ledger (GL) management: Maintains the company's master record of financial transactions, ensuring accurate classification and aggregation.
- Accounts payable (AP): Processes and manages all money the company owes to its vendors and creditors.
- Accounts receivable (AR): Manages the collection of payments from customers and clients for goods or services rendered.
- Payroll: Ensures employees are paid accurately and on time, including managing salaries, wages, bonuses, and deductions.
- Financial statement preparation: Prepares and presents financial reports such as the balance sheet, income statement, and statement of cash flows.
- Financial controls: Implements and monitors internal controls to protect company assets and prevent fraud.

## Financial planning and analysis (FP&A)

FP&A supports strategic decision-making by analyzing financial data and forecasting future performance.

- Budgeting: Develops the annual operating budget, projecting revenues and expenses for the coming year.
- Forecasting: Creates projections of future financial performance to help management anticipate trends and potential challenges.
- Performance reporting: Analyzes and reports on financial performance against budgets and forecasts, calculating variance and identifying key business drivers.
- Financial modeling: Builds "what-if" scenarios and models to evaluate potential investments, business strategies, and capital structure changes.
- Business partnering: Works directly with other departments to align financial plans with business unit strategies.

## Treasury

This function manages the organization's capital and liquidity, ensuring there is enough cash to meet daily obligations while maximizing financial gain from its cash holdings.

- Cash management: Monitors and manages the company's daily cash flow and banking relationships.
- Capital structure management: Manages the balance of debt and equity used to finance the business.
- Financing: Raises capital through loans, credit facilities, or equity issuance to fund operations and expansion.
- Investment management: Oversees the investment of excess cash in short-term financial instruments to generate returns.
- Financial risk management: Identifies and mitigates financial risks, such as foreign exchange, interest rate, and credit risk.

## Tax and compliance

This group handles all tax-related matters and ensures the company adheres to all legal and regulatory financial standards.

- Tax planning and compliance: Develops strategies to minimize tax liabilities while ensuring compliance with all tax laws.
- Regulatory reporting: Files all required tax returns and financial statements with relevant governmental authorities.
- Internal audit and controls: Conducts audits to ensure the integrity of financial reporting and adherence to internal policies and controls.
- International tax management: Manages tax obligations for companies operating in multiple countries, which adds complexity due to different international tax laws.

## Strategic and executive functions

Led by the Chief Financial Officer (CFO), this area oversees the entire finance domain and provides strategic direction to the company.

- Executive leadership: The CFO provides high-level financial strategy to the CEO and the board of directors.
- Mergers and acquisitions (M&A): Identifies and evaluates opportunities for corporate development, such as buying, selling, or merging with other companies.
- Investor relations: Manages communication with investors, shareholders, and financial analysts.
- Stakeholder management: Serves as a key liaison for external parties, including auditors, banks, insurers, and regulators.

# FP&A Ontology

This `.trig` file defines the FP&A ontology classes for processes, assets, parties, events, issues, and measures, while incorporating legal constraints via LKIF core norms. It references FIBO for process and measurement concepts and links LKIF legal constraints as regulatory obligations applicable to FP&A processes.

This ontology provides a standards-compliant semantic scaffold to support agentic AI, vector DB integration, and advanced FP&A analytics with regulatory compliance baked-in. Further expansion can model more fine-grained sub-processes and domain-specific legal norms.

```
@prefix fpaf: < http://example.org/fpaf# > .
@prefix fibo-proc: < https://spec.edmcouncil.org/fibo/ontology/BE/Process/Process/ > .
@prefix fibo-meas: < https://spec.edmcouncil.org/fibo/ontology/BE/Measurement/Measurement/ > .
@prefix lkif-core: < http://www.estrellaproject.org/lkif-core/lkif-core.owl# > .
@prefix owl: < http://www.w3.org/2002/07/owl# > .
@prefix rdfs: < http://www.w3.org/2000/01/rdf-schema# > .
@prefix xsd: < http://www.w3.org/2001/XMLSchema# > .
# FP&A Ontology Core Classes
fpaf:financial-planning-process a owl:Class ;
  rdfs:subClassOf fibo-proc:process ;
  rdfs:label "Financial Planning Process"@en ;
  rdfs:comment "A process for strategic planning, budgeting, forecasting, and analysis in
finance."@en .
fpaf:budgeting-process a owl:Class ;
  rdfs:subClassOf fpaf:financial-planning-process ;
  rdfs:label "Budgeting Process"@en .
fpaf:forecasting-process a owl:Class ;
  rdfs:subClassOf fpaf:financial-planning-process ;
  rdfs:label "Forecasting Process"@en .
fpaf:performance-reporting-process a owl:Class ;
  rdfs:subClassOf fpaf:financial-planning-process ;
  rdfs:label "Performance Reporting Process"@en .
fpaf:financial-modeling-process a owl:Class ;
  rdfs:subClassOf fpaf:financial-planning-process ;
  rdfs:label "Financial Modeling Process"@en .
fpaf:business-partnering-process a owl:Class ;
  rdfs:subClassOf fpaf:financial-planning-process ;
  rdfs:label "Business Partnering Process"@en .
# Assets
fpaf:financial-asset a owl:Class ;
  rdfs:label "Financial Asset"@en ;
  rdfs:comment "Assets such as budgets, forecasts, reports, models."@en .
# Parties
fpaf:finance-party a owl:Class ;
  rdfs:label "Finance Party"@en ;
  rdfs:comment "Participants such as finance teams, business units, management."@en .
# Events
fpaf:financial-event a owl:Class ;
  rdfs:label "Financial Event"@en .
# Issues
fpaf:financial-issue a owl:Class ;
```

```
  rdfs:label "Financial Issue"@en ;
  rdfs:comment "Issues such as variances, data discrepancies, compliance risks."@en .
# Measures (from FIBO Measurement Ontology)
fpaf:variance a owl:Class ;
  rdfs:subClassOf fibo-meas:measurement ;
  rdfs:label "Variance"@en .
fpaf:key-performance-indicator a owl:Class ;
  rdfs:subClassOf fibo-meas:measurement ;
  rdfs:label "Key Performance Indicator"@en .
fpaf:confidence-interval a owl:Class ;
  rdfs:subClassOf fibo-meas:measurement ;
  rdfs:label "Confidence Interval"@en .
# Legal and Regulatory Constraints using LKIF Core
fpaf:legal-constraint a owl:Class ;
  rdfs:subClassOf lkif-core:norm ;
  rdfs:label "Legal Constraint"@en ;
  rdfs:comment "Legal constraints and obligations applicable to FP&A processes."@en .
# Properties
fpaf:hasAsset a owl:ObjectProperty ;
  rdfs:domain fpaf:financial-planning-process ;
  rdfs:range fpaf:financial-asset ;
  rdfs:label "has asset"@en .
fpaf:hasParticipant a owl:ObjectProperty ;
  rdfs:domain fpaf:financial-planning-process ;
  rdfs:range fpaf:finance-party ;
  rdfs:label "has participant"@en .
fpaf:triggersEvent a owl:ObjectProperty ;
  rdfs:domain fpaf:financial-planning-process ;
  rdfs:range fpaf:financial-event ;
  rdfs:label "triggers event"@en .
fpaf:hasIssue a owl:ObjectProperty ;
  rdfs:domain fpaf:financial-planning-process ;
  rdfs:range fpaf:financial-issue ;
  rdfs:label "has issue"@en .
fpaf:measures a owl:ObjectProperty ;
  rdfs:domain fpaf:financial-planning-process ;
  rdfs:range fibo-meas:measurement ;
  rdfs:label "measures"@en .
fpaf:isSubjectTo a owl:ObjectProperty ;
  rdfs:domain fpaf:financial-planning-process ;
  rdfs:range fpaf:legal-constraint ;
  rdfs:label "is subject to legal constraint"@en .
# Example individuals (instances)
fpaf:budget-2026 a fpaf:financial-asset ;
  rdfs:label "2026 Annual Budget"@en .
fpaf:chief-financial-officer a fpaf:finance-party ;
  rdfs:label "Chief Financial Officer"@en .
fpaf:budget-approved-event a fpaf:financial-event ;
  rdfs:label "Budget Approved Event"@en .
fpaf:variance-issue a fpaf:financial-issue ;
  rdfs:label "Budget Variance Issue"@en .
fpaf:budgeting a fpaf:budgeting-process ;
  fpaf:hasAsset fpaf:budget-2026 ;
  fpaf:hasParticipant fpaf:chief-financial-officer ;
```

```
  fpaf:triggersEvent fpaf:budget-approved-event ;
  fpaf:hasIssue fpaf:variance-issue ;
  fpaf:measures fpaf:variance ;
  fpaf:isSubjectTo < http://example.org/legal#SarbanesOxleyAct > .
# Semantic imports
<> owl:imports fibo-proc: .
<> owl:imports fibo-meas: .
<> owl:imports lkif-core: .
```

# FP&A Ontology: Process-Centric Model

This section describes a structured FP&A ontology aligned with the mentioned core services, built around a process-centric domain reference model, and mapping to assets, parties, events, issues, and measures. The ontology adopts standards from FIBO, LKIF, and the enterprise modeling principles outlined in the references, positioning processes as the first-class citizens.

## FP&A Ontology: Process-Centric Domain Reference Model

## Core Classes and Relationships

- Process Class (First Class)
  - *Financial Process*: Budgeting, Forecasting, Performance Reporting, Financial Modeling, Business Partnering
  - *Associated Assets*: Financial Statements, Budgets, Forecast Reports, Models, Performance Metrics
  - *Parties*: Finance Department, Business Units, Executive Management, External Regulators
  - *Events*: Budget Submission, Forecast Updates, Variance Reports, Strategic Reviews, Model Simulations
  - *Issues*: Budget Overruns, Forecast Deviations, Data Discrepancies, Model Assumptions, Stakeholder Conflicts
  - *Measures*: Variance, Accuracy, Confidence Interval, KPIs, Scenario Outcomes

## Process-Asset-Party-Event Mapping

| Service | Core Process | Assets | Parties | Events | Issues | Measures |
|---|---|---|---|---|---|---|
| Financial Planning & Analysis | Developmen ts of Plans and Forecasts | Financial Statements , Budget Plans | Finance Team, Business Units | Budget Submissio n, Forecast Revision | Data Inaccuracy, Misalignment | Variance, KPI, Confidenc e |
| Budgetin g | Budget Developmen t | Budget Reports, Projections | CFO, Departmen t Heads, CFO | Budget Approval, Submissio n | Underfunding, Over-commit ment | Budget Variance, Cost Savings |
| Forecasti ng | Create and Update Projections | Forecast Documents , Trend Data | CFO, Strategic Planners | Forecast Revision, External Data Updates | Assumption Errors, External Shocks | Forecast Error, Confidenc e |
| Performa nce Reporting | Variance and KPI Reporting | Variance Reports, Performan ce Dashboard s | Manageme nt, Stakeholde rs | Report Generation , Review Cycles | Data Discrepancies, Misinterpretati on | Variance %, KPI Achievem ent |
| Financial Modeling | Scenario & "What-If" Simulation | Scenario Models, Investment Models | Analysts, Strategic Planners | Scenario Runs, Model Adjustmen ts | Model Assumptions, Data Gaps | Model Accuracy, Scenario Outcome s |

| Service | Core Process | Assets | Parties | Events | Issues | Measures |
|---|---|---|---|---|---|---|
| Business Partnerin g | Strategy Alignment | Collaborati on Plans, Meeting Minutes | Business Units, Executives | Strategy Sessions, Feedback Loops | Misalignment, Stakeholder Resistance | Alignment Score, Issue Resolutio n |

## Standardized References

- Use FIBO for financial instruments, KPIs, and process concepts.
- Integrate LKIF for legal and regulatory constraints affecting processes.
- Follow OWL/RDF principles and OMG core ontologies for formalizing the model.
- Ensure alignment with ISO/IEC/IEEE 24765 terminology and enterprise reference ontologies for consistency and interoperability.

# Ontology Requirements Specification (ORSD)

## About This Specification

The Ontology Requirements Specification Document (ORSD) is the requirements template defined by the **NeOn Methodology** for building ontology networks (Suárez-Figueroa, Gómez-Pérez and Villazón-Terrazas, 2009). It is a widely adopted community methodology rather than an ISO standard; no ISO standard prescribes a template for ontology requirements. The version in this section extends the NeOn template with document control, change-management, and release sections, so the same document governs both what the ontology must do and how it evolves.

### How the Template Is Organized

- **Purpose, scope, and implementation language:** why the ontology exists, what it covers, and the formal language used (here, OWL 2 DL).
- **Intended end-users and uses:** who will rely on the ontology and for what tasks.
- **Functional requirements as competency questions:** the questions the ontology must be able to answer, such as "Which cost centers exceeded budget by more than 10%?".
- **Non-functional requirements:** performance, maintainability, versioning, and security expectations.
- **Pre-glossary of terms:** the key terms extracted from the requirements, which seed the ontology's vocabulary.
- **Extensions used here:** document control, a change management and release plan, and validation and verification criteria.

### Related ISO Standards

| Standard | Scope | Relevance to the ORSD |
|---|---|---|
| ISO/IEC/IEEE 29148 | Requirements engineering | Quality criteria for requirements (unambiguous, verifiable, traceable) that ORSD requirements and competency questions should meet |
| ISO 704 and ISO 1087 | Terminology work: principles and vocabulary | How the glossary defines concepts and terms consistently |
| ISO/IEC 21838 | Top-level ontologies | Requirements for upper ontologies, relevant to the core ontologies the FP&A ontology extends |
| ISO/IEC/IEEE 24765 | Systems and software engineering vocabulary | Consistent engineering terminology, cited elsewhere in this document |

In practice, each competency question becomes a test case (typically a SPARQL query) in the validation criteria (item 10 below). That makes ORSD requirements verifiable in the sense of ISO/IEC/IEEE 29148.

# FP&A Ontology Requirements

## Ontology Requirements and Release Specification Document

## FP&A Sub-domain in Finance Domain

### 1. Document Control

- Version: 1.0
- Date: 2025-10-21
- Authors: [Names of financial domain experts, ontology engineers]
- Reviewers: [Stakeholders from finance, compliance, IT]
- Approvals: [Governance committee]

### 2. Ontology Purpose and Goals

- To formally represent FP&A concepts including budgeting, forecasting, variance analysis, financial reporting, and driver-based planning.
- Enable consistent financial data interpretation across systems for enhanced decision-making and reporting accuracy.
- Support integration with broader corporate financial ontologies and regulatory compliance frameworks.

### 3. Scope

- Coverage of FP&A key processes: Budget Allocation, Financial Forecasting, Performance Metrics, Variance Analysis, Cash Flow Projections.
- Modeling of FP&A roles (analysts, managers), financial instruments, cost centers, and drivers.
- Excludes detailed accounting transactions and ledger-level details (covered in accounting ontology).

### 4. Implementation Language and Standards

- OWL 2 DL for formal ontology modeling.
- Conformance with Financial Industry Business Ontology (FIBO) where applicable.
- Use existing taxonomy standards for financial measures and KPIs (e.g., IFRS KPIs).

### 5. Intended End-Users and Stakeholders

- FP&A professionals and financial analysts for enhanced data-driven insights and planning.
- IT architects and application developers for integration with ERP and planning systems.
- Compliance officers monitoring financial governance and control.

### 6. Functional Requirements

- Ability to express budgeting cycles and forecast revisions.
- Support competency questions such as: "What is the forecasted revenue for Q4 2025?" or "Which cost centers exceeded budget by more than 10%?"
- Trace financial drivers impacting performance metrics and variances.
- Link financial plans to actual results and identify discrepancies.

### 7. Non-functional Requirements

- Ontology must support scalable real-time querying over large financial data sets.
- Must be maintainable with clear versioning aligned to financial period releases.
- Secure access control for sensitive financial concepts and data.

### 8. Glossary and Terminology

- Budget: Planned allocation of financial resources over a period.
- Forecast: Projections of future financial performance based on current data.
- Variance: Difference between budget and actuals.
- KPI: Key Performance Indicator measuring financial performance.

### 9. Change Management and Release Plan

- Changes to FP&A ontology governed via ORSD/KGCL integration for traceability.
- Version aligned with corporate financial periods (quarterly/annual).
- Reviews by finance and compliance teams prior to production deployment.

### 10. Validation and Verification Criteria

- Validation via competency question test cases and financial reporting scenario simulations.
- Logical consistency checked using ontology reasoners.
- Performance testing on query response times and concurrency.

### 11. Appendices

- References: FIBO, IFRS standards, company-specific financial policies.
- Sample data models for budgeting and forecasting instances.
- Contact information for ontology stewards and FP&A domain experts.

# KGCL-to-ORSD Mapping

## What Is KGCL?

The **Knowledge Graph Change Language (KGCL)** is an open standard for describing changes to ontologies and knowledge graphs. It was developed by Chris Mungall and collaborators in the OBO ontology community and published in the journal Database in 2025. Each change is a single, human-readable command (rename, obsolete, add a synonym, create a class, move a term under a new parent) backed by a formal data model written in LinkML. The same change can therefore be read by people and applied by software.

### How KGCL Works

- **Controlled natural language:** commands such as rename X from "A" to "B" follow a defined grammar, so they are precise but readable.
- **Machine-applicable:** tools such as the Ontology Access Kit (OAK) and the ROBOT KGCL plugin apply commands directly to OWL ontologies, and can compute the KGCL changes between two ontology versions.
- **Workflow-friendly:** in the OBO community, change requests written as KGCL in GitHub issues are turned into pull requests by an automated agent (Ontobot) and merged after curator review.
- **Durable change log:** changes can be stored as text, JSON, YAML, RDF, or tables, giving a complete, replayable history.

| Change | Example (as used in this appendix) | Effect |
|---|---|---|
| Rename | rename FP&A:BudgetAllocation from "Budget Allocation" to "Budget Distribution" | Changes a term's label; the identifier stays the same |
| Synonym | add synonym FP&A:CashFlowProjection "Liquidity Forecast" | Adds an alternative name for search and cross-team use |
| Obsolete | obsolete FP&A:DriverBasedPlanning | Deprecates a term without deleting it, optionally pointing to a replacement |
| Subclass | add subclass FP&A:VarianceAnalysis FP&A:FinancialAnalysis | Adds a parent (subclass-of) relationship |

The examples follow the wording used in this document. KGCL's formal grammar names some commands slightly differently (for example, create synonym and create edge), so templates should follow the published grammar when they are automated.

### How KGCL and the ORSD Work Together

The two records answer different questions. The ORSD (Section A.4) records **why** and **under what governance** the ontology changes: requirements, rationale, impact, approvals, and the release plan. KGCL records **exactly what** changed, in a form a machine can apply. Every approved change therefore has two linked records:

- An ORSD change entry with description, reason, impact, approval, and verification (ORSD item 9, Change Management and Release Plan).

- A KGCL statement that implements the change, applied to the ontology in the knowledge graph and versioned with the release.

The table below shows this pairing for several FP&A changes, and Section A.6 traces one change through every ORSD section it affects. Together they give an auditable trail from business intent to executable change.

## KGCL-to-ORSD Mapping for FP&A Changes

The following example maps KGCL change commands to ORSD change documentation for FP&A ontology updates:

| KGCL Change Statement | Corresponding ORSD Entry |
|---|---|
| rename FP&A:BudgetAllocation from "Budget Allocation" to "Budget Distribution". | Change Description: Renaming of BudgetAllocation concept to Budget Distribution to better reflect usage in variance analysis. Reason: Clarify terminology aligning with business glossary. Impact: All budgeting workflows and reports referencing this term will update accordingly. Approval: FP&A Committee, IT Governance. |
| add synonym FP&A:CashFlowProjection "Liquidity Forecast" | Change Description: Added synonym "Liquidity Forecast" to CashFlowProjection concept for improved cross-team understanding. Reason: Multiple teams use different terminology for same concept. |
| | Verification: Verify through end-user review and system search. |
| obsolete FP&A:DriverBasedPlanning | Change Description: Obsolete DriverBasedPlanning concept due to redesign of driver logic in updated financial model. Reason: Deprecated in favor of FP&A:FinancialDriverAnalysis concept introduced in latest fiscal cycle. Version: Effective from 2025 Q4 release. |
| add subclass FP&A:VarianceAnalysis FP&A:FinancialAnalysis | Change Description: Added VarianceAnalysis as subclass of FinancialAnalysis to support detailed variance reporting scenarios. Reason: New reporting requirements for variance details in FP&A dashboards. Testing: Included in FP&A scenario testing with validation on sample datasets. |

This mapping shows how high-level change intents documented in ORSD (rationale, impact, approval) directly correspond with executable KGCL commands that modify the

ontology accordingly. This dual documentation ensures traceability, governance, and automation across the FP&A ontology lifecycle.

Such structured change management improves accuracy and communication around ontology evolution while supporting automated application and versioning in GraphDB or other knowledge graph platforms.

This approach can be expanded to include detailed competency questions, validation criteria, change request identifiers, and stakeholder comments linked bi-directionally between KGCL and ORSD artifacts.

# KGCL-to-ORSD Mapping (Detailed)

This section follows a single KGCL command, the rename of BudgetAllocation, through each ORSD section it affects. Section A.5 explains KGCL and how it pairs with the ORSD.

## Mapping an FP&A KGCL Change to ORSD Sections with Rationale (Example)

**KGCL Change:**

```
rename FP&A:BudgetAllocation from "Budget Allocation" to "Budget
Distribution"
```

**Mapped ORSD Sections:**

1. Change Management and Release Plan
   - Document this rename as a formal ontology change in the release plan.
   - Ensure versioning captures this change aligned with the next financial reporting period.
2. Change Description (part of Functional Requirements or Change Management)
   - Define the change clearly: renaming "Budget Allocation" to "Budget Distribution".
3. Rationale
   - Clarify that the rename reflects improved alignment with financial planning terminology and variance analysis use cases as gathered from FP&A stakeholders.
4. Impact and Stakeholders
   - Identify all reporting workflows, analyses, and systems impacted by this concept rename.
   - Note relevant users and governance approvals necessary for the change.
5. Validation and Verification Criteria
   - Outline planned tests and validation to ensure that all references to "Budget Allocation" update correctly and do not break queries or reports.
6. Glossary and Terminology
   - Update glossary with the new preferred term "Budget Distribution" and mark "Budget Allocation" as deprecated or synonym as applicable.

## Summary

This KGCL change represents a controlled, traceable ontology modification. It should be documented in the ORSD change management sections to capture why and how the vocabulary evolves, its impact on FP&A processes, and the verification steps ensuring smooth adoption across the organization. This linkage supports transparent governance and automated lifecycle management using KGCL and GraphDB.

# Mapping FP&A Metrics to LKIF Statements

## What Is LKIF?

The **Legal Knowledge Interchange Format (LKIF)** was developed in the EU-funded ESTRELLA project as a standard way to represent and exchange legal knowledge such as legislation, regulations, and cases. Its centerpiece, the **LKIF Core** ontology, is written in OWL and defines basic legal concepts in modules, including norms (obligations, permissions, prohibitions), actions, roles, processes, time, and place. LKIF combines OWL with rules (SWRL) so that legal conditions can be evaluated automatically.

In this appendix, LKIF supplies the vocabulary for stating that an FP&A metric or process is subject to a norm. For example, a forecast carries an obligation to stay within a variance threshold, so a reasoner can check compliance instead of a person.

### LKIF and ISO Standards for Legal and Compliance Matters

LKIF is not itself an ISO standard, and ISO does not publish a legal ontology. LKIF's role is to make the obligations that ISO management-system standards ask organizations to manage machine-readable and checkable:

| Standard | Focus | How LKIF-based models support it |
|---|---|---|
| ISO 37301 (compliance management systems) | Identifying, maintaining, and monitoring compliance obligations | Each obligation is an LKIF norm linked to the processes, roles, and metrics it governs, forming a traceable obligations register |
| ISO 31000 (risk management) | Identifying, assessing, and treating risk | Norm violations and sanctions are modeled explicitly, so compliance risk can be queried and reported |
| ISO/IEC 42001 (AI management systems) | Responsible governance of AI systems | Agents check proposed actions against LKIF norms before acting, producing auditable decisions |
| ISO 704 and ISO 1087 (terminology) | Consistent definition of concepts and terms | LKIF concepts are formally defined, keeping legal terms consistent across systems |
| OASIS LegalRuleML and Akoma Ntoso | Open standards for legal rules and legal documents | Carry the source legal texts and rules that LKIF norms reference (for example, through governedBy links) |

External regulations, such as Sarbanes-Oxley Section 404 in the example below, are modeled as the sources of LKIF norms. This keeps each FP&A control traceable to the regulation that requires it.

## Mapping FP&A Metrics to Deontic Statements

Mapping FP&A metrics to LKIF deontic statements, creating semantic links to model obligations, permissions, and prohibitions about financial performance and compliance:

## FP&A Metrics

- Variance: Difference between budgeted and actual values.
- Forecast Accuracy: How close forecasts match actual results.

- Key Performance Indicators (KPIs): Quantitative measures of business performance.
- Scenario Variance: Difference between scenario assumptions and realized results.
- Cash Conversion Cycle: Operational efficiency metric measuring cash flow dynamics.

## LKIF Deontic Statements (Norms)

- Obligation: A duty to achieve a target metric or stay within thresholds (e.g., "variance must be under 5%").
- Permission: Allowed variance range or forecasting tolerance.
- Prohibition: Forbidden deviations or actions (e.g., misstating forecast).
- Violation: Breach when KPIs or metrics fall outside allowed bounds.
- Sanction: Consequences tied to violation of financial norms or controls.

## Property Mappings

| FP&A Metric | LKIF Deontic Link | Description |
|---|---|---|
| Variance | lkif:obligation | Mandates variance within acceptable limits |
| Forecast Accuracy | lkif:permission | Allows specific accuracy range for forecasting |
| KPIs | lkif:obligation / lkif:prohibition | Defines must-meet performance and forbidden outcomes |
| Scenario Variance | lkif:violation | Flags when actual outcomes deviate beyond scenario assumptions |
| Cash Conversion Cycle | lkif:obligation | Imperative to maintain operational cash flow efficiency |

This mapping enables agentic AI systems to reason over financial metrics semantics linked to formal deontic constraints, automating compliance checks, alerts, and governance decision support in FP&A.

Such integration wraps quantitative FP&A monitoring within a normative framework built on LKIF, improving regulatory alignment and process transparency.

```
@prefix fpaf: < http://example.org/fpaf# > .
@prefix lkif-core: < http://www.estrellaproject.org/lkif-core/lkif-core.owl# > .
@prefix owl: < http://www.w3.org/2002/07/owl# > .
@prefix xsd: < http://www.w3.org/2001/XMLSchema# > .
# Class Definitions
fpaf:financial-forecast a owl:Class ;
  rdfs:label "Financial Forecast" ;
  rdfs:comment "A predicted financial outcome over a planning horizon." .
lkif-core:obligation a owl:Class ;
  rdfs:label "Obligation" ;
  rdfs:comment "A deontic statement imposing a duty or requirement." .
lkif-core:permission a owl:Class ;
  rdfs:label "Permission" ;
  rdfs:comment "A deontic statement granting permission." .
# Instances
fpaf:q4-forecast-2025 a fpaf:financial-forecast ;
```

```
  rdfs:label "Q4 2025 Financial Forecast" ;
  fpaf:hasValue "1000000"^^xsd:decimal ;
  fpaf:forecastDate "2025-10-01"^^xsd:date .
# Linking forecast to legal rules
fpaf:q4-forecast-2025 lkif-core:isSubjectTo [
  a lkif-core:obligation ;
  lkif-core:imposesOn fpaf:q4-forecast-2025 ;
  lkif-core:hasCondition "Forecast variance must be less than 5%"^^xsd:string ;
  lkif-core:relatedNorm "SOX Compliance Section 404"^^xsd:string
] .
fpaf:q4-forecast-2025 lkif-core:isSubjectTo [
  a lkif-core:permission ;
  lkif-core:permitsOn fpaf:q4-forecast-2025 ;
  lkif-core:hasCondition "Adjustments allowed within 2% forecast range"^^xsd:string
] .
```

- The `FinancialForecast` instance `Q4Forecast2025` is linked to LKIF deontic statements of `Obligation` (must keep variance under 5%, e.g., Sarbanes-Oxley compliance) and `Permission` (allowed forecast adjustments).
- `lkif-core:isSubjectTo` connects the forecast to applicable normative statements.
- This structure allows automated reasoning over FP&A forecasts with legal compliance constraints, supporting auditability and decision support.

This example can be extended with more specific predicates and property values, and integrated with vector and agentic AI knowledge scaffolds to enforce real-time compliance and governance.

# Mapping FP&A Classes to LKIF Statements

Building on Section A.7, this section maps FP&A classes onto the LKIF Core modules, so every financial process, party, and event can be linked to the norms that govern it.

Core FP&A classes and properties mapped to LKIF legal constraints, based on LKIF's modular legal ontology structure, focus on integrating FP&A financial processes with legal norms, roles, and actions.

## Core FP&A Classes to Map

- FinancialPlanningProcess (extends LKIF's Process)
- BudgetingProcess, ForecastingProcess, PerformanceReportingProcess, FinancialModelingProcess, BusinessPartneringProcess (subclasses)
- FinancialAsset (documents, reports, budgets)
- FinanceParty (roles in LKIF legal-role: finance officers, auditors)
- FinancialEvent (actions or triggers linked to LKIF Action or LegalAction)
- FinancialIssue (potentially mapped to LKIF Norm Violations or Legal Issues)
- LegalConstraint (norms, obligations, constraints from LKIF Norm module)

## Key Properties (object and data)

- hasParticipant (link Process to LKIF Role/Party involved)
- triggersEvent (Process → Event)
- hasIssue (Process → Issue related to legal non-compliance or risks)
- isSubjectTo (Process → LegalConstraint from LKIF Norm)
- measures (Process → Measures e.g., Variance, KPIs)
- governedBy (link FinancialAsset or Process to specific LKIF legislation or source documents)

## Mapping Rationale

- LKIF Process Module models causal financial activities (budgeting, forecasting) as Legal Processes.
- LKIF Role Module defines finance participants' legal roles (e.g., fiduciary duty, auditor role).
- LKIF Action Module captures compliance actions and reporting events.
- LKIF Norm Module defines obligations, permissions, and prohibitions applicable to FP&A processes (e.g., SOX compliance).
- Using these mappings supports semantic reasoning about legal constraints on financial processes, automating compliance checks, and linking regulations to operational FP&A activities.

This approach leverages LKIF's modular ontology design for clear legal-semantic integration, critical for auditability and governance of FP&A workflows.

# Scenario Mapping in Vector Databases

A process-centric approach to mapping scenarios in vector databases aligns with best practices in semantic data integration and knowledge engineering. Making processes first-class citizens yields several advantages:

1. Contextual Grounding: Processes provide meaningful context around scenarios, enabling vector embeddings to be semantically anchored rather than floating in isolation. This alignment improves precision in similarity search and recall by factoring in process role and workflow dependencies.
2. Structured Semantic Scaffolding: Mapping vectorized scenarios via domain-specific ontologies centered on processes helps preserve rich relationships between entities, events, assets, and parties. This ensures vector searches respect underlying business logic and ontological constraints.
3. Interoperability and Versioning: Anchoring scenario mappings to formal process instances enables consistent version control, change impact analysis, and lifecycle management using standards-based approaches (e.g., OMG's ODM and DOL).
4. Cross-Domain Integration: The approach supports multi-process and

   cross-ontology scenario mapping, essential for complex enterprise environments where scenarios span functions or business units.
5. Hybrid Reasoning and Querying: This foundation facilitates hybrid search strategies combining vector similarity with symbolic reasoning, maximizing relevancy and interpretability for agentic AI applications.

The approach is practical and conceptually robust, facilitating semantically rich, scalable, and adaptable scenario management in vector-backed knowledge systems. Consider incorporating ongoing validation with stakeholders and iterative alignment of vector embeddings with process semantics to optimize outcomes further.

Yes, inserting a lightweight scaffold into the vector database to maintain lineage back to the richer context in the ontologies for two-level search by domain-specific agents is a sensible and effective strategy.

## Reasons:

1. Efficient Indexing & Querying:

- The scaffold acts as a semantic pointer or anchor inside the vector DB, enabling fast approximate similarity searches at the vector level.
  - The detailed ontological context remains in the richer ontology store, referenced as needed for precise symbolic reasoning.
  - This preserves performance by avoiding heavy semantic reasoning at query time over the entire dataset.

2. Lineage and Traceability:

- Embedding process and domain identifiers in the vector scaffold allows each vector entry to be traced back to exact ontology elements (processes, scenarios, legal constraints, assets).

    - This traceability supports explainability and audit requirements, crucial in regulated domains.

3. Stable Backbone with Extensible Flexibility:

- Assuming domains and processes are stable, this scaffold provides a persistent anchoring schema that reduces re-indexing needs when scenario data changes.
    - The scaffold structure can be incrementally evolved alongside ontological updates, supporting versioning and evolution without major disruption.

4. Hybrid 2-Level Search:

- The vector DB performs semantic similarity or approximate nearest neighbor search using lightweight embeddings.
    - Domain-specific agents retrieve richer knowledge via ontology references for confirmatory reasoning, validation, or complex queries.
    - This hybrid approach optimizes speed and accuracy—vector DB filters candidates, then ontology-enabled agents handle interpretation and compliance checks.

5. Best Practice Alignment:

- Research and practice in hybrid graph-vector database systems view ontologies as providing semantic grounding, while vector indexes enable efficient search over unstructured or semi-structured data.
    - A lightweight scaffold in vector DB aligns with this best practice, capturing just enough context for mapping without duplication of the full ontological logic inside vectors.

## Conclusion

Inserting a lightweight, process- and domain-tagged scaffold inside vector DBs with explicit references back to rich, versioned ontologies is a practical and effective design. It balances query performance, semantic integrity, and flexibility, enabling two-level searches orchestrated by domain-specific agents that optimize relevancy, explanation, and governance

The following domain-specific vector database schema for FP&A scenarios focuses on capturing vector embeddings, minimal semantic scaffolding for mapping, and supporting metadata for lineage and versioning. The schema uses typical data types for vector storage and supports linking to domain-specific ontological identifiers. `sql`

```
CREATE TABLE fpaf_vectors (
    id SERIAL PRIMARY KEY ,
    process_instance_uri VARCHAR ( 512 ) NOT NULL, -- URI for stable
domain-specific process instance
    scenario_uri VARCHAR ( 512 ) NOT NULL,         -- URI for
specific scenario mapped
    embedding VECTOR( 1024 ) NOT NULL,            -- 1024
dimensional vector embedding
    domain_tag VARCHAR ( 128 ) NOT NULL,           -- Domain
identifier, e.g. 'FP&A'
    asset_uri VARCHAR ( 512 ),                      -- Related
financial asset URI (budget, forecast doc)
    party_uri VARCHAR ( 512 ),                      -- Related party
or role URI (e.g. CFO)
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP , -- Ingestion
or update time
    version INTEGER DEFAULT 1 ,                    -- Version for
```

```
embedding/schema evolution
    metadata JSONB                              -- Flexible
metadata for any additional info or provenance
);
CREATE INDEX idx_fpaf_vectors_process_uri ON
fpaf_vectors(process_instance_uri);
CREATE INDEX idx_fpaf_vectors_domain_tag ON
fpaf_vectors(domain_tag);
CREATE INDEX idx_fpaf_vectors_embedding ON fpaf_vectors USING
ivfflat (embedding vector_l2_ops);  -- For vector similarity
search; example with PGVector
```

## Explanation:

- process_instance_uri: URI linking to the stable reference process model in the ontology.
- scenario_uri: URI that identifies the specific scenario mapped within a process instance.
- embedding: Multi-dimensional vector representing semantic content of the scenario.
- domain_tag: Simple tag to filter across domains if multi-domain system.
- asset_uri and party_uri: Optional ontological links for enriched semantic context.
- timestamp & version: For auditing and version control of vector embeddings.
- metadata: JSON for extensible, flexible details including lineage, source system, or calculation method.

This DDL schema is scalable for millions of vectors, supports fast approximate nearest neighbor vector search (optimized via the "embedding" index), and allows critical traceability back to ontological URIs for richer semantic lookup and agentic AI collaboration. It aligns with best practices in vector DB domain modeling and semantic governance.

A more detailed schema, with relational tables for normalized ontological entities or a hybrid graph and vector approach, can be designed as needed.

# Vector Database Integration with Ontologies

## Design Specification: Ontology-Vector Database Integration Scaffold

## Objective

Create a sustainable semantic scaffold that integrates domain-specific ontologies with vector databases to enhance search accuracy, relevancy, and recall, supporting agentic AI applications. The design adopts established standards including OMG's Ontology Definition Metamodel (ODM), ISO/IEC/IEEE 24765 systems engineering standards, and ontology best practices from multiple OMG specifications.

## Key Principles and Standards

- Ontology Modeling & Interoperability:
  - Use OMG's Ontology Definition Metamodel (ODM) for formal ontology representation supporting multiple ontology languages (OWL, RDF) with strong model-driven architecture (MDA) foundation.
  - Follow ISO/IEC/IEEE 24765 for systems and software engineering to ensure modular, maintainable, and interoperable architectural design.
  - Employ OMG Distributed Ontology, Model and Specification Language (DOL) for ontology integration and mapping across heterogeneous domain ontologies.
  - Use OMG URI standards for elements to ensure unique identification and standard reference.
- Vector Database Integration:
  - Embed ontological concepts and relationships as vectors maintaining semantic context using ontology embedding techniques.
  - Link vector embeddings with ontological entities via metadata stored alongside vectors, enabling hybrid symbolic and vector-based retrieval.
  - Enable persistent, versioned storage of mappings between ontology elements and vectorized scenarios.

### 1. Mapping Process Instances to Vector DB Scenarios

- Map individual process instances in domain-specific ontologies as formal ontological entities, each with a unique URI.
- Represent the scenario features for each instance as vector embeddings within the vector database, grounded in the ontology schema using ODM and DOL standards.
- Use metadata tagging within the vector DB to associate each vector scenario representation explicitly with its corresponding ontology process instance URI.
- Implement software adapters to translate between ontology-based process descriptions and vector embeddings, ensuring semantic consistency.

### 2. Maintaining Mappings and Versioning

- Store mappings in a dedicated ontological metadata repository, separate but linked with the vector database, using version control systems aligned with ISO/IEC/IEEE standards for configuration management.
- Versions of domain-specific ontologies and scenario mappings are tracked using semantic versioning, with persistent URIs referencing specific versions.
- Use OMG standards such as ODM and DOL to capture evolving ontologies and their mappings, supported by tools that manage ontology lifecycle (e.g., ontology repositories with versioning, change tracking, and validation workflows).
- Integration pipelines must automate synchronization between updated ontologies and vector embedding updates, preserving mapping integrity.

### 3. Handling Scenarios Mapping to Multiple Processes/Ontologies

- Model scenarios as first-class ontological entities that can have relations (e.g., "participatesIn", "relatesToProcess") to multiple process instances across one or multiple domain ontologies.
- Use composite mappings that aggregate vector embeddings corresponding to these scenario entities and their multiple linked processes, applying multi-ontology integration principles via DOL.
- Apply reasoning engines (enabled by ODM semantics) to infer scenario-level connections across process boundaries and ontologies, promoting holistic retrieval and precision.
- Store cross-ontology scenario-process mappings as explicitly modeled relationships with version control to track dependencies and changes.

## Implementation Outline

| Step | Description | Supported Standards |
|---|---|---|
| Ontology Ingestion | Import OWL/RDF ontologies into ODM-compliant repositories | OMG ODM, OWL, RDF |
| Vector Embedding | Compute embeddings for ontology concepts, relations, and processes | Embedding methods + Metadata via OMG URI |
| Metadata Linking | Tag vectors with ontology URIs and version info | OMG URI standards, ISO config mgmt |
| Hybrid Query Engine | Combine vector similarity search + ontology reasoning for precision | OMG DOL, ISO/IEEE 24765 system design |
| Mapping Persistence | Store mappings/version mappings in dedicated repos with lifecycle mgmt | OMG ODM, DOL, ISO/IEEE config & version |
| Multi-ontology Merging | Support composite scenario mappings with reasoning across domains | OMG DOL, OWL semantics |

This specification ensures a durable, standards-compliant framework that supports evolving ontologies integrated with vector search capabilities tailored for domain-specific agentic AI, addressing accuracy, relevancy, and recall improvements systematically.

# FP&A Sequence Diagrams

The following sequence diagrams, presented as textual walkthroughs of typical diagram flows, describe FP&A (Financial Planning & Analysis) processes within a finance sub-domain. These diagrams cover microservices, knowledge graphs (RDF with SHACL), AI agentic/Vector DB flows, MCP middleware integration, and governance service controls. Each sequence represents a core FP&A scenario (such as planning, forecasting, analysis, policy enforcement, and model updates).

## FP&A Sequence Diagram 1: Budget Forecast Creation

**1. User/Analyst → FP&A Agent (UI/API)**

- Requests new budget forecast for a period (e.g., Q1 Revenue by business unit)

**2. FP&A Agent → Vector DB Service (domain-aligned)**

- Sends query context (domain: FP&A, task: "budget projection", drivers: revenue/expenses, vocab/taxonomy: RDF graph-aligned)

**3. Vector DB Service → Knowledge Graph (RDF/SHACL endpoint via MCP)**

- Requests entity relationships, policy-constrained assumptions (foreign exchange rates, headcount limits, seasonal factors)

**4. Knowledge Graph → SHACL Engine (Policy/Validation Service)**

- Applies domain policies (e.g., no scenario uses expense ratios above X%, validates forecast structure, data completeness)

**5. SHACL Engine → Knowledge Graph / Agent**

- Returns validation results (pass/fail, reasons, constraint messages)

**6. Vector DB → FP&A Agent**

- Returns synthesized context/embeddings, mapped to planning inputs

**7. FP&A Agent → MCP Broker**

- Initiates tool invocation: launches budget calculation, report generation, downstream ETL (if compliant)

**8. MCP Broker → FP&A Domain Microservices**

- Triggers:
  - Budget Calculation Microservice
  - Report Generation Microservice
  - Notification/Audit Logging Microservice

**9. Microservices → MCP Broker → FP&A Agent / User**

- Reports status, deliverables, exceptions; triggers workflow updates

**10. Governance/Audit Service**

- Logs all steps, validation checks, and user actions for compliance reporting

## FP&A Sequence Diagram 2: Policy Validation During Workflow

**1. Scenario Change (User or Automated Event) → FP&A Agent**

- E.g., update salary forecast due to new hiring policy

**2. FP&A Agent → Knowledge Graph Service**

- Updates salary projection data for relevant entities

**3. Knowledge Graph Service → SHACL Validation Engine**

- Triggers shape validation for:
  - Maximum salary caps
  - FTE restriction
  - Fund allocation policies

**4. SHACL Engine → Knowledge Graph Service / FP&A Agent**

- Returns validation pass/fail, details (linked to policy artifact)

**5. If Fail:**

- FP&A Agent → User, with actionable feedback and constraint violations

**6. If Pass:**

- Proceeds to downstream scenario/planning steps, triggers update in forecast models via MCP

**7. Audit Service**

- Records validation cycle, policy artifact involved, and user/system actions

## FP&A Sequence Diagram 3: AI-Driven Variance Analysis

**1. Scheduled Event → FP&A Agent**

- Regular variance analysis between planned and actuals for the quarter

**2. FP&A Agent → Vector DB Service**

- Pulls embeddings for prior plan scenarios, actual results, drivers

**3. Vector DB Service → Knowledge Graph (via MCP)**

- Asks for relationships, metadata, anomalies by entity (cost centers, revenue streams, macro factors)

**4. Knowledge Graph → SHACL Engine**

- Validates if all variance explanations are policy-compliant and supported by source data

**5. SHACL Engine → Knowledge Graph / FP&A Agent**

- Flags cases needing human review (e.g., unexplained >2% deviation from plan)

**6. FP&A Agent → User (or Automated Notification)**

- Delivers root cause breakdown, supporting data, and action suggestions

**7. Governance Service**

- Updates audit log with analysis provenance, validation outcomes

## FP&A Sequence Diagram 4: Workflow Integration/Orchestration (Tool/Process)

**1. External System (e.g., ERP, BI Tool) → MCP Broker**

- Requests trigger of workflow or agent action (e.g., new data load, report refresh)

**2. MCP Broker → Tool Registry**

- Verifies available microservices/tools and required interfaces

**3. MCP Broker Coordinates:**

- Triggers ETL Microservice (data ingest to KG)
- Launches FP&A Agent action/service
- If needed, calls out to external data sources/tools

**4. All integration points pass through audit, policy validation, and governance review, with the flow mirrored in logs and compliance reports**

These sequence flows, if mapped visually, would use UML sequence diagram notation or BPMN, showing clear service boundaries, async interactions, policy enforcement points, MCP middleware routing, agent-vector-KG loops, and audit/service governance. Each microservice and process in the FP&A sub-domain is orchestrated securely and transparently, ensuring true standards-based, SHACL-policy compliant, and agentic integration at enterprise scale

# FP&A Use Cases

This section breaks down FP&A sequence diagrams for three core use cases—Close, Forecast, and Variance Analysis—illustrating how domain microservices, knowledge graph (KG) services with SHACL, MCP middleware, agentic AI/vector DBs, and governance audit services interact in each scenario.

## FP&A Use Case: Period-End Close

Sequence Flow:

1. User/Controller → FP&A Close Service
   - Initiates close cycle (monthly/quarterly/year-end).
2. FP&A Close Service → Data Integration Microservice
   - Ingests finalized transactions, journals, and adjustments from ERP/accounting tools.
3. Data Integration Microservice → Domain Knowledge Graph (RDF via MCP)
   - Loads data into graph, linking all entries, entities, and processes to FP&A domain models.
4. Knowledge Graph → SHACL Validation Service
   - Runs compliance checks for entity completeness, policy adherence (e.g., cutoff dates, approval steps).
5. SHACL Validation Service → FP&A Close Service
   - Flags anomalies or missing statements (e.g., missed expense/report, late entries).
6. FP&A Close Service → Controller/User
   - Reports issues for actionable resolutions, tracks status.
7. FP&A Close Service → Governance/Audit Service
   - Logs all actions, validation checks, and memos.
8. FP&A Close Service → MCP Broker
   - Triggers reporting and compliance tools, downstream processes (external audit prep, archiving, improvement review).

## FP&A Use Case: Forecast Process

Sequence Flow:

1. Analyst/FP&A User → Forecast Agent (UI/API)
   - Requests new forecasts or scenario models (e.g., quarterly revenue/expense forecasts).
2. Forecast Agent → Vector DB (FP&A-aligned)
   - Searches historic trends, embeddings for key drivers and prior forecasts.
3. Vector DB → Knowledge Graph Service (via MCP)
   - Pulls authoritative assumptions and policy-controlled limits (org headcounts, cost ratios, macro factors).
4. Knowledge Graph → SHACL Validation Engine

- Validates forecast assumptions, checks policy constraints (e.g., no expense growth outside policy, business rule adherence).

5. SHACL Validation Engine → Forecast Agent
   - Communicates pass/fail, constraint violations.
6. Forecast Agent → MCP Broker
   - Initiates forecast computation, scenario iteration, automated reporting.
7. Forecast Microservices → Forecast Agent → User
   - Returns model outputs, visualizations, and action prompts.
8. Governance/Audit Service
   - Records data lineage, validations, human overrides, compliance outcomes.

## FP&A Use Case: Variance Analysis

Sequence Flow:

1. Scheduled Event/User → Variance Analysis Agent
   - Launches analysis (actuals vs. planned results).
2. Variance Analysis Agent → Vector DB Service
   - Gathers embeddings for historical results, actuals, planned scenarios.
3. Vector DB → Knowledge Graph Service (via MCP)
   - Pulls entity relationships, commentary drivers, metadata (product, cost center, business unit).
4. Knowledge Graph → SHACL Engine
   - Validates explanations against supported data and business policies (e.g., deviation thresholds, documentation).
5. SHACL/Knowledge Graph → Variance Analysis Agent
   - Returns flagged cases for review, ties explanations to policy artifacts.
6. Variance Analysis Agent → User (FP&A/Controller)
   - Sends variance dashboard, drill-downs, and actionable insights (trend flags, anomaly causes).
7. Governance/Audit Service
   - Logs analysis steps, variances, and resulting actions for compliance and continuous improvement.

Each use case sequence ensures auditable, standards-driven, and policy-compliant orchestration across microservices, knowledge graphs, agentic AI modules, and middleware within FP&A. The touchpoints with SHACL policy validation and MCP broker integration are central to achieving enterprise-grade FP&A automation and regulatory control.

# MCP and Microservices Integration

To design a standards-driven integration topology that unifies knowledge graphs (RDF), AI agents (with domain-specific vector databases), systems, and tools using microservices and Model Context Protocol (MCP), consider the following layered and modular architecture:

## Core Integration Principles

- Microservices Orchestration: Each functional capability—domain graph access, agent query interface, workflow control, policy validation—is encapsulated as a separate microservice. These interact via secure APIs and message brokers, which can scale and be managed independently.
- Knowledge Graphs by Domain: Each business or operational domain has its own RDF-based knowledge graph, modeled using core ontologies plus domain taxonomies. Process-centric graph design ensures the graph reflects real-world entities, flows, and dependencies in that domain.
- Policy Compliance via SHACL: SHACL shapes are used within each graph to declare and enforce constraints (validation, access, workflow logic, etc.), ensuring that policy compliance and business logic are embedded and traceable within domain knowledge.
- Agentic Layer (Vector DBs, LLMs): AI-enabled agents leverage domain-specific graph schemas for knowledge retrieval, but execute reasoning and semantic search over dedicated vector databases, tuned to the same vocabularies and ontologies as the underlying knowledge graphs.
- MCP as Universal Integration Tier: MCP servers act as a standards-based middleware, exposing tools, data, and workflow endpoints as callable resources for AI agents, simplifying cross-system integration and providing a policy-aware interface.

## Topology: Reference Architecture

| Layer | Components/Responsibilities |
|---|---|
| Presentation | End-user UIs, API gateways, agent frontends. |
| Agentic/Augmentation | AI agents, RAG chains, vector DBs (with schemas aligned to RDF KGs), microservices for semantic search, dialogue, workflow actions. |
| MCP Integration | MCP servers with connectors for KG, workflow engines, file systems, external APIs. Tool/resource registries define callable interfaces and policy contracts. |
| Knowledge Graph | Modular RDF graphs by domain (process-centric), SHACL validation engine(s), ontology management and alignment, ETL into graph stores. |
| Policy/Control | Central registry of SHACL shapes, policy dashboard, reporting/audit logs for graph and system compliance via SHACL validation, metadata linkage to workflows/tools. |
| Systems/Tools | Core enterprise systems, event/message buses, process automation platforms, source systems feeding graphs or being automated by agents. |

## Microservices & MCP-Enabled Flows

- Domain Graph Service: API for graph CRUD, SPARQL/GraphQL endpoint, SHACL validation as a service.

- Policy Validation Service: Asserts SHACL shapes, enforces on data, processes, and triggers event hooks on violations for remediation.
- Vector DB/Agent Service: Provides vector search & context retrieval, aligns embeddings/taxonomy config to graph schema to ensure semantic consistency.
- MCP Broker: Manages registration of tools, resources, and workflow steps; exposes MCP-compliant endpoints to both agents and legacy systems. Facilitates secure, auditable interactions across domains and systems.
- Governance & Audit Service: Tracks integration operations, policy compliance, and graph changes; links validation outcomes and provenance to data and process artifacts.

## Standards and Policy Anchoring

- SHACL as Policy Anchor: SHACL shapes serve as structured policy artifacts—both validating RDF data and surfacing policy compliance across the ecosystem, from datasets to workflows to agent actions.
- Ontology Registry: Ensures both knowledge graphs and vector DBs use a consistent, governed set of vocabularies and schemas, preferably with a published versioning scheme and change process.
- MCP for Secure, Standardized Operations: With MCP, agents always access data and trigger tools/workflows via well-defined, standard contracts—supporting security, logging, and rollback.

## Example Flow

1. A user or system event triggers an agent (e.g., via UI or integration bus).
2. The agent consults the vector database for context, using embeddings developed from domain-specific KGs.
3. When authoritative or controlled knowledge is required, the agent queries the RDF KG via the MCP endpoint, with SHACL validation ensuring only policy-compliant data/actions are reachable.
4. The MCP server routes requests to the relevant microservices: KG access, policy validation, workflow triggers, or external tools/systems.
5. All transactions and validations are logged by the governance service, linking outcomes to SHACL policy artifacts and enterprise audit records.

This model is modular, standards-based, auditable, and built for process- and policy-driven control over enterprise knowledge, workflows, and agentic AI enablement.

APPENDIX B

# Conversational AI

A design and reference architecture for conversational AI that lets subject-matter experts talk to their data: components, layers, phased delivery plan, KPIs, and the context engineering module.

B.1 Conversational AI Design
B.2 Conversational AI Architecture

# Conversational AI Design

## Detailed Design: Conversational AI for SMEs to Talk to Their Data

### Overview

A conversational AI platform tailored for business SMEs enables natural language querying, exploration, and insights generation from their operational and analytical data using a combination of prompt engineering and context engineering to optimize relevance, accuracy, and usability.

### Core Components

1. Data Integration Layer
- Connectors to various SME data sources: CRM, ERP, sales, marketing, finance systems, databases, spreadsheets.
    - Preprocessing pipelines convert raw data to structured semantic formats and embed relevant metadata for retrieval.
2. Semantic Knowledge Layer (Knowledge Engineering)
- Develop a domain-specific ontology and knowledge graph representing key business concepts, entities, and relationships.
    - Annotate data sources for semantic retrieval and disambiguation, supporting natural language understanding for SME-specific terminology.
3. Retrieval and Vector Database
- Store embeddings of data segments, documents, and past conversations to retrieve contextually relevant information dynamically.
    - Supports semantic search to surface the most pertinent data elements in response to queries.
4. Context Engineering Module
- Orchestrates the dynamic assembly of context merging: retrieved semantic data, conversation history, SME role/profile, system instructions, and compliance metadata.
    - Filters and ranks contextual inputs to optimize prompt size and relevance while enforcing policy constraints.
5. Prompt Engineering Module
- Template library creating structured, modular prompts customized to common SME intents (reports, insights, alerts, explanations).
    - Constructs and adapts prompts based on user input and enriched context to guide LLMs effectively.
6. Conversational AI Engine (LLM)
- Uses a tuned large language model (OpenAI GPT, Claude, or similar) to interpret user inputs, generate responses, and maintain conversational flow.
    - Supports multi-turn dialogue, clarifications, and follow-ups leveraging contextual memory.
7. Dialogue Management and Orchestration

- Manages conversation state, handles user intents, manages escalation paths (e.g., handoff to human agents), and tracks task progress.
    - Integration with external tools for actions like report generation, alert setup, or data exports.

8. User Interface

- Multi-channel support: web or mobile apps, messaging platforms (Slack, MS Teams), voice assistants.
    - Intuitive interface including text input, voice, dashboards for resuming or exploring results.

9. Monitoring and Analytics

- Tracks usage metrics, response relevancy, error/hallucination rates, user feedback.
    - Continuous learning pipeline refines prompts, context rules, and LLM fine-tuning.

## Workflow Example

- User Query: "Show me last quarter's sales trends by region."
- Context Engineering: Retrieves relevant sales data, historical conversation, recent queries, and SME role.
- Prompt Engineering: Creates tailored prompt with system instructions and data summary.
- LLM Response: Generates a clear, data-backed narrative with charts or key metrics.
- Follow-Up: User asks, "Which product categories drove growth?"—system recalls previous context and builds new prompt.

## Implementation Considerations

- Security & Governance: Embed semantic governance policies ensuring data access compliance and privacy.
- Domain Adaptation: Invest in SME-specific ontology and prompt tuning.
- Cost & Performance: Balance context size and retrieval latency; use caching and incremental context updates.
- User Training: Provide onboarding and contextual help to increase adoption.
- Iterative Improvement: Use monitoring data to expand FAQ sets, refine knowledge graph, and optimize prompts.

## Benefits for SMEs

- Democratizes access to complex data without needing technical expertise.
- Accelerates decision-making with conversational insights and alerts.
- Improves productivity by automating routine queries and report generation.
- Scales conversational intelligence across departments (sales, finance, operations).

This approach combines prompt engineering's role in crafting effective LLM inputs with context engineering's orchestration of dynamic, multi-source information to create a conversational AI that truly lets SMEs "talk to their data" in an intuitive, productive manner.

# Conversational AI Architecture

## System Architecture for SME Conversational AI

### 1. User Interface Layer

- Multi-modal interfaces supporting text (chatbots, web, mobile apps), voice assistants, and messaging platforms (Slack, Teams).
- Provides user input capture and display of AI-generated responses.
- Supports conversation history viewing and interaction controls for clarifications or follow-ups.

### 2. Natural Language Understanding (NLU) Module

- Intent classification to identify user goals from queries.
- Entity extraction to identify domain-specific elements (e.g., products, dates, metrics).
- Language parsing using transformers or transformer-based encoders tailored with SME domain data.

### 3. Dialogue Management and Orchestration

- Tracks conversation state, manages multi-turn dialogue flows.
- Integrates feedback loops for disambiguation, error correction, and context updates.
- Applies business rules and defines fallback strategies or human handoff.

### 4. Context Engineering Module

- Dynamically assembles context by combining: user profile, conversation history, retrieved semantic data, business role, and compliance metadata.
- Manages prompt size, relevance filtering, and context versioning.
- Uses context templates and policy-driven context governance to tailor the information sent to the LLM.

### 5. Prompt Engineering Module

- Maintains a library of prompt templates customized for typical SME requests (financial reports, sales insights, customer queries).
- Adapts and sequences prompts based on real-time context and workflow state.
- Fine-tuned prompt tuning and examples for accuracy and clarity.

### 6. Semantic Knowledge Layer

- Semantic models including business ontologies and knowledge graphs representing SME domain concepts.
- Supports semantic retrieval, disambiguation, and grounding for LLM inputs.
- Enables precise contextual understanding and reasoning.

### 7. Retrieval and Vector Search Engine

- Embedding-based retrieval of relevant documents, FAQs, reports, and prior interactions.
- Integrates with knowledge layer to augment context dynamically.

### 8. Large Language Model (LLM) Integration

- Connects to LLM APIs or on-prem models (OpenAI GPT, Anthropic Claude, etc.).
- Executes the generation of natural language responses based on engineered prompts and context.
- Supports multi-step reasoning, summarization, and explanation.

### 9. Backend Data and System Connectors

- Interfaces with SME data sources: CRM, ERP, analytics platforms, spreadsheets, and databases.
- Preprocesses and pipelines data into semantic formats.
- Ensures real-time or batch data synchronization.

### 10. Monitoring, Analytics, and Feedback Loop

- Tracks user engagement, response accuracy, error rates, and latency.
- Collects user feedback for conversational improvements.
- Enables iterative updates to prompt libraries, context rules, and knowledge schemas.

## Architectural Flow

1. User submits query via UI.
2. NLU module processes intent and entities.
3. Dialogue manager maintains conversation state.
4. Context engineering module assembles enriched context from retrieval and semantic knowledge.
5. Prompt engineering crafts the LLM input prompt with context and templates.
6. LLM generates a response.
7. Response delivered to user with ongoing monitoring.
8. Feedback loop for continuous adaptation.

This architecture supports scalable, accurate, and domain-specific conversational AI tailored for SMEs to "talk to their data" by tightly integrating prompt and context engineering with semantic knowledge and retrieval layers.

The seven phases below are delivery phases for this project. They are separate from the six-stage Semantic Knowledge Lifecycle (SKL) that governs semantic assets once built; see the lifecycle note in Section 2.4.

## Phase 1: Planning & Requirements (Week 1–4)

Goals: Define scope, gather requirements, identify key stakeholders.

Milestones:

- Stakeholder interviews completed
- Use case and domain scope defined
- System architecture outline approved

Deliverables: Requirements specification document, high-level architecture diagram.

## Phase 2: Design & Foundation Setup (Weeks 5–8)

Goals: Develop data models, ontologies, knowledge bases, and select technology stack.

Milestones:

- Domain ontology and knowledge graph modeled
- Data sources integrated and preprocessed
- Technology stack selected (LLMs, retrieval tools, UI platforms)

Deliverables: Knowledge schema, data pipeline prototypes, architecture design.

## Phase 3: Prototype Development (Weeks 9–14)

Goals: Build core modules—NLU, dialogue management, retrieval, prompt templates.

Milestones:

- Basic conversational interface with data connection
- Retrieval engine operational with relevant data
- Prompt templates and context assembly rules created

Deliverables: Working prototype with basic conversation flows.

## Phase 4: Context & Prompt Engineering Tuning (Weeks 15–18)

Goals: Refine prompt templates and context assembly for high accuracy and relevance.

Milestones:

- Engineered prompts stored and tested in real dialogues
- Dynamic context assembly using retrieval and knowledge layers
- Initial user testing completed

Deliverables: Fine-tuned prompt library, context engineering rules documentation.

## Phase 5: Testing & Iteration (Weeks 19–22)

Goals: Conduct extensive testing, gather feedback, optimize performance.

Milestones:

- Usability testing with SMEs completed
- Response relevance, accuracy, and coherence metrics improved
- Security and compliance checked

Deliverables: Test reports, usability feedback, performance metrics.

## Phase 6: Pilot Deployment & Feedback (Weeks 23–26)

Goals: Launch pilot with select SMEs, monitor engagement, and refine.

Milestones:

- Pilot platform live with real SME users
- Usage analytics and feedback collected

- Rapid iteration based on insights

Deliverables: Pilot report, improvement backlog, user training materials.

## Phase 7: Full Deployment & Continuous Improvement (Weeks 27+)

Goals: Scale deployment, establish monitoring, and optimize iteratively.

Milestones:

- Full deployment completed
- Monitoring dashboards operational
- Ongoing updates and tuning based on analytics

Deliverables: Final release, ongoing support plan, metrics and reporting system.

## Summary Timeline & Milestones

| Phase | Duration | Key Milestones |
|---|---|---|
| Planning & Requirements | 4 weeks | Stakeholder agreement & requirements sign-off |
| Design & Foundation Setup | 4 weeks | Knowledge graphs and architecture setup |
| Prototype Development | 6 weeks | Initial working prototype |
| Context & Prompt Tuning | 4 weeks | Optimized prompts & context assembly |
| Testing & Iteration | 4 weeks | Performance and usability validated |
| Pilot Deployment | 4 weeks | Pilot launched, initial feedback collected |
| Full Deployment & Scaling | Ongoing | Full rollout, continuous monitoring & improvement |

This roadmap ensures systematic development, testing, validation, and scaling, aligned with typical project management practices and best practices for AI deployment.

## Measurable KPIs

1. User Adoption Rate
- Percentage of target SMEs actively using the AI system within a defined period post-launch.
  - Indicator of initial acceptance and onboarding success.
2. Query Resolution Rate
- Percentage of user queries successfully answered or fulfilled by the AI without human intervention.
  - Measures effectiveness and reliability of LLM responses.
3. Response Accuracy / Relevance
- LLM outputs rated for correctness and relevance, assessed by automated scoring or manual audits.
  - Reflects quality of context engineering and prompt design.
4. Task Completion Time
- Average time from user query to delivery of actionable insights or results.
  - Measures system efficiency and user satisfaction.

5. User Satisfaction Score (CSAT)
- Survey score collected post-interaction measuring user happiness with AI responses.
    - Qualitative indicator tied to adoption and trust.
6. Hallucination Rate
- Frequency of AI responses containing fabricated or incorrect information.
    - Reflects prompt and context engineering precision.
7. System Uptime and Latency
- Technical metrics for availability and performance ensuring a smooth user experience.
8. Cost Per Interaction
- Economic metric showing average operational cost per AI user interaction.
    - Useful for budgeting and scaling decisions.

## Context Engineering Module: Detailed Design

## Purpose

The Context Engineering Module dynamically assembles, manages, and optimizes the information environment delivered to the AI model at inference time. It ensures that the large language model (LLM) sees the most relevant, structured, and policy-compliant context to generate accurate, coherent, and reliable responses.

## Key Functional Components

1. Context Orchestrator
- Central controller that manages context assembly workflows.
    - Coordinates inputs from multiple sources such as conversation history, retrieval results, user profile, system instructions, and external tools.
    - Enforces token and size constraints to fit within LLM context windows.
2. Retrieval Integrator
- Interfaces with vector search engines and semantic knowledge graphs to fetch domain-relevant documents, data segments, or FAQ entries.
    - Ranks and filters retrieved content based on relevance scores and operational policies.
    - Supports multi-modal sources (structured data, unstructured text, API outputs).
3. Memory Manager
- Tracks short-term conversation history and long-term user preferences or organizational memories.
    - Prunes or summarizes prior context items to maintain relevance and avoid context overflow.
    - Handles persistent session state and personalized context tailoring.
4. Policy and Compliance Filter
- Applies governance rules and compliance constraints on what data and instructions are included or excluded.
    - Ensures sensitive or regulated information is handled securely and audit trails are maintained.

    - Integrates with semantic governance systems to enforce organizational standards.

5. Prompt Template Engine

- Hosts libraries of reusable prompt templates with placeholders.
    - Dynamically instantiates templates with assembled context pieces.
    - Supports conditional template branching to handle multi-turn dialogue scenarios and diverse user intents.

6. Context Formatter

- Structures and formats the final context bundle into LLM-compatible input.
    - Adds explicit context delimiters, metadata tags, and schema definitions to improve LLM parsing and output consistency.
    - Optimizes token usage by compressing and summarizing content where possible.

7. Feedback & Adaptation Loop

- Monitors LLM output quality, user feedback, and interaction logs.
    - Adjusts retrieval, ranking, and prompt selection strategies based on feedback.
    - Supports active learning pipelines for continuous context engineering improvement.

1. Input Collection: User query, session state, user profile, and operational metadata are received.
2. Context Assembly: Retrieval Integrator fetches relevant knowledge; Memory Manager adds conversational and personalized history.
3. Policy Enforcement: Policy Filter removes or modifies context as required for compliance.
4. Prompt Instantiation: Prompt Template Engine combines and formats context into a structured prompt.
5. Output Formatting: Context Formatter finalizes prompt and sends it to the LLM.
6. Response Generation: LLM produces output, logged for monitoring.
7. Feedback Analysis: Feedback Loop evaluates response and tunes context assembly rules.

## Design Considerations

- Modularity: Each functional component is modular, enabling independent updates and scalability.
- Latency Optimization: Context assembly optimized for low-latency applications.
- Security: Sensitive data handling, encryption, and audit logs embedded.
- Extensibility: Supports integration of new data sources, tools, and policy engines.

## Technologies & Tools

- Vector databases (e.g., Pinecone, Milvus)
- Knowledge graph frameworks (e.g., GraphDB, Neo4j)
- LLM APIs (OpenAI GPT, Anthropic Claude)
- Workflow orchestrators (e.g., Apache Airflow, Temporal)
- Schema/ontology editors (e.g., Protégé)
- Secure data storage and audit logging systems

This Context Engineering Module design delivers a systematic, scalable, and secure approach to providing AI models with precisely curated, governed, and contextually enriched inputs—enabling sophisticated conversational AI that truly understands and adapts to SME business data and workflows

The proposed module architecture layers and data flow for the Context Engineering Module are as follows.

## Module Architecture Layers

1. Input Layer
- Receives inputs from user queries, session state, user profile.
    - Interfaces with retrieval systems and external tools for supplemental data.
2. Context Assembly Layer
- Retrieves relevant external knowledge using vector search and semantic graphs.
    - Combines conversation history, user metadata, and retrieved data.
    - Applies ranking and filtering for relevance.
3. Policy & Compliance Layer
- Enforces semantic governance and compliance policies.
    - Filters or redacts sensitive data.
    - Ensures context complies with organizational rules.
4. Prompt Template Layer
- Applies structured prompt templates dynamically filled with assembled context.
    - Handles multi-turn flow with branching and conditional prompts.
5. Formatter & Optimizer Layer
- Formats final prompt, manages token limits.
    - Summarizes or compresses context to fit model restrictions.
6. Feedback Loop Layer
- Collects output quality metrics, user feedback.
    - Feeds adaptation signals to retrieval, filtering, and templates.

## Data Flow Overview

1. User Query Received
- Input Layer captures query and session info.
2. Relevant Context Retrieved
- Context Assembly Layer queries vector DBs and knowledge graphs for relevant docs.
    - Merges retrieved info with conversational memory.
3. Context Filtering
- Policy Layer applies filters and compliance checks on assembled context.
4. Prompt Construction
- Prompt Template Layer inserts filtered context into templates.
5. Prompt Formatting

- Formatter optimizes prompt structure and content length.

6. LLM Interaction
- Prepared prompt sent to LLM for response.

7. Monitoring & Feedback
- Outputs delivered to user, monitored, and logged.

## Data Flow Diagram (Conceptual Level 1)

- External Inputs: User Query, Metadata, Retrieval Systems
- Processes: Input Processing → Retrieval & Assembly → Filtering & Compliance → Prompt Engineering → Formatting → LLM Invocation → Output Delivery → Feedback Collection
- Data Stores: Conversation Memory, Semantic Knowledge Graphs, Prompt Template Repository, Policy Rules Engine
- Data Flows: Queries & Responses, Context Data, Compliance Flags, Feedback Metrics

## Summary

This layered architecture combined with clear data flow enables scalable, flexible, and governed context engineering. It ensures dynamic assembling of relevant, compliant, and optimized inputs, maximizing the conversational AI's reliability, relevance, and responsiveness.

This design aligns with industry best practices for complex AI workflows and knowledge-driven conversational systems.

APPENDIX C

# Simulation and Causal Reasoning

Emerging techniques that extend context engineering: simulation-driven context with the Simulator, Reflector, and Curator pattern, and causal reasoning over ontologies with CausalGraph2LLM.

C.1 Simulation for Context Engineering
C.2 Causal Reasoning with CausalGraph2LLM

# Simulation for Context Engineering

Simulations can generate dynamic context for language models. A simulation acts as a world model: it represents the current state of an environment and how it changes in response to actions, so a model can reason about possible futures instead of relying on static prompts or retrieval. This section summarizes how simulation fits into context engineering, which kinds of simulation are most useful, and how to measure their effect.

## Simulator, Reflector, and Curator

A common pattern separates the work into three layers that form a feedback loop:

- **Simulator:** an environment model (discrete-event, continuous, or hybrid) that produces states, outcomes, and scenario results through an API that other components can query.
- **Reflector:** monitors simulator output alongside interaction history, detecting state changes, anomalies, and insights relevant to the agent's goals.
- **Curator:** filters, ranks, and assembles simulator output and reflections into structured context, enforcing policy and compliance, managing memory, and maintaining prompt templates.

In operation the Simulator runs scenarios, the Reflector interprets the results, and the Curator composes enriched context for the agent, whose decisions in turn steer the next simulation. Modular components, event-driven orchestration, continuous evaluation, and policy encoded in the Curator keep the loop trustworthy.

## How simulation produces context

- **State and transitions:** environment state is encoded in structured form, and the model predicts next states from current states and actions.
- **Guided rollouts:** multi-step simulated trajectories provide both training data and evolving context.
- **Action history:** prior actions, observations, and simulated states maintain situational awareness across turns.
- **Hybrid transitions:** deterministic rules combined with generative prediction balance validity and diversity.

## Simulation types that produce useful context

| Type | What it contributes |
|---|---|
| Quality-diversity | Archives of diverse, high-performing solutions that models can combine through in-context learning |
| Agent-based | Realistic interactions among autonomous agents in social, physical, or cyber settings |
| Hybrid rule-based and generative | Novel but plausible futures that remain logically consistent |
| Scenario and state | Step-wise environment states that support multi-turn reasoning and planning |

## Measuring the effect

| Metric | What it shows |
|---|---|
| Answer relevancy | Whether responses address the task given the simulated context |
| Faithfulness | Whether outputs are consistent with the simulated facts |
| Context relevancy | Whether the simulation contributes useful, concise information |
| Task completion rate | How often multi-step tasks succeed with simulated context |
| Hallucination rate | How often outputs contain fabricated information |
| Robustness | How stable outputs are when the simulated context varies slightly |
| Throughput and latency | Whether simulation can run within real-time constraints |

## How Simulation Connects to the Core Pipeline

- **Mapping quality assurance (Section 2.3, steps 6–7):** replay representative queries against a simulated copy of the knowledge graph with the proposed mappings applied, and promote them only if answer relevancy and faithfulness improve.
- **FP&A scenario modeling (Appendix A.11 and A.12):** the Forecast and Variance Analysis agents use scenario simulations as dynamic context, validated by the same SHACL policy gate.
- **Feedback controls (Section 2.1):** simulation results feed the staging partition, never the authoritative vector store, until reviewed.

# Causal Reasoning with CausalGraph2LLM

To leverage CausalGraph2LLM with a semantic knowledge layer built with ontologies designed as process-central models for business domains, mapping to metadata for all digital assets, the following approach applies:

## Integration Strategy

1. Map Ontologies to Causal Graph Structures: Use the domain ontologies to

   define causal relationships between entities, processes, and data assets. Since CausalGraph2LLM works with causal graphs (directed acyclic graphs), represent the ontological classes and relationships as nodes and directed edges describing causal influence or dependency.

2. Generate Verbalized or Encoded Causal Queries: Encode the causal graph knowledge extracted from the semantic layer in one of the supported formats (e.g., JSON, adjacency matrix, GraphML) compatible with CausalGraph2LLM. This serves as input for LLMs to interpret causal relationships per the domain models.
3. Use CausalGraph2LLM as an Evaluation and Inference Tool: Run causal reasoning queries on the domain causal graphs through LLMs guided by CausalGraph2LLM tasks, which include parent/child identification, confounder detection, and causal intervention analysis. This tests and generates insights aligned with the process models and metadata mappings.
4. Iterate for Domain-Specific Tuning: Use the feedback loop from causal query results to refine the semantic models, enhancing the expressiveness and accuracy of causal relationships within ontologies. This iterative tuning supports alignment between the formal semantic layer and practical causal grounding.
5. Incorporate Causal Reasoning into Enterprise Workflows: Embed the causal insights obtained via CausalGraph2LLM evaluations into operational decision-making workflows, data governance, risk management, and scenario simulations, all grounded in the semantic knowledge fabric.

## Benefits of This Approach

- Enhanced causal interpretability of ontology-driven models through rigorous LLM-based causal reasoning.
- Systematic evaluation of causal assumptions encoded in ontologies against benchmarked tasks.
- Scalable, domain-aligned causal inference enabling robust explanation, intervention, and decision-making.
- Semantic integration ensures causal models are directly linked to metadata and digital assets, enriching context and traceability.

Encoding the semantic ontologies as causal graphs and using CausalGraph2LLM as a benchmark and reasoning engine creates a powerful synergy between formal semantic governance and advanced causal reasoning for enterprise AI applications

## How Causal Reasoning Connects to the Core Pipeline

- **Variance analysis (Appendix A.12):** the Variance Analysis agent uses causal queries over the FP&A ontology to rank likely root causes before presenting them to the user.
- **Ontology quality assurance (Section 2.3):** causal checks flag mapped relationships whose implied cause and effect contradicts observed data, routing them to manual review.
- **Status:** like Appendix C.1, this is an optional enhancement planned after MVP1.

APPENDIX D

# Statement of Work (MVP1 Prep)

The Statement of Work for the 12-week build of semantic services supporting the HR 360 agent: scope, SKL footprint, services, measurement, traceability, service level improvement, deliverables, and acceptance criteria.

## D.1 Statement of Work (MVP1 Prep)

# Statement of Work (MVP1 Prep)

This Statement of Work commits the Knowledge Engineering team to a 12-week Proof of Capability that puts a governed Semantic Knowledge Layer (SKL) under the HR 360 agent, starting with Employee Benefits & Retirement Sponsorship. It replaces the May 2025 Statement of Work for Marketplace semantic services.

## Statement of Work: 12-Week Build of Semantic Services to Support the HR 360 Agent Rollout

**Project Title:** 12-Week Build of Semantic Services to Support the HR 360 Agent Rollout

**Date:** November 1, 2026

**Term:** 12 weeks, Monday, November 2, 2026 to Friday, January 22, 2027

**Client:** Enterprise Analytics

**Prepared For:** Enterprise AI Strategy & Solutions

**Prepared By:** Knowledge Engineering (KE) Team

**Deliverable Type:** Proof of Capability (MVP1), not a production release

**Governing Architecture:** Keel and North Star (Sections 1.5 and 2.1); Federated RAG 2.0 (Section 2.1); inbound microservices and agents (Section 2.6)

## 1. Goal & Terms

**Goal:** Prove that a governed SKL makes the HR 360 agent answer client natural-language questions about benefits and retirement more accurately, more consistently, and with an audit trail, and that the improvement can be measured before and after.

**What "proved" means at Week 12:** The HR 360 agent answers an agreed set of benefits and retirement competency questions through the SKL; every answer cites the governed terms, SSOT rule versions, and source documents it used; and the before and after measures in Sections 8 and 9 show the change.

**Terms used in this SOW: KE** is Knowledge Engineering. **SKL** is the Semantic Knowledge Layer (Section 2.6), the keel; it is distinct from the six-stage Semantic Knowledge Lifecycle in Section 2.4. **SSOT** is the Single Source of Truth for rules, constraints, logic, and calculations (Section 3.5). **CQ** is a competency question, a test the knowledge must pass (Appendix A.4). Open items are listed in Section 10.6.

## 2. Scope

Employee Benefits & Retirement Sponsorship is the anchor domain and is modeled in full. The other seven domains are modeled only as far as a benefits or retirement question needs them: their concepts enter through the HR Service Operations container or, for other areas, through enterprise service catalog entries (Section 3.1), never as full domain ontologies.

| Domain | Depth in MVP1 | Why a benefits or retirement question needs it | Example HR 360 question |
|---|---|---|---|
| Employee Benefits & Retirement Sponsorship | **Full:** domain ontology, vocabulary, taxonomies, SSOT rules, vector partition | The questions themselves: plans, eligibility, enrollment, contributions, vesting, distributions | "When am I fully vested in the company match?" |
| Workforce Administration & Payroll | **Slice:** deductions, pay periods, compensation, hours, employment status | Elections become payroll deductions; eligibility depends on hours and status; match depends on compensation | "Why did my 401(k) deduction change this paycheck?" |
| Government Compliance & Employer Risk Management | **Slice:** legal norms and notices that govern benefits steps (ERISA, IRC, ACA, COBRA, HIPAA, state rules) | Answers must respect the rule that governs the step; some questions must be escalated, not answered | "Do we have to offer COBRA to a terminated employee?" |
| Client Relationship & Business Development | **Slice:** worksite client, co-employment, plan sponsorship and participating-employer roles, client plan selections | Which plans a worksite offers and who sponsors them decides which answer applies | "Which medical plans does our company offer this year?" |
| HR Technology Platform & Employee Experience | **Slice:** source systems, channels, self-service steps, documents | Answers resolve to the system or document where the employee acts | "Where do I change my beneficiary?" |
| Talent Management & Organizational Development | **Reference:** hire, transfer, leave, and termination events only | These events trigger eligibility, waiting periods, COBRA, and distribution options | "What happens to my benefits if I go part-time?" |
| Workforce Development, Coaching & Performance Measurement | **Reference:** process-step and measure classes only | Supplies the measure pattern for answer quality; no benefits content | Not queried in MVP1 |
| Organizational Values, Culture & Ethics Stewardship | **Reference:** fiduciary duty, answer ethics, no-advice policy | Governs how the agent answers (information, not financial or legal advice) | "Should I pick the Roth or the pre-tax option?" (answered as information plus referral) |

### In Scope

- Questions from worksite employees, client HR administrators, and internal HR and benefits staff about health and welfare benefits and retirement plans.
- The SKL assets, SSOT items, semantic services, and measures in Sections 4 to 9.
- Integration with the HR 360 agent through the search facade and Federated RAG 2.0 (Section 2.1).
- Development and test environments, and a stage environment for the Week 12 demonstration.

### Out of Scope

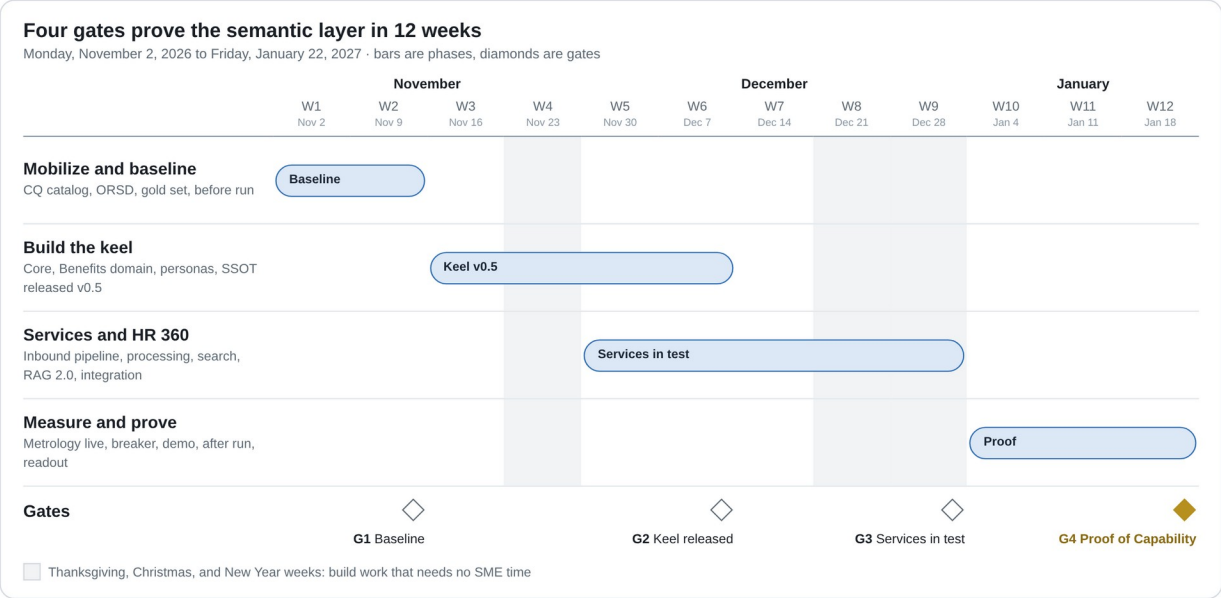
- Transactions: the agent explains and points to where to act; it does not enroll, change elections, or move money.

- Full domain ontologies for the seven supporting domains (planned for MVP2 and later).
- Production release, production support, and service levels beyond the demonstration period.
- Individual financial, tax, or legal advice. Plan documents, not the SKL, remain the legal source.

## 3. Objectives & 12-Week Plan

Five objectives, each with a section that defines it and a gate that proves it.

| # | Objective | Defined in | Proved at |
|---|---|---|---|
| O1 | Develop the initial SKL footprint: core, domains, personas, SSOT | Section 4 | Gate 2 (Week 6) |
| O2 | Develop semantic services that serve HR 360 client natural-language queries | Section 5 | Gate 3 (Week 9) |
| O3 | Establish the operating model: monitoring and measurement at design time and run time | Section 6 | Gate 3 (Week 9), live through Week 12 |
| O4 | Demonstrate end-to-end traceability, trust, and compliance | Section 7 | Gate 4 (Week 12) |
| O5 | Demonstrate service level improvement, before and after | Sections 8 and 9 | Gate 1 (baseline) and Gate 4 (after) |




Phases overlap by design: services start in Week 5 against an early keel draft, so Gate 2 releases a keel that has already been exercised. Each gate's pass criteria are in Section 10.2.

**Calendar notes:** Week 4 contains Thanksgiving; Weeks 8 and 9 contain Christmas and New Year. The plan puts build work that needs no SME time in those weeks. November is open enrollment season, which gives rich baseline questions but limits benefits SME availability in Weeks 1 to 4.

## 4. SKL Footprint (Objective O1)

The Week 6 footprint is the keel for one domain: a small core reused unchanged by every later domain, the Benefits & Retirement domain, the HR Service Operations container that the supporting domains share, 12 personas, and 40 SSOT items. All assets follow the Part III standards (naming, URIs, term definitions, change control), are authored in the KE Workbench (Appendix E), committed to source control, and released through the pipeline in Section 3.3.

### 4.1. Core: Enterprise Vocabularies, Taxonomies, Ontologies

The core is built once and owned by Architecture; Benefits is its first consumer.

| Asset | Type | Content in MVP1 | Standard basis | Owner |
|---|---|---|---|---|
| Core ontology (`core:`) | Ontology | Process, Process Step, Service, Asset, Product, Party, Role, Location, Time, Event, Issue, Measure | BFO (ISO/IEC 21838-2), OWL 2 RL | Architecture |
| Unifying ontology | Ontology | Process step as anchor: performedBy, consumes / produces, triggers / triggeredBy, governedBy, measuredBy, precedes / follows | Section 3.1 | Architecture |
| Legal norm module | Ontology | Norm, Obligation, Permission, Prohibition, Jurisdiction, Effective period; links norms to steps via governedBy | LKIF-Core (Appendix A.7) | Compliance with KE |
| Provenance and catalog | Ontology (reused) | Digital asset descriptions, distributions, lineage of every load and answer | DCAT 3, PROV-O, Dublin Core | KE |
| Measure model | Ontology | Measure, Target, Observation, Threshold, Dimension (trust, quality, accuracy, completeness) | Section 3.6; W3C Data Cube for observations | KE |
| Persona vocabulary (`persona:`) | Vocabulary | 12 persona individuals (Section 4.3), expertise levels, tones, channels | SKOS, SKOS-XL (Section 3.8) | KE with HR Operations |
| Enterprise core vocabulary | Vocabulary | Preferred, alternate, and hidden labels and definitions for every core term | SKOS; Term Definition Standard (Section 3.7) | Data stewards |
| Lifecycle and classification schemes | Vocabulary | SKL lifecycle stages; data classification (Public, Internal, Confidential, Restricted); status values | Sections 2.4 and 3.3 | KE |
| Question intent taxonomy | Taxonomy | Informational, eligibility, how-to, status, calculation, compliance, advice-seeking, escalation | New in this SOW | KE with HR 360 product |
| Document type taxonomy | Taxonomy | Plan document, SPD, SBC, enrollment guide, policy, notice, form, FAQ, knowledge article | DCAT, Dublin Core types | Data stewards |
| Regulatory source | Taxonomy | Federal statute, regulation, | LKIF source types | Compliance |

| Asset | Type | Content in MVP1 | Standard basis | Owner |
|---|---|---|---|---|
| taxonomy | | agency guidance, state law, plan document, company policy | | |
| Relations and attributes taxonomies | Taxonomy | Reused as written | Section 3.9 | KE |
| Jurisdiction scheme | Vocabulary | United States federal and the states where worksite clients operate | ISO 3166-2 codes | Compliance |

### 4.2. Domains Needed for Benefits & Retirement Queries

One domain ontology and its area container. Each supporting domain contributes only the classes and properties listed; their full ontologies are MVP2 work.

| Module | Prefix | Key classes | Key properties and inter-domain edges |
|---|---|---|---|
| Employee Benefits & Retirement Sponsorship (domain) | `benefits:` | Benefit Plan (Medical, Dental, Vision, Life, Disability, Spending Account, Health Savings Account, Employee Assistance, Voluntary); Retirement Plan (401(k), Roth Deferral, Safe Harbor feature, Auto-Enrollment feature); Plan Year; Coverage Tier; Eligibility Rule; Waiting Period; Enrollment Window; Qualifying Life Event; Election; Dependent; Beneficiary; Contribution (Employee Deferral, Employer Match, Catch-Up); Vesting Schedule; Loan; Hardship Distribution; Rollover; Plan Sponsor; Plan Administrator; Fiduciary; Recordkeeper | hasPlanYear, offersCoverageTier, hasEligibilityRule, hasVestingSchedule, electsPlan, coversDependent, designatesBeneficiary, sponsoredBy, administeredBy |
| Workforce Administration & Payroll (slice) | `payroll:` | Payroll Deduction, Pay Period, Compensation (Plan Compensation), Hours of Service, Employment Status, Employee Class | Benefits Election determinesDeduction Payroll Deduction; Hours of Service determinesEligibility Eligibility Rule |
| Government Compliance & Employer Risk (slice) | `compliance:` | Legal Norm (ERISA, IRC, ACA, COBRA, HIPAA, state continuation), Required Notice, Filing, Nondiscrimination Test, Annual Limit | Step governedBy Legal Norm; Termination Step createsObligation COBRA Notice; Contribution limitedBy Annual Limit |
| Client Relationship (slice) | `client:` | Worksite Client, Co-Employment Arrangement, Participating Employer, Client Plan Selection | Worksite Client selectsPlan Benefit Plan; Participating Employer participatesIn Retirement Plan |
| HR Technology & Employee Experience (slice) | `experience:` | Source System, Self-Service Step, Channel, Form | Self-Service Step realizes Process Step; Benefit Plan documentedIn Digital Asset |
| Talent Management (reference) | `talent:` | Hire, Status Change, Leave of Absence, Termination (as | Event triggers Eligibility Determination, Enrollment Window, |

| Module | Prefix | Key classes | Key properties and inter-domain edges |
|---|---|---|---|
| | | Events) | COBRA Notice, Distribution Option |
| Workforce Development (reference) | (core only) | Reuses core Measure and Process Step | None in MVP1 |
| Values, Culture & Ethics (reference) | `ethics:` | Fiduciary Duty, Answer Policy (information, not advice), Escalation Policy | Answer governedBy Answer Policy |

The HR Service Operations container holds every edge above that stays within the area and needs approval from the Benefits, Payroll, and Compliance owners and Architecture; edges to domains in other areas go through their enterprise service catalog entries (Section 3.1).

**Benefits vocabularies and taxonomies:** benefit type taxonomy; retirement plan feature taxonomy; qualifying life event taxonomy; dependent relationship scheme; contribution source scheme; distribution type taxonomy; plus persona labels such as "401k match", "company match", "vesting", "QLE", "open enrollment", and "COBRA".

### 4.3. Persona Instances

Twelve personas, each an individual of `core:persona` with the attributes in Section 3.8. Personas select relevance, labels, tone, and prompt template; they never grant access, and they are set from the authenticated role, not from the prompt.

| Persona | User group | Typical benefits and retirement needs | Expertise | Channel |
|---|---|---|---|---|
| `persona:worksite-employee` | Co-employed worksite employee | Eligibility, costs, deadlines, how to enroll, own match and vesting | Novice | HR 360 agent, employee portal |
| `persona:new-hire-employee` | Employee in the waiting period or first enrollment window | What to choose, by when, what happens if nothing is chosen (auto-enrollment) | Novice | HR 360 agent, onboarding |
| `persona:life-event-employee` | Employee with a qualifying life event (birth, marriage, loss of coverage) | Which changes are allowed and the election window | Novice | HR 360 agent |
| `persona:departing-employee` | Employee leaving or reducing hours | COBRA, final deductions, loans, rollover and distribution options | Novice | HR 360 agent |
| `persona:client-human-resources-administrator` | HR contact at the worksite client | Plan offerings, eligibility rules, deadlines, employer obligations | Practitioner | HR 360 agent, client portal |
| `persona:client-owner-executive` | Business owner or executive at the worksite client | Plan sponsorship, employer cost, compliance exposure | Practitioner | HR 360 agent, advisor meetings |
| `persona:human-resources-specialist` | Internal HR specialist or advisor answering client questions | Complete, sourced answers to repeat and edge-case questions | Expert | HR 360 agent, internal search |

| Persona | User group | Typical benefits and retirement needs | Expertise | Channel |
|---|---|---|---|---|
| `persona:benefits-specialist` | Internal benefits operations | Plan rules, carrier terms, enrollment exceptions | Expert | Internal search, HR 360 agent |
| `persona:retirement-plan-specialist` | Internal 401(k) services | Plan document provisions, limits, testing, distributions | Expert | Internal search, HR 360 agent |
| `persona:payroll-specialist` | Internal payroll operations (Section 3.8 example) | Deduction timing, compensation definitions, corrections | Practitioner | Payroll application, internal search |
| `persona:compliance-analyst` | Internal compliance and audit (`persona:compliance-auditor` in Section 3.8) | Which norm governs a step; evidence of answers given | Expert | Operations Workbench, internal search |
| `persona:insights-analyst` | Enterprise Analytics consumer of HR 360 measures | Question trends, answer quality, gaps by persona and domain | Practitioner | Dashboards, SPARQL endpoint |

Design-time roles (knowledge engineer, data steward, SME, change reviewer) are roles in the operating model (Appendix E.1), not personas.

### 4.4. SSOT: Initial Rules, Constraints, Logic, and Calculations

Forty items, each authored once in the SSOT repository (Section 3.5) and generated into SHACL, DMN, or typed interfaces. Values that change by year or by plan (limits, match formulas, waiting periods, vesting schedules) are **versioned parameters** with an effective period and a source citation; none is typed into a prompt or into code.

**Constraints** (SHACL; what must always be true)

| ID | Constraint |
|---|---|
| C-01 | A Benefit Plan has exactly one Plan Sponsor, one Plan Administrator, and at least one Plan Year. |
| C-02 | An Election references exactly one Plan, one Plan Year, and one Coverage Tier. |
| C-03 | A covered Dependent has a relationship type from the dependent relationship scheme. |
| C-04 | A deferral election percentage lies within the plan's minimum and maximum parameters. |
| C-05 | A Qualifying Life Event has an event date and an election window end date after it. |
| C-06 | A Vesting Schedule's percentages never decrease with years of service and end at 100%. |
| C-07 | Every Legal Norm has a jurisdiction, an effective period, and a source citation. |
| C-08 | Every Digital Asset has an owner, a classification, a source system, and an access URL. |
| C-09 | Every SKOS concept in `benefits:` has a preferred label, a definition, and a path to a core class. |
| C-10 | A persona label never encodes a protected attribute (Section 3.8). |
| C-11 | Every HR 360 answer record cites at least one governed IRI, and a source asset for every fact it states. |
| C-12 | Every SSOT parameter has an effective-from date, an owner, and a source citation. |

**Rules** (DMN decision tables behind the Decision API; what decision follows)

| ID | Rule |
|---|---|
| R-01 | Health and welfare eligibility: by employee class, hours of service, and waiting period. |
| R-02 | Enrollment window: open enrollment, new-hire window, or qualifying-life-event window. |
| R-03 | Allowed election change: which changes a given qualifying life event permits. |
| R-04 | Retirement plan eligibility: by age and service as the plan document defines. |
| R-05 | Auto-enrollment: applies when the plan feature is on and the employee has made no election. |
| R-06 | Catch-up eligibility: by age in the plan year, per the current IRS parameter. |
| R-07 | Vesting percentage: by years of vesting service and the plan's schedule. |
| R-08 | COBRA notice: which events require a notice and to whom. |
| R-09 | ACA full-time status: by measurement method and hours. |
| R-10 | Hardship and loan availability: by plan provisions and reason category. |
| R-11 | Answer policy: a question classified as advice-seeking gets information plus a referral, never a recommendation. |
| R-12 | Escalation: questions touching legal interpretation, disputes, or personal financial hardship route to a person. |
| R-13 | Entitlement scope: an employee sees only their own elections; a client administrator only their worksite. |
| R-14 | Source precedence: plan document over SPD over FAQ over knowledge article when sources conflict. |

**Calculations** (declarative formulas with test vectors)

| ID | Calculation |
|---|---|
| K-01 | Employer match = match formula applied to the employee's deferral and plan compensation, up to the plan's cap. |
| K-02 | Vested balance = employer-source balance × vesting percentage (R-07). |
| K-03 | Per-paycheck deduction = employee's annual premium share ÷ pay periods remaining in the plan year. |
| K-04 | Remaining deferral room = annual deferral limit (+ catch-up if R-06) − year-to-date deferrals. |
| K-05 | Years of vesting service = counted plan years meeting the plan's hours threshold. |
| K-06 | Election window end = event date + plan-defined window length. |
| K-07 | COBRA premium = plan cost plus the administrative fee the law allows. |
| K-08 | ACA affordability check = employee's lowest-cost self-only premium against the current affordability parameter. |

**Logic** (code that references SSOT terms and versions)

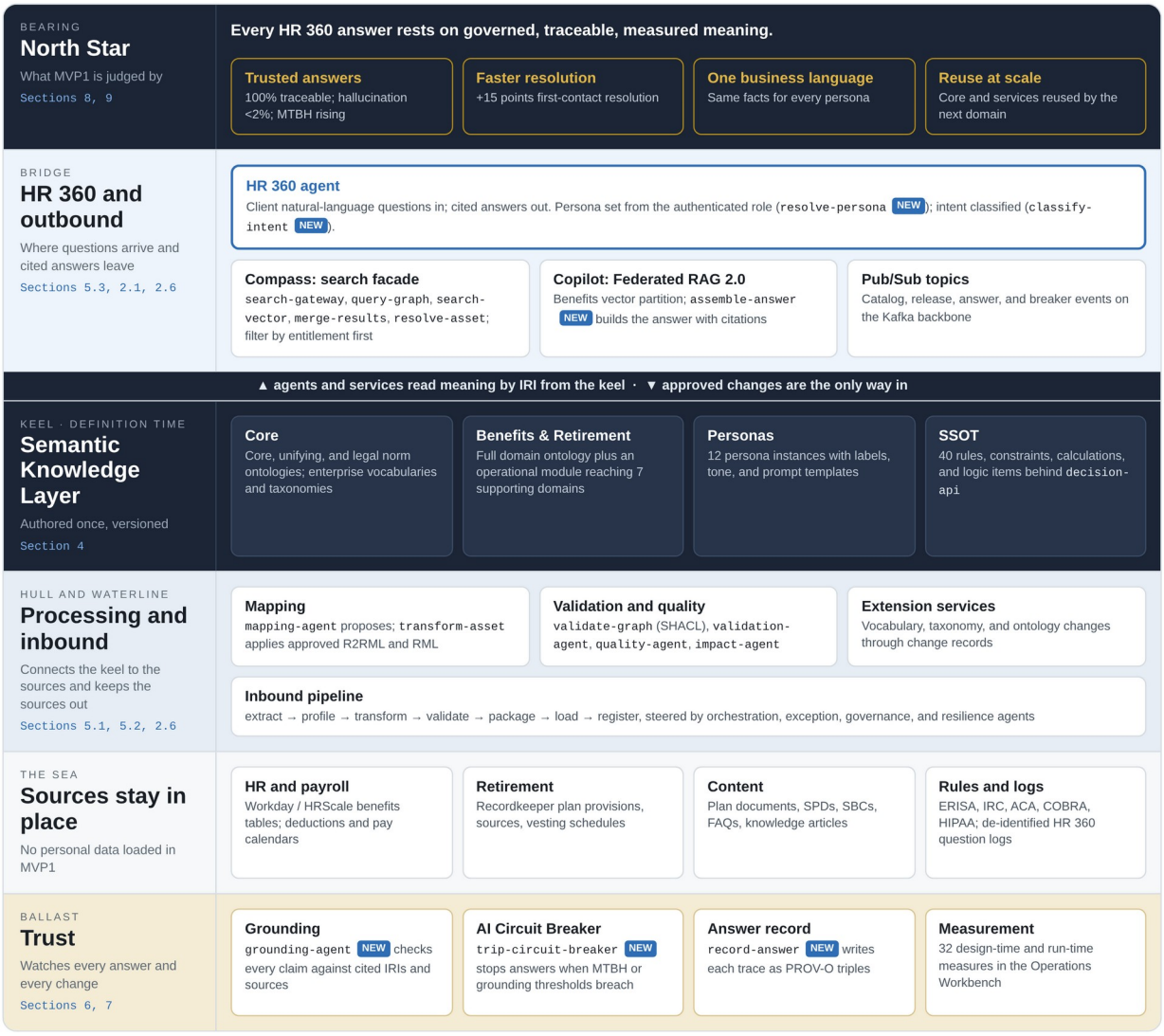
| ID | Logic |
|---|---|
| L-01 | Life-event flow: event → R-03 allowed changes → K-06 window → election → payroll deduction. |

| ID | Logic |
|---|---|
| L-02 | Termination flow: event → R-08 COBRA notice → final deductions → loan and distribution options. |
| L-03 | Question flow: authenticated role → persona → intent (R-11, R-12) → domain routing → retrieval. |
| L-04 | Answer assembly: retrieved facts + Decision API results + citations → answer → C-11 check. |
| L-05 | Conflict handling: two sources disagree → R-14 → answer with the winning source and log the conflict as a defect. |
| L-06 | Parameter refresh: a new IRS or plan parameter is approved → new version effective on its date; prior answers keep the version they used. |

SMEs maintain constraints through data-dictionary forms and rules as DMN grids, and approve calculation test vectors (Section 3.5).

## 5. Semantic Services (Objective O2)

MVP1 reuses the Section 2.6 microservices, agents, and search facade unchanged, configures them for benefits sources, and adds six HR 360 services (marked **new**). Microservices do deterministic work; agents decide; nothing writes meaning anywhere but the keel.




MVP1 service architecture on the Keel and North Star layers. NEW marks the six services added for HR 360; all others are reused from Section 2.6.

Questions go down to the keel and cited answers come back up; changes rise from sources only through mapping, validation, and approval. The ballast watches both directions and reports to the North Star scorecard.

## 5.1. Inbound: Data Structures and Content Structures

| Source (confirm in Week 1) | Structure | What it brings to the SKL | Mapped by |
|---|---|---|---|
| HR system of record (Workday / HRScale): benefits plans, elections, dependents, beneficiaries | Data: tables or reports | Plan, Election, Dependent, Beneficiary instances (metadata and test data only in MVP1) | R2RML in RDF Studio |
| Payroll: deduction codes and pay calendars | Data | Payroll Deduction, Pay Period | R2RML |
| Retirement recordkeeper extract: plan provisions, sources, vesting schedules | Data: file feed | Retirement Plan, Contribution sources, Vesting Schedule parameters | RML |
| Client plan selections by | Data | Client Plan Selection, | R2RML |

| Source (confirm in Week 1) | Structure | What it brings to the SKL | Mapped by |
|---|---|---|---|
| worksite | | Participating Employer | |
| Plan documents, SPDs, SBCs, enrollment guides | Content: PDF, HTML | Digital Asset descriptions, concepts and steps each asset is about, passages for the vector partition | profile-asset and mapping-agent proposals, steward review |
| Benefits FAQs, knowledge articles, HR 360 answer library | Content | Same, plus persona labels harvested by vocabulary-agent | Same |
| Regulatory references (ERISA, IRC, ACA, COBRA, HIPAA, state) | Content | Legal Norm individuals and governedBy links | Compliance-authored, KE-modeled |
| HR 360 question logs (de-identified) | Data | Competency questions, intents, persona vocabulary, baseline measures | vocabulary-agent, steward review |

Inbound runs the Section 2.6 flow (receive-request → locate-asset → access-asset → extract-asset → profile-asset → transform-asset → validate-graph → package-graph → load-graph → register-asset), steered by the orchestration-agent, exception-agent, governance-agent, and resilience-agent. No personal data is loaded in MVP1: instance data is synthetic or de-identified.

### 5.2. Processing: Mapping, Validation, Extension, Decisions

| Service or agent | Kind | Does in MVP1 | Keel and North Star layer |
|---|---|---|---|
| transform-asset and mapping registry | Microservice | Applies approved R2RML and RML mappings | Hull |
| mapping-agent | Agent | Proposes mappings for unfamiliar fields and documents; steward approves | Hull, waterline |
| validate-graph and validation-agent | Microservice and agent | Runs SHACL C-01 to C-12; separates data issues from real defects | Ballast |
| quality-agent, impact-agent | Agents | Checks naming, definitions, orphans; lists what each change affects | Waterline |
| vocabulary-agent | Agent | Harvests candidate alternate and persona labels from question logs | Waterline |
| governance-agent | Agent | Sets classification, access labels, and retention for each asset graph | Ballast |
| Vocabulary, taxonomy, and ontology extension services | Microservices (stubs in the 2025 SOW, working in MVP1) | Create, read, update, and deprecate SKOS concepts and OWL classes through change records | Keel (via change control) |
| decision-api | Microservice | Resolves SSOT rule and parameter versions (R-01 to R-14, K-01 to K-08) and returns result, version, and citation | Keel |
| **resolve-persona** (new) | Microservice | Sets the persona from the authenticated role, never from the prompt | Bridge |
| **classify-intent** (new) | Microservice | Classifies the question against the question intent taxonomy; applies R-11 | Bridge |

| Service or agent | Kind | Does in MVP1 | Keel and North Star layer |
|---|---|---|---|
| | | and R-12 | |

### 5.3. Outbound: Pub/Sub, Search, and RAG 2.0

| Service or agent | Kind | Does in MVP1 | Keel and North Star layer |
|---|---|---|---|
| search-gateway, search-index, search-vector, query-graph, merge-results, resolve-asset | Microservices (Section 2.6) | Single entry point; filter by entitlement first, then search by meaning, then resolve to source | Bridge (Compass) |
| query-planning-agent, federation-agent, result-explanation-agent | Agents (Section 2.6) | Turn a question into a graph query using persona vocabulary; plan federation; explain each match | Bridge (Compass) |
| Benefits vector partition | Data service (Section 2.5) | Domain partition for Federated RAG 2.0: ontology, mappings, approved question and answer pairs, feedback | Bridge (Copilot), hull |
| **assemble-answer** (new) | Microservice | Builds the HR 360 answer from retrieved facts and decision-api results; attaches citations; enforces C-11 | Bridge (Copilot) |
| **grounding-agent** (new) | Agent | Checks every claim in a draft answer against cited IRIs and sources; blocks or escalates ungrounded claims | Ballast |
| **trip-circuit-breaker** (new) | Microservice | AI Circuit Breaker: stops or degrades HR 360 answers in a domain when hallucination or grounding thresholds are breached (Section 6) | Ballast |
| **record-answer** (new) | Microservice | Writes each answer's trace (persona, intent, template, retrieved IRIs, rule versions, sources, model) as PROV-O triples | North Star |
| HR 360 integration | MCP tools or REST | HR 360 calls search-gateway and assemble-answer; no direct graph or vector access | Bridge |

**Pub/Sub topics** (Section 2.6 naming conventions)

| Topic | Published by | Consumed by |
|---|---|---|
| `knowledge.catalog.asset.{registered \| updated \| withdrawn}.v1` | register-asset | search-index, search-vector |
| `knowledge.release.ontology.published.v1` (new) | Release pipeline | search-index, search-vector, query-planning-agent caches |
| `knowledge.release.ssot.published.v1` (new) | SSOT pipeline | decision-api, assemble-answer |

| Topic | Published by | Consumed by |
|---|---|---|
| `knowledge.answer.recorded.v1` (new) | record-answer | Measurement services (Section 6), Operations Workbench |
| `knowledge.answer.flagged.v1` (new) | HR 360 feedback, grounding-agent | Run-time issue queue (Section 3.6), feedback validation controls (Section 2.1) |
| `knowledge.breaker.{tripped \| reset}.v1` (new) | trip-circuit-breaker | HR 360, SG Operations, Operations Workbench |

## 6. Monitoring & Measurement (Objective O3)

Thirty-two measures make up the MVP1 metrology: 14 at design time and 18 at run time, each tagged to one of four dimensions (Trust, Quality, Accuracy, Completeness). Every measure is an individual of the measure model (Section 4.1), computed from triples or event topics, and shown in the Operations Workbench (Section 1.3). Targets extend Section 3.6 and are starting values to calibrate at Gate 2.

### 6.1. Design-Time Measures

| ID | Measure | Inputs | Calculation | Output and initial target | Dimension |
|---|---|---|---|---|---|
| DT-01 | Competency question coverage | CQ catalog; CQs with an owning concept | CQs linked to at least one SKL concept ÷ all CQs | % per release; 100% by Gate 2 | Completeness |
| DT-02 | Competency question pass rate | CQ SPARQL tests; release build | Passing CQ tests ÷ all CQ tests | % per release; ≥80% at Gate 2, 100% before Gate 4 | Accuracy |
| DT-03 | Definition completeness | SKOS concepts in scope | Concepts with prefLabel, definition, and source ÷ all concepts | %; 100% | Completeness |
| DT-04 | Orphan term rate | Concepts; subclass paths | Concepts with no path to a core class ÷ all concepts | %; 0% | Quality |
| DT-05 | SHACL conformance | Shapes C-01 to C-12; test graphs | Conforming focus nodes ÷ all focus nodes | %; 100% on release | Quality |
| DT-06 | Mapping coverage | In-scope inbound schema elements; mapping registry | Mapped elements ÷ in-scope elements | %; ≥90% (Section 3.6) | Completeness |
| DT-07 | Mapping acceptance | mapping-agent proposals; steward decisions | Accepted ÷ proposed | %; ≥80% (Section 3.6) | Accuracy |
| DT-08 | SSOT test vector pass rate | K-01 to K-08 test vectors; generated code | Passing vectors ÷ all vectors | %; 100% before release | Accuracy |
| DT-09 | DMN | R-01 to R-14 decision | Tables with no gaps or | %; 100% | Completeness |

| ID | Measure | Inputs | Calculation | Output and initial target | Dimension |
|---|---|---|---|---|---|
| | completeness | tables | overlaps ÷ all tables | | |
| DT-10 | Source citation coverage | Legal norms, SSOT parameters, assertions needing a source | Items with a source citation ÷ items needing one | %; 100% | Trust |
| DT-11 | Persona label coverage | Top question terms from logs; persona labels | Top 200 user terms matched to a concept label ÷ 200 | %; ≥90% | Completeness |
| DT-12 | Change lead time | Change records: approved and released timestamps | Median of (released − approved) | Days; ≤14 for MINOR (Section 3.3) | Quality |
| DT-13 | Defect escape rate | Defects found before and after release | After-release defects ÷ changes released | %; below 5% (Section 3.6) | Quality |
| DT-14 | Approval traceability | Released artifacts; change records with approver | Released artifacts with an approved change record ÷ all released | %; 100% | Trust |

## 6.2. Run-Time Measures

| ID | Measure | Inputs | Calculation | Output and initial target | Dimension |
|---|---|---|---|---|---|
| RT-01 | Answer traceability | `knowledge.answer.recorded.v1` | Answers citing a governed IRI and a source per fact ÷ all answers | %; 100% (Section 3.6) | Trust |
| RT-02 | Grounded claim rate | grounding-agent claim checks | Claims supported by a cited IRI or source ÷ all claims | %; ≥98% | Trust |
| RT-03 | Hallucination rate | Gold-set review; flagged answers confirmed by SME | Answers with an unsupported or false claim ÷ answers reviewed | %; below 2% on the gold set | Accuracy |
| RT-04 | Mean Time Between Hallucinations (MTBH) | Confirmed hallucination timestamps; answer volume | Answers served (or hours in service) between confirmed hallucinations | Answers per hallucination; rising trend; breaker threshold set at Gate 2 | Trust |
| RT-05 | Circuit breaker trips | `knowledge.breaker.tripped.v1` | Count and duration of trips per domain | Count and minutes; every trip has a root cause within 2 business days | Trust |
| RT-06 | Answer accuracy (gold set) | Gold set of 150 SME-approved question and answer pairs | Answers judged correct and complete ÷ gold-set questions | %; ≥90% at Gate 4 | Accuracy |
| RT-07 | Answer completeness | Gold-set rubric (all required facts present) | Required facts present ÷ required facts | %; ≥95% | Completeness |

| ID | Measure | Inputs | Calculation | Output and initial target | Dimension |
|---|---|---|---|---|---|
| RT-08 | Rule-version correctness | Answer traces; SSOT versions effective on the question date | Answers using the effective rule and parameter version ÷ answers using a rule | %; 100% | Accuracy |
| RT-09 | Policy compliance | Answers classified advice-seeking or escalation (R-11, R-12) | Answers that followed the policy ÷ answers the policy applied to | %; 100% | Trust |
| RT-10 | Entitlement violations | search-gateway and answer audit logs | Results or answers shown outside the user's entitlements | Count; 0 | Trust |
| RT-11 | Search success | Searches; user acceptance signal | Searches with an accepted result ÷ all searches | %; ≥85% (Section 3.6) | Quality |
| RT-12 | Zero-result rate | Search logs | Searches with no result ÷ all searches | %; below 5% | Completeness |
| RT-13 | First-contact resolution | HR 360 sessions; escalations and repeat questions within 7 days | Sessions resolved without human handoff or repeat ÷ all sessions | %; baseline at Gate 1, +15 points at Gate 4 | Quality |
| RT-14 | Answer latency | Answer events | 95th percentile time from question to answer | Seconds; ≤2 for search, ≤8 for assembled answers | Quality |
| RT-15 | User-rated helpfulness | HR 360 ratings by persona | Positive ratings ÷ rated answers | %; baseline at Gate 1, +10 points at Gate 4 | Quality |
| RT-16 | Low-confidence queue age | Review queue items (Section 2.1 controls) | Median age of open items | Days; ≤3 | Quality |
| RT-17 | Source freshness | Asset catalog; source change events | Time from source change to SKL update | Business days; ≤1 (Section 3.6) | Completeness |
| RT-18 | Pipeline autonomy | Inbound pipeline run outcomes | Runs completed without manual intervention ÷ all runs | %; ≥99.5% (Section 3.6) | Quality |

### 6.3. How Measures Are Produced and Used

1. Each measure's definition, inputs, formula, and target are knowledge artifacts under change control, so a changed formula is a new version, not a silent edit.
2. Run-time inputs arrive on the event topics in Section 5.3; design-time inputs come from the release pipeline and change records.
3. A scheduled job writes each observation as triples linked to the measure, the release, and the period, and publishes it to the Operations Workbench.

4. A breach opens an issue (Section 3.6). RT-02, RT-03, and RT-04 also feed the AI Circuit Breaker: a breach trips trip-circuit-breaker for that domain until the cause is fixed.
5. Measures are reviewed weekly from Week 6; the full set is reported at Gates 3 and 4.

## 7. End-to-End Traceability, Trust & Compliance (Objective O4)

At Gate 4 any HR 360 answer can be traced back to the requirement that asked for it and forward to the measure that judged it, with one SPARQL query against the answer record. The chain extends the Section 3.6 traceability chain with the answer itself:

```
Requirement → Competency question → Change record → Release (ontology + SSOT versions)
  → Question (persona, intent) → Retrieved IRIs + decision-api results (rule, parameter,
version)
  → Source assets + legal norms → Answer → Grounding check → Measure observation
  → Feedback or defect → New requirement
```

### 7.1. Demonstration Scenarios

Five scripted scenarios are run live at Gate 4, each before (current HR 360) and after (HR 360 on the SKL).

| # | Persona | Question | What the trace must show |
|---|---|---|---|
| D1 | WorksiteEmployee | "Am I eligible for the company 401(k) match, and when am I fully vested?" | R-04, R-07, K-02, and K-05 with versions; the plan provision cited from the plan document; the answer limited to the employee's own plan |
| D2 | LifeEventEmployee | "We just had a baby. Can I add her to my medical plan, and by when?" | Qualifying Life Event → R-03 → K-06 window; the SPD passage cited; the payroll deduction step it triggers (L-01) |
| D3 | ClientHRAdministrator | "An employee is terminating Friday. What do we owe him on benefits?" | L-02: R-08 COBRA notice, the governing legal norm, final deduction, distribution options; employer and administrator obligations separated |
| D4 | WorksiteEmployee | "Should I move my 401(k) into the Roth option?" | R-11 fires: information on both options with sources, no recommendation, referral offered; RT-09 logs compliance |
| D5 | HRSpecialist | Same as D1, asked after a new plan-year parameter is released mid-demonstration | L-06: the new parameter version is used for new answers; D1's earlier answer still shows the version it used |

A sixth, unscripted run triggers the AI Circuit Breaker by forcing an ungrounded answer in test, showing the trip, the HR 360 fallback message, the issue raised, and the reset.

### 7.2. Trust Controls Demonstrated

| Control | Where it lives | Evidence at Gate 4 |
|---|---|---|
| Persona from identity, filter before tailoring | resolve-persona, search-gateway (Section 3.8) | Same question from two personas: same facts, different labels and depth; no out-of-entitlement result (RT-10 = 0) |
| Every claim grounded or withheld | grounding-agent, C-11 | Answer record with claim-by-claim citations; a blocked claim in the log |

| Control | Where it lives | Evidence at Gate 4 |
|---|---|---|
| No advice; escalate when required | R-11, R-12 | D4 and an escalation case in the answer log |
| Nothing learned without approval | Feedback validation controls (Section 2.1) | A user correction moving through the review queue to a change record |
| Nothing authoritative without review | Change control (Section 3.3) | DT-14 = 100%; the change record behind each rule used in D1 to D5 |
| Stop when trust drops | AI Circuit Breaker with MTBH (Section 6) | The sixth run above |

### 7.3. Compliance Alignment

| Framework | What MVP1 demonstrates |
|---|---|
| ISO/IEC 42001 (AI management system) | Answer traceability, human oversight, documented risk controls, and monitoring (Sections 3.6 and 3.8) |
| NIST AI RMF (Govern, Map, Measure, Manage) | Measures in Section 6 mapped to Measure; the circuit breaker and escalation mapped to Manage |
| ISO 30401 and ISO/IEC 5392 | Knowledge lifecycle and engineering steps (Section 3.4) applied to the Benefits domain |
| ERISA, IRC, ACA, COBRA, HIPAA as they govern benefits steps | Each in-scope step governedBy its norm; answers cite the norm or plan document and never interpret beyond it |
| Privacy | No personal data loaded; de-identified logs; entitlement filtering tested |

## 8. Service Level Improvement: Before & After (Objective O5)

"Before" is measured in Weeks 1 and 2 by running the 150-question gold set and a sample of real HR 360 questions through the current HR 360 agent; "after" is the same set at Week 12 on the SKL, with the same questions, reviewers, and rubric. Targets are proposed here and confirmed at Gate 1 once the baseline is known.

| Lens | Measure | Before (today) | After (Week 12 target) | Source |
|---|---|---|---|---|
| **Customer Value:** customer outcomes | Answer accuracy on the gold set | Unmeasured; baselined at Gate 1 | ≥90%, and at least 20 points above baseline | RT-06 |
| Customer Value | First-contact resolution | Baselined at Gate 1 | +15 points | RT-13 |
| Customer Value | User-rated helpfulness | Baselined at Gate 1 | +10 points | RT-15 |
| Customer Value | Same answer for the same question across channels and personas | Not assured: answers drawn from separate content | 100% of gold-set facts identical across personas | RT-06 by persona |
| **Business Operations:** cost | HR specialist minutes per benefits inquiry | Baselined from ticket and handle-time data | −25% | Ticketing data, RT-13 |

| Lens | Measure | Before (today) | After (Week 12 target) | Source |
|---|---|---|---|---|
| Business Operations: cost | Escalations to benefits and retirement specialists | Baselined at Gate 1 | −20%, with all required escalations still made | RT-09, ticketing |
| **Business Operations:** agility | Time to reflect a plan or rule change in HR 360 answers | Content rewrite and retest; weeks | One business day or less after approval | RT-17, DT-12 |
| Business Operations: agility | Time to add the next domain | No reusable pattern | Core, measures, and services reused unchanged; only the domain module and mappings added | Gate 4 reuse inventory |
| **Business Operations:** quality | Hallucination rate on the gold set | Baselined at Gate 1 | Below 2% | RT-03 |
| Business Operations: quality | Answer completeness | Baselined at Gate 1 | ≥95% | RT-07 |
| **Governance & Compliance:** trust | Answers traceable to governed terms and sources | Not available | 100% | RT-01 |
| Governance & Compliance: trust | Mean Time Between Hallucinations | Not measured | Measured, trended, and wired to the AI Circuit Breaker | RT-04, RT-05 |
| **Governance & Compliance:** risk | Advice and escalation policy followed | Not measured | 100% | RT-09 |
| Governance & Compliance: risk | Answers using an outdated rule or limit | Not detectable | 0, detectable by query | RT-08 |
| Governance & Compliance: risk | Entitlement violations | Not tested | 0 | RT-10 |
| **Technology & Architecture:** reuse | Share of MVP1 assets reusable by the next domain | None | ≥70% of services; 100% of the core and measure model | Reuse inventory |
| **Technology & Architecture:** integration | Sources reached through the governed inbound pipeline | Point integrations per content set | All in-scope sources through one pipeline and one search facade | Asset catalog |
| **Technology & Architecture:** interoperability | Assets published in open standards | Proprietary content formats | 100% of SKL assets in RDF, OWL, SKOS, SHACL, and DMN; consumable by Copilot, Compass, and HR 360 alike | Release manifest |

## 9. MVP1 Operational Metrics by Strategic Outcome

Eighteen operational metrics, three per Strategic Outcome, form the MVP1 scorecard. Each rolls up from the measures in Section 6, so the scorecard adds no new data collection.

| Strategic Outcome | Operational metric | Definition | MVP1 target (Week 12) | Built from |
|---|---|---|---|---|
| **Trusted BI & AI:** smarter, more reliable AI and analytics | Traceable answer rate | Answers citing governed terms and a source per fact | 100% | RT-01 |
| Trusted BI & AI | Hallucination rate and MTBH | Confirmed unsupported claims; answers between them | Below 2%; MTBH rising week over week | RT-03, RT-04 |
| Trusted BI & AI | Grounded claim rate | Claims backed by a cited IRI or source | ≥98% | RT-02 |
| **Improved Decision Making:** more informed decisions, faster | Time to a usable answer | 95th percentile question-to-answer time | 8 seconds or less | RT-14 |
| Improved Decision Making | First-contact resolution | Sessions resolved without handoff or repeat | +15 points over baseline | RT-13 |
| Improved Decision Making | Decision explainability | Rule-based answers showing the rule, version, and inputs used | 100% | RT-08, record-answer |
| **Business Literacy:** a single source of truth for business language | Definition completeness | In-scope concepts with label, definition, and source | 100% | DT-03 |
| Business Literacy | User-term match | Top 200 user terms that resolve to a governed concept | ≥90% | DT-11 |
| Business Literacy | Cross-persona consistency | Gold-set facts identical across personas | 100% | RT-06 by persona |
| **Knowledge Preservation:** living business knowledge that evolves with us | Change lead time | Approved change to released artifact | 14 days or less (MINOR) | DT-12 |
| Knowledge Preservation | Source freshness | Source change to SKL update | One business day or less | RT-17 |
| Knowledge Preservation | Feedback-to-knowledge conversion | Approved user corrections that became a released change record | At least 50% of approved corrections released within the term | Review queue, change records |
| **Meaning at Scale:** future-proof, standards-based knowledge | Standards conformance | SKL assets in open standards and passing SHACL | 100% | DT-05, release manifest |
| Meaning at Scale | Competency question pass rate | CQ tests passing per release | 100% before Gate 4 | DT-02 |

| Strategic Outcome | Operational metric | Definition | MVP1 target (Week 12) | Built from |
|---|---|---|---|---|
| Meaning at Scale | Core reuse readiness | Core and measure model used unchanged by the next domain's design | 100% | Gate 4 reuse inventory |
| **Align Business with Technology:** one semantic layer connecting people and systems | Mapping coverage | In-scope source elements mapped to the SKL | ≥90% | DT-06 |
| Align Business with Technology | Single entry point adoption | HR 360 retrieval calls made through search-gateway | 100% | search-gateway logs |
| Align Business with Technology | Approval traceability | Released artifacts with a business-approved change record | 100% | DT-14 |

# 10. Deliverables, Acceptance, Team & Terms

## 10.1. Deliverables

| # | Deliverable | Due | Objective |
|---|---|---|---|
| 1 | Competency question catalog (Benefits & Retirement) and Ontology Requirements Specification (ORSD) | Week 2 | O1 |
| 2 | Gold set: 150 SME-approved question and answer pairs, tagged by persona and intent | Week 2 | O5 |
| 3 | Baseline measurement report ("before") | Week 2 | O5 |
| 4 | Core ontology, unifying ontology, legal norm module, core vocabularies and taxonomies (Section 4.1), released v0.5 | Week 6 | O1 |
| 5 | Benefits & Retirement domain ontology, vocabularies, taxonomies, and operational module (Section 4.2), released v0.5 | Week 6 | O1 |
| 6 | Persona vocabulary: 12 personas with labels and prompt templates (Section 4.3) | Week 6 | O1 |
| 7 | SSOT v0.5: 40 items with shapes, DMN tables, test vectors, and versioned parameters (Section 4.4) | Week 6 | O1 |
| 8 | Inbound pipeline configured for in-scope sources, with approved mappings (Section 5.1) | Week 7 | O2 |
| 9 | Processing and outbound services, including the six new services and topics (Sections 5.2 and 5.3) | Week 9 | O2 |
| 10 | Benefits vector partition and HR 360 integration in test | Week 9 | O2 |
| 11 | Metrology: 32 measures live in the Operations Workbench (Section 6) | Week 9 | O3 |
| 12 | AI Circuit Breaker configured with MTBH thresholds | Week 10 | O3, O4 |
| 13 | SKL v1.0 and services in stage for the demonstration | Week 11 | O1, O2 |
| 14 | Traceability, trust, and compliance demonstration (Section 7) | Week 12 | O4 |

| # | Deliverable | Due | Objective |
|---|---|---|---|
| 15 | After measurement report and Strategic Outcome scorecard (Sections 8 and 9) | Week 12 | O5 |
| 16 | MVP2 recommendation: next domain, reuse inventory, production path | Week 12 | All |

### 10.2. Gates & Acceptance Criteria

| Gate | Week | Passes when | Approver |
|---|---|---|---|
| G1: Baseline | 2 | Deliverables 1 to 3 approved; targets in Sections 8 and 9 confirmed against the baseline | Enterprise AI Strategy & Solutions; Enterprise Analytics |
| G2: Keel released | 6 | Deliverables 4 to 7 released through the pipeline; DT-02 ≥80%, DT-03 = 100%, DT-05 = 100%, DT-08 = 100% | KE lead; Benefits and Compliance SMEs; Architecture |
| G3: Services live in test | 9 | HR 360 answers gold-set questions end to end through the SKL; RT-01 = 100% in test; measures reporting | KE lead; HR 360 product owner |
| G4: Proof of Capability | 12 | Section 7 scenarios pass live; the after report shows the Section 8 targets met or explains each gap; MVP2 recommendation delivered | Enterprise AI Strategy & Solutions; Enterprise Analytics |

Each deliverable is accepted when:

- For any knowledge artifact: it is released through change control with an approver (Section 3.3).
- For any service: it answers working calls against its documented contract with sample data, and its measures are reporting.
- For documentation: it is complete and clear as agreed with the client lead.

### 10.3. Project Team & Responsibilities

| Role | Responsibility | Commitment |
|---|---|---|
| Executive sponsor (Enterprise AI Strategy & Solutions) | Direction, gate approvals, removing blockers | Gates and biweekly steering |
| Client lead (Enterprise Analytics) | Owns outcomes and the scorecard; accepts deliverables | Weekly |
| Senior semantic architect (KE lead) | Architecture, SKL design, keel integrity; delivery lead | Full time |
| Knowledge engineers (2) | Ontologies, vocabularies, taxonomies, mappings, SHACL, SSOT | Full time |
| Semantic services engineer | Microservices, agents, topics, HR 360 integration | Full time |
| Data steward (Benefits & Retirement) | Labels, definitions, mapping review | 50% |
| Benefits SME and retirement plan SME | Competency questions, gold set, rule and test vector approval | 4 to 6 hours a week each |
| Compliance and Legal reviewer | Legal norms, answer policy, escalation rules | 2 to 4 hours a week |

| Role | Responsibility | Commitment |
|---|---|---|
| HR 360 product owner and engineer | Agent integration, feedback signals, demonstration | 25% |
| Enterprise Analytics analyst | Baseline, measures, scorecard | 50% |

### 10.4. Assumptions & Dependencies

- Access in Week 1 to the HR 360 test environment, de-identified question logs, plan documents and SPDs, and metadata for the sources in Section 5.1.
- Tooling available by Week 2: RDF Studio, source control, graph store, Kafka, vector store, and Claude.ai for drafting (drafts are reviewed and never committed by AI on its own; Appendix E).
- The HR 360 agent can call external tools (MCP or REST) and pass the authenticated user role.
- No personal data is required; synthetic or de-identified data is sufficient for the Proof of Capability.
- Each of the seven supporting domains provides an SME contact for slice-level review only.

### 10.5. Risks

| Risk | Effect | Mitigation |
|---|---|---|
| SME time is short during open enrollment and the holidays | Gold set or rule approval slips | Gold set fixed by Week 2; holiday weeks used for build work that needs no SMEs |
| Plan provisions vary by worksite client and plan | Rules multiply; answers apply to the wrong plan | Parameters keyed by plan and plan year; MVP1 limited to 2 to 3 representative plans |
| Source access or tooling arrives late | Weeks 3 to 7 compress | Week 1 access checklist owned by the sponsor; synthetic data as fallback |
| Baseline is not comparable to the after run | Improvement cannot be shown | Same gold set, reviewers, and rubric before and after; rubric frozen at Gate 1 |
| Answers read as advice | Compliance exposure | R-11 and R-12, Legal review of the answer policy, RT-09 at 100% |
| Scope pull from the seven supporting domains | Benefits depth suffers | Depth fixed by Section 2; additions go to the MVP2 backlog |

### 10.6. SOW Amendments & Open Items

Any change to this SOW is documented in a change request and approved by the executive sponsor and the client lead before implementation. Effort is internal to the organization; no pricing section applies. The following items must be completed before signature or by the date shown:

| Item | Owner | Needed by |
|---|---|---|
| Named team members and SMEs | KE lead; HR Operations | SOW signature |
| In-scope plans (2 to 3) and source systems confirmed | Benefits SME; KE lead | Week 1 |
| HR 360 integration path (MCP or REST) and test environment | HR 360 product owner | Week 1 |
| Baseline source for handle time and escalations | Enterprise Analytics | Week 2 |

| Item | Owner | Needed by |
|---|---|---|
| Circuit breaker thresholds (RT-02, RT-03, RT-04) | KE lead; Compliance | Gate 2 |

APPENDIX E

# Applying Semantic Standards in the KE Workbench

A stepwise guide to applying the Part III standards with the KE Workbench, from reference ontologies to operations monitoring; checklists and known behaviors, including RDF Studio as an alternative modeling tool; and the coverage of every standard by a workbench page.


E.1 Purpose, Roles, and Setup
E.2 Stepwise Guide
E.3 Checklists and Tool Behaviors
E.4 Workbench Coverage of the Standards

# Purpose, Roles, and Setup

This appendix is a stepwise guide to applying the semantic standards in Part III with the KE Workbench, the Company's implementation of the Semantic Modeling Workbench and the Operations Workbench (Section 1.3). It turns the standards into concrete actions on each workbench page, names the check that proves each one, and records known tool behaviors. RDF Studio remains an alternative for visual modeling and R2RML mapping (Section E.3).

## Roles

| Role | Responsibility in the workbench |
|---|---|
| Knowledge engineer | Ingests sources, resolves conformance findings, builds and extends ontologies, assembles services, and loads releases |
| Data steward | Maintains vocabularies and taxonomies, primitives, labels, and dispositions for their domain |
| Subject-matter expert | Reviews definitions, SIPOCM drafts, personas, jobs, and outcomes; records verdicts and approvals |
| Domain owner | Approves changes to the domain and its catalog entries; owns its primitives |
| Architecture | Owns the core, the area containers with the domain owners, the reference registry, and this guide |
| Change reviewer | Reviews change records and the Operations Dashboard actions; signs off releases |

## What the Workbench Is, and Is Not

- **It is the place where models are edited:** every page reads from and writes to GraphDB; Turtle and TriG files are exported releases of the graph.
- **It is where the standards are checked:** the Part III checks run in Ingestion, Conformance Review, Standards Conformance, and the builders, with the same rules.
- **It is not a run-time component:** nothing in production depends on the workbench; services read the released graphs.

## Workspace Setup

1. Start the workbench server (start-workbench.command, or python3 proxy.py in the KE Workbench folder) and open it at http://localhost:3000/, never from a file. It serves the pages and proxies GraphDB at /graphdb/.
2. Choose the GraphDB repository in the settings (shared by every page). The connection pill shows whether GraphDB is reachable; without it, pages that can work from files do so.
3. Review the ingestion configuration (ingest/ke-ingest.config.json): base URL and namespace rules, approved acronyms and expansions, organizations whose content is removed, publisher, conformsTo (the current edition of this guide), loadBlockingRules, and learnFromReview.
4. For drafted definitions, put the API key in ~/.ke-workbench.env; every drafted definition carries an editorial note asking for review before the term moves to Candidate.

5. Agree the review rule before drafting: no relationship, domain, or range enters a model without a person confirming it.

# Stepwise Guide

Follow the steps in order for a new domain; for a change to an existing one, start at the step that matches the artifact being changed. Each step names the standard it applies, the page, and the check that proves it.

## Step 1. Load the Reference Ontologies and the Core

Standard: BFO and RDF foundations (Section 3.10); design (Section 3.1). Page: Ingestion › Reference ontologies; graphdb-rebuild.command.

- Load BFO 2020, RO Core, and the RO extract (Load all three); each goes into its own named graph under its own IRI.
- Load Baseline Core v1.0 (vocabularies, taxonomies, the core domain ontology, and core shapes) in dependency order; the rebuild script backs up the repository, keeps the change log, and checks lineage.

Check: the lineage check reports every core class reaching BFO and every relation reaching BFO, RO, or IAO.

## Step 2. Ingest Sources

Standard: naming and URI conventions (Section 3.2); asset metadata (Section 3.11). Page: Ingestion.

1. Scan the source folder (Turtle, N-Triples, RDF/XML); each file must hold one owl:Ontology or skos:ConceptScheme. Reference ontologies given here are skipped and sent to Step 1.
2. Set the destination (ttl/ and trig/), author, editor, starting version (0.1.0), and status (Draft).
3. Ingest: identifiers are minted in lower case joined by hyphens, header fields are written, taxonomies are converted to SKOS, taxonomy concepts are linked to their vocabulary terms, and a change record, a crosswalk of old to new IRIs, and an ingestion report are written. Fixes recorded in earlier reviews are replayed.

Check: the report lists every asset with its findings; files with errors open in Conformance Review.

## Step 3. Resolve Findings

Standard: term definition (Section 3.7); labels (Section 3.8); quality gates (Section 3.4). Page: Conformance Review.

- Work through other organizations' content, acronyms, broken and foreign references, missing definitions, terms not in a vocabulary, and concepts not formalized by a class.
- Stage fixes or removals (the removal plan shows what happens to children and links), or record a disposition with a rationale: accept as is, not an issue, defer, or needs a modeling decision.
- Approve acronyms or expansions into the configuration so later ingestions use them.
- Save to files or apply to GraphDB; each save bumps the version, adds a change note, and writes a change record.

Check: no blocking findings remain; every other finding is fixed or dispositioned.

## Step 4. Edit Vocabularies and Primitives

Standard: primitives and term definitions (Section 3.7). Page: Model Editor (Vocabulary).

- Add only primitives the core and other domains lack; search the core first. Mark promotion candidates.
- Complete label, definition (genus and differentia), scope note, alternative labels, and status for every term.

Check: conformance as you type shows no errors; Save & load into GraphDB writes the files, the change record, and the graph.

## Step 5. Arrange Taxonomies

Standard: taxonomy attributes (Section 3.9); polymorphism (Section 3.11). Page: Model Editor (Taxonomy).

- Drag vocabulary terms onto the taxonomy: a dropped term becomes a concept with its primitives as dcterms:subject and the head as core:hasPrimaryTerm; the vocabulary term itself never joins the scheme.
- Use skos:broader only for "is a kind of"; link parents in another scheme with skos:broadMatch.

Check: every concept is in the scheme with one preferred label, top concepts are flagged, and no move creates a cycle.

## Step 6. Build the Domain Ontology

Standard: design and insulation (Section 3.1); relations (Sections 3.9 and 3.10). Page: Model Editor (Ontology).

1. New ontology… creates it at 0.1.0, Draft, importing the core and the area container; it is titled by the naming standard.
2. New class… adds each class under a core or container parent, with its primary term and other primitives; New class from this term… builds a class from a primitive or concept, or records core:formalizedBy.
3. Relate classes with the + handle; the picker lists the ontology's own relations, then core, then BFO and RO relations.
4. Refer only to the core, the area container, parent domains, the domain itself, and the APQC registry.

Check: the tiers show the domain's relations as primary and the imports as secondary; the build reports no insulation error and every class reaches BFO.

## Step 7. Assemble Services and Processes

Standard: services, processes, and the reference registry (Section 3.12). Page: Service Assembly; KE Domains builders.

- Instantiate the APQC activities the domain owns and their steps, with the SIPOCM of each step; classify each by its APQC element or as a PEO extension.
- Assemble each service from the processes that enable it and its delivery step; publish it to the enterprise service catalog.

Check: the topology shows Area › Domain › Service › Process with each step's validation status; shapes and assembly checks pass; shared processes and steps show their copy number.

## Step 8. Connect Personas, Jobs, and Outcomes

Standard: personas, jobs, and demand (Section 3.13). Pages: Appreciative Inquiry, Buyer Personas & Jobs, PEO Job Study, PEO Outcome Review, PEO Outcome Playbook, Demand Gateway, PEO Service Map.

- Code interviews into jobs, job steps, and desired outcomes; load survey scores; have business and compliance SMEs review every item; admit demand requests through the gateway.

Check: every desired outcome has a review decision, every admitted request passes the six gateway rules, and the PEO Service Map lists the questions no service answers.

## Step 9. Load, Record, and Release

Standard: change records and the release pipeline (Section 3.3). Pages: Load into GraphDB; load scripts; Release Notes.

1. Load: each asset replaces its named graph, and the change records go to the change log.
2. Confirm the change record names the approver and the requirement (required extensions).
3. Record the release: owl:versionInfo and skos:changeNote on each asset, and the release in the workbench release notes when pages changed.
4. Move an asset to Candidate only when it meets the entry criteria (Section 3.3).

Check: Standards Conformance against the loaded graphs returns no errors.

## Step 10. Monitor and Improve

Standard: target operating model (Section 3.6); process policies (Section 3.4). Page: Operations Dashboard.

- Rebuild the snapshot after each working session; review new findings, policy failures, and the causes of each.
- Work the action list by priority; mark each action's status; export it for the change review.

Check: the number of open high-priority actions falls from one snapshot to the next.

# Checklists and Tool Behaviors

The checklist below replaces the RDF Studio checklist of v1.9. The known behaviors that follow include those of RDF Studio, kept for teams that model there and import the result through Ingestion (Step 2).

## Standards Checklist

| Standard | Applied in step | Proof |
|---|---|---|
| Base URL, URI patterns, and named graphs (3.2) | Steps 2, 6, 7 | All IRIs start with the Company base URL and follow the patterns; each asset is in the graph named by its IRI |
| Hyphen-joined identifiers and asset titles (3.2) | Steps 2, 4–6 | Identifier and title checks return no findings |
| No acronyms in identifiers or preferred labels (3.2) | Steps 2–4 | Acronym findings resolved; acronyms are alternative labels |
| Design, areas, and insulation (3.1) | Steps 6, 7 | No reference outside the core, the area, parent domains, the domain, and the registry |
| Change records and versions (3.3) | Steps 2–4, 9 | A change record for every change set, with approver and requirement |
| Quality gates and process policies (3.4) | Steps 3, 10 | No blocking findings; every finding dispositioned; policies P01–P09 tracked |
| Operations and improvement (3.6) | Step 10 | Snapshot rebuilt; actions worked |
| Primitives and term definitions (3.7) | Steps 3, 4 | Definitions present; every concept has a primary term; no duplicate primitive |
| Labels and personas (3.8) | Steps 3, 8 | Labels carry language tags and sources; personas carry no protected attributes |
| Taxonomy attributes, ISO 25964 (3.9) | Step 5 | One scheme per file; every concept in scheme with one preferred label |
| BFO stack and relations (3.10) | Steps 1, 6 | Lineage check passes; every property has a parent family |
| Semantic asset metadata (3.11) | Steps 2, 9 | Header fields complete; conformsTo names edition 2.0 |
| Services and processes (3.12) | Step 7 | Shapes and assembly checks pass; SIPOCM complete |
| Personas, jobs, and demand (3.13) | Step 8 | Review decisions recorded; demand admitted by the gateway |

## Known Workbench Behaviors

| Behavior | What to do |
|---|---|
| A page opened from a file cannot reach GraphDB | Open the workbench at http://localhost:3000/ through the workbench server |
| A page shows an old folder after files moved | Remembered folders that no longer exist fall back to the current destination; choose the folder again if needed |
| An update to a page does not appear | The server disables caching; reload the page. Restart the server after |

| Behavior | What to do |
|---|---|
| | updating server code (ingest/) |
| Load into GraphDB stops | A blocking finding is present; resolve it in Conformance Review, or use Load anyway as a recorded exception |
| Editor saves are not on disk | Saves made in GraphDB are exported with the next release; the Operations Dashboard lists records found only in the graph |
| Triples were loaded into the default graph | Pages still read them and show a note; move them into the asset's named graph |

## Using RDF Studio

RDF Studio remains available for visual ontology and taxonomy design and for R2RML mapping. Model there under the v2.0 naming rules (Section 3.2), export, and bring the files in through Ingestion (Step 2), which applies the checks and writes the change record. Mappings build subject IRIs from stable source keys with the instance pattern in Section 3.2.

## Known RDF Studio Behaviors

| Behavior | What to do |
|---|---|
| Model editor reports the ontology is empty | Declare owl:Class and typed properties; individuals and annotations alone do not populate the editor |
| Relationships are not drawn between classes | Give the property both rdfs:domain and rdfs:range; restrictions and SHACL shapes are not drawn as edges |
| Taxonomy file is rejected | Declare exactly one skos:ConceptScheme; add skos:inScheme and one skos:prefLabel to every concept; flag top concepts |
| Mapping import is rejected | Import only mappings exported by RDF Studio; load ontologies and vocabularies through their own imports |
| SPARQL panel fails with "expected CONSTRUCT" | Use SELECT or CONSTRUCT only; make changes in the editors or source files |
| Assistant settings revert after a configuration import | Re-enter the API key and temperature in the settings screen after any import |

# Workbench Coverage of the Standards

This section shows that every workbench page is governed by a standard in this guide and that every standard is applied by a page, so the guide and the workbench have no gaps between them. It also records where each v1.9 divergence is resolved in this edition.

## Pages to Standards

| Group | Page | Standards | Procedure |
|---|---|---|---|
| Model & standards | Model Manager | 3.2, 3.3, 3.11 | E.2 Step 9 |
| | Ingestion (and Reference ontologies) | 3.2, 3.3, 3.7, 3.10, 3.11 | E.2 Steps 1–2 |
| | Conformance Review | 2.7, 3.2, 3.3, 3.4, 3.6, 3.7, 3.8, 3.13 | E.2 Step 3 |
| | Model Editor (Vocabulary) | 3.2, 3.3, 3.7 | E.2 Step 4 |
| | Model Editor (Taxonomy) | 3.7, 3.8, 3.9, 3.11 | E.2 Step 5 |
| | Model Editor (Ontology) | 3.1, 3.7, 3.9, 3.10 | E.2 Step 6 |
| | Service Assembly | 3.1, 3.9, 3.12 | E.2 Step 7 |
| | Relation Taxonomy | 3.9, 3.10 | E.2 Step 6 |
| | Standards Conformance | 2.7, 3.2, 3.3, 3.7, 3.8, 3.11, 3.13, E.3 | E.2 Step 9 |
| Rules & logic | SSOT Rules & Standards Map; SSOT Editor; Rules, Constraints & Logic | 3.3, 3.5 | Section 3.5 |
| | Decision Question Baseline | 3.5, A.4 | Appendix A.4 |
| Agents & operations | Operations Dashboard | 3.3, 3.4, 3.6, 3.11 | E.2 Step 10 |
| | Mapping Agent Dashboard; Agent & Service Map | 1.5, 2.6, 2.7 | Sections 1.5, 2.6 |
| | PLM Lifecycle Dashboard | 3.3, 3.4 | Section 3.3 |
| Customer 360 | Appreciative Inquiry; Buyer Personas & Jobs; PEO Job Study | 3.8, 3.13 | E.2 Step 8 |
| | PEO Outcome Review | 3.3, 3.4, 3.13 | E.2 Step 8 |
| | PEO Outcome Playbook | 3.13, 1.5 | E.2 Step 8 |
| | Demand Gateway | 3.3, 3.5, 3.13 | E.2 Step 8 |
| Products & Services | PEO Service Map | 3.12, 3.13 | E.2 Step 8 |
| | PEO Service Landscape | 3.12 | Section 3.12 |
| Reference | HSGA Governance Overview | 2.1 | Section 2.1 |
| | APQC PCF 7.4 | 3.12 | Section 3.12 |

| Group | Page | Standards | Procedure |
|---|---|---|---|
| | Release Notes; this guide | 3.3 | E.2 Step 9 |

## Standards to Pages

Every Part III section is applied by at least one page; sections without a page are applied by review.

| Section | Applied by | Status |
|---|---|---|
| 3.1 Design | Model Editor (Ontology); Service Assembly; builders | Applied |
| 3.2 Standards | Ingestion; Conformance Review; Standards Conformance; editors | Applied |
| 3.3 Change Control | All editing pages write change records; Operations Dashboard | Applied; approver and requirement fields open (P03, P04) |
| 3.4 Governance | Conformance Review; PEO Outcome Review; Demand Gateway; Operations Dashboard | Applied |
| 3.5 Single Source of Truth | SSOT pages; Demand Gateway | Applied |
| 3.6 Target Operating Model | Operations Dashboard; Conformance Review (learning loop) | Applied; measured agent assurance proposed |
| 3.7 Term Definition Standard | Model Editor (Vocabulary); Conformance Review; builders | Applied |
| 3.8 Labels and Personas | Conformance Review; Customer 360 pages | Applied; SKOS-XL persona labels by review |
| 3.9 Attributes, Properties and Relations | Model Editor (Taxonomy, Ontology); Relation Taxonomy | Applied |
| 3.10 BFO and RDF Foundations | Ingestion › Reference ontologies; build and lineage checks | Applied |
| 3.11 Semantic Asset Metadata | Ingestion; Model Manager; Standards Conformance | Applied; conformsTo moves to 2.0 on rebuild |
| 3.12 Services, Processes, and the Reference Registry | Service Assembly; APQC PCF 7.4; PEO Service Map | Applied |
| 3.13 Personas, Jobs, and Demand | Customer 360 pages; PEO Service Map | Applied |

## v1.9 Divergences Resolved

The Operations Dashboard listed 21 places where v1.9 no longer matched practice. Each is resolved here as follows; items marked "decision pending" are written as proposals and stay open as actions until decided.

| Item | Topic | Resolved in | Status |
|---|---|---|---|
| G01 | Identifiers of classes and taxonomy concepts | 3.2 Naming Conventions; examples throughout | Resolved |
| G02 | Identifiers of individuals and instance | 3.2 Instance Identifiers | Proposed, decision |

| Item | Topic | Resolved in | Status |
|---|---|---|---|
| | graphs | | pending |
| G03 | URI patterns for core, areas, and nested domains | 3.2 URI Conventions | Resolved |
| G04 | Named graph pattern | 3.2 Named Graphs; 3.11 | Resolved |
| G05 | Asset titles | 3.2 Asset Titles | Resolved |
| G06 | Operational module versus operational areas | 3.1; 1.3; 3.3 Human in the Loop; Appendix D | Resolved |
| G07 | Primitives and polymorphic terms | 3.7 Primitives and Polymorphic Terms | Resolved |
| G08 | Reference ontology stack | 3.10 Reference Ontology Stack and Load Order | Resolved |
| G09 | Change record model | 3.3 Change Record Profile | Resolved; tool fields open (P03, P04) |
| G10 | Deployment records and the release pipeline | 3.3 Current Practice | Resolved; deployment records open (P05) |
| G11 | Versioning before 1.0.0 | 3.3 Versioning | Resolved |
| G12 | Lifecycle promotion criteria | 3.3 Candidate Entry Criteria | Proposed, decision pending |
| G13 | Human review and learning loop | 3.4 Implemented Quality Gates; 3.6 Recursive Learning | Resolved |
| G14 | Personas and jobs to be done | 3.8 The Persona Model; 3.13 | Resolved |
| G15 | Service assembly model and process registry | 3.12 | Resolved |
| G16 | Operating model: areas, domains, and the service catalog | 1.3; 3.1 | Resolved |
| G17 | Tooling: RDF Studio steps | Appendix E | Resolved |
| G18 | Worked example | Appendix A identifiers and URIs | Resolved (re-keyed; content unchanged) |
| G19 | Glossary | Appendix F | Resolved |
| G20 | Measured agent assurance | 2.7 Decision Evaluation; 3.6 Measured Agent Assurance | Resolved in v2.1 (AI circuit breaker gateway) |
| G21 | Edition and conformance claims | Front matter; 3.11 Edition Conformance | Resolved on publication; rebuild assets |

APPENDIX F

# Terms & Acronyms

Definitions of the acronyms and specialist terms used throughout this document.

## F.1 Terms & Acronyms

APPENDIX F • SECTION F.1

# Terms & Acronyms

The table below defines the acronyms and specialist terms used in this document.

| Term | Meaning | Description |
|---|---|---|
| A2A | Agent-to-Agent protocol | Open protocol for discovery, delegation, and secure messaging between autonomous AI agents. |
| Agentic fabric | — | The governed set of AI agents that build, maintain, and use the knowledge fabric; agents propose and people approve (Section 1.5). |
| AI | Artificial Intelligence | Systems that perform tasks requiring human-like perception, reasoning, or language. |
| AI circuit breaker gateway | AI circuit breaker gateway | The control point between every decision and its effect: it intercepts the decision envelope, measures Ds, Db and Dr against the pinned Keel release, and returns PASS, COMPLY, FREEZE or HALT (Section 2.7). |
| Akoma Ntoso | Architecture for Knowledge-Oriented Management of African Normative Texts using Open Standards and Ontologies | OASIS XML standard (LegalDocML) for legislative and legal documents. |
| API | Application Programming Interface | A defined contract through which software components call one another. |
| APQC PCF | APQC Process Classification Framework | Open taxonomy of business processes (category › process group › process › activity › task); the reference registry for internal domain services (Section 3.12). |
| Assembly index | Assembly index (modeled pathway) | Number of joins in a service's closure, shared parts counted once; an upper bound on the true assembly index. |
| BFO | Basic Formal Ontology | Top-level ontology standardized as ISO/IEC 21838-2; the core ontology aligns to it. |
| BI | Business Intelligence | Reporting and analytics on business data. |
| BIBO | Bibliographic Ontology | Community RDF vocabulary whose bibo:status values are written at publish time to express lifecycle status to consumers outside the Company (Section 3.11). |
| Bounded context | Bounded context | Second level (L2) of the HR Services & PEO Enterprise Business Model, within an L1 domain. |
| BPMN | Business Process Model and Notation | OMG standard notation for modeling business processes. |
| BT / NT / RT | Broader Term / Narrower Term / Related Term | ISO 25964 relations between concepts, recorded as skos:broader, skos:narrower, and skos:related. |
| C360 | Customer 360 | The personas, jobs, desired outcomes, reviews, and demand of Section 3.13, and the workbench pages that maintain them. |
| CFO | Chief Financial Officer | Executive responsible for the finance function. |
| Change record | Change record | PROV-O activity in the change log describing one change set: what changed, why, who made and |

| Term | Meaning | Description |
|---|---|---|
| | | approved it, what it used and generated. |
| Claim check | — | Pattern in which large content is stored separately and the event carries only its location and checksum. |
| CloudEvents | — | Cloud Native Computing Foundation specification for a common event envelope. |
| Container | Area container | The vocabulary, taxonomy, and ontology of an operational area, holding what two or more of its domains share. |
| Copy number | Copy number | Number of services that contain a part; shared parts with high copy number are platform and automation candidates. |
| CQ | Competency Question | A question the knowledge must be able to answer, written as a test that runs in each release (Appendix A.4, Appendix D). |
| CRM | Customer Relationship Management | Systems that manage customer data and interactions. |
| CRUD | Create, Read, Update, Delete | The four basic operations on stored data. |
| DB | Database | An organized store of data; a vector DB stores embeddings. |
| DCAT | Data Catalog Vocabulary | W3C RDF vocabulary for describing datasets and data catalogs. |
| DCMI | Dublin Core Metadata Initiative | Maintainer of the Dublin Core metadata terms standardized as ISO 15836; the dcterms: properties used for titles, descriptions, creators, dates, and subjects. |
| DDL | Data Definition Language | SQL statements that define database structures, such as CREATE TABLE. |
| Decision envelope | Decision envelope | What an agent or a person emits for a decision: output, persona, evidence, model or reviewer, and the pinned release; evaluated by the AI circuit breaker gateway (Section 2.7). |
| Dead-letter topic | — | Kafka topic holding events that could not be processed after all allowed recovery actions. |
| Decision event | Decision event | Event on knowledge.decision topics carrying an evaluation request or a gateway verdict (Section 2.7). |
| Desired outcome | Desired outcome | Measurable statement of how a persona judges success at a job step: direction, metric, object, and clarifier. |
| Disposition | Disposition | Recorded decision on a conformance finding: fixed, accepted as is, not an issue, deferred, or needs a modeling decision. |
| DMN | Decision Model and Notation | OMG standard for decision tables; the authoring format for business rules in the SSOT. |
| DOL | Distributed Ontology, Model and Specification Language | OMG standard for integrating and mapping heterogeneous ontologies and models. |
| Domain holon | Domain holon | A domain packaged as a self-contained whole and a |

| Term | Meaning | Description |
|---|---|---|
| | | part of the enterprise: its ontology, mappings, vector partition and agents, insulated from other domains (Section 2.7). |
| Domain service | Domain service | Third level (L3) of the business model: a service a domain delivers, enabled by business processes. |
| DPROD | Data Product Ontology | OMG specification for describing data products, their metadata, and lineage. |
| DS | Domain-Specific | Applies to a single business domain, as in DS ontologies or DS agents. |
| Structural / behavioral / relevancy distance | Ds / Db / Dr | The three distances the AI circuit breaker gateway measures: facts that disagree with the ontology, output against intent, and persona-weighted relevancy across five dimensions. |
| Enterprise service catalog | Enterprise service catalog | Catalog above the operational areas where each area publishes its services; the only place content from different areas is combined. |
| ERP | Enterprise Resource Planning | Integrated systems for finance, supply chain, and operations. |
| ETL | Extract, Transform, Load | Process for moving and reshaping data between systems. |
| FAIR | Findable, Accessible, Interoperable, Reusable | Principles for making data and metadata reusable. |
| Feedback event | Feedback event | Event on knowledge.feedback topics carrying a signal from agents, services, search or users; an input to the learning services, never a write to knowledge (Section 2.7). |
| FIBO | Financial Industry Business Ontology | EDM Council's open ontology of financial concepts. |
| FP&A | Financial Planning and Analysis | Finance function for budgeting, forecasting, and performance analysis. |
| FTE | Full-Time Equivalent | Unit of workforce capacity. |
| GCP | Google Cloud Platform | Google's cloud computing services. |
| GL | General Ledger | The master record of an organization's financial transactions. |
| GQL | Graph Query Language | ISO/IEC 39075 query language for property graphs. |
| HR 360 | HR 360 agent | The AI agent that answers client HR questions in natural language; the first consumer of the semantic services in Appendix D. |
| IAO | Information Artifact Ontology | BFO-based ontology for information entities such as documents, records, and identifiers; source of the is-about relation. |
| IEEE | Institute of Electrical and Electronics Engineers | Standards body; with ISO and IEC it publishes joint ISO/IEC/IEEE standards. |
| IFRS | International Financial Reporting Standards | Global accounting standards issued by the IASB. |

| Term | Meaning | Description |
|---|---|---|
| IRI / URI | Internationalized / Uniform Resource Identifier | Global identifiers for resources; every ontology term has one. |
| ISO | International Organization for Standardization | International standards body (ISO/IEC and ISO/IEC/IEEE denote joint standards). |
| Job map stage | Job map stage | One of the eight universal stages of a job map; distinct from the steps of one job. |
| JSON / JSON-LD | JavaScript Object Notation / JSON for Linked Data | Lightweight data format; JSON-LD encodes RDF in JSON. |
| JTBD | Jobs to be done | Framing in which customers hire products and services to get jobs done; the basis of the jobs model. |
| Kafka | Apache Kafka | Distributed event streaming platform used as the inbound event backbone (Section 2.6). |
| KE | Knowledge Engineering | The discipline and team that build and manage ontologies, vocabularies, taxonomies, and knowledge graphs; owner of the semantic services in Appendix D. |
| KE Workbench | Knowledge Engineering Workbench | The Company's implementation of the Semantic Modeling Workbench and Operations Workbench (Section 1.3, Appendix E). |
| Keel | Keel | The governed, deterministic semantic foundation (Section 2.1); the released knowledge layer the AI circuit breaker gateway measures against. |
| KG | Knowledge Graph | A graph of entities and relationships, governed by ontologies. |
| KGCL | Knowledge Graph Change Language | Standard language for describing and applying changes to ontologies (see Section A.5). |
| Knowledge component | — | A domain packaged with its ontology, vocabulary, rules, shapes, mappings, agents, and published interface, released as one versioned unit (Section 1.5). |
| Knowledge fabric | — | The governed, federated body of ontologies, vocabularies, taxonomies, rules, and mappings connected through the core (Section 1.5). |
| KPI | Key Performance Indicator | A measurable value that tracks performance against goals. |
| LegalRuleML | Legal Rule Markup Language | OASIS standard for representing legal norms and rules. |
| LinkML | Linked Data Modeling Language | Schema language used, among other things, to define KGCL's data model. |
| LKIF | Legal Knowledge Interchange Format | OWL-based format and ontology for legal knowledge (see Section A.7). |
| LLM | Large Language Model | A neural model trained on large text corpora to understand and generate language. |
| Load gate | Load gate | Rule that stops loading into GraphDB while a file has a blocking conformance finding. |
| LOM | Learning Object Metadata | IEEE 1484.12.1 metadata standard; the source of the version, status, and contribution elements in Section 3.11. |

| Term | Meaning | Description |
|---|---|---|
| MBSE | Model-Based Systems Engineering | Engineering approach that uses formal models as the primary artifacts. |
| MCP | Model Context Protocol | Open protocol that exposes tools, data, and workflows to AI agents through standard interfaces. |
| MTBH | Mean Time Between Hallucinations | Answers served (or hours in service) between confirmed hallucinations; drives the AI Circuit Breaker (Appendix D, Section 6). |
| Multiword lexical unit | Multiword lexical unit | Primitive of several words whose meaning is not the sum of its words, such as circuit breaker. |
| MVP1 | Minimum Viable Product, release 1 | The first deliverable increment of the semantic governance platform. |
| Named graph | — | A set of triples identified by an IRI within an RDF dataset; each knowledge artifact's current release is held in its own named graph (Section 3.11). |
| NIST AI RMF | NIST AI Risk Management Framework | U.S. framework for managing risks of AI systems. |
| NLQ | Natural Language Query | A question posed in everyday language rather than a query language. |
| NLU | Natural Language Understanding | Interpreting intent and entities from text. |
| OAK | Ontology Access Kit | Python library and command-line tool for working with ontologies, including applying KGCL changes. |
| OASIS | Organization for the Advancement of Structured Information Standards | Open standards consortium (LegalRuleML, Akoma Ntoso). |
| OBO | Open Biological and Biomedical Ontologies | Community of interoperable open ontologies where KGCL originated. |
| ODM | Ontology Definition Metamodel | OMG standard metamodel for ontologies (OWL, RDF) in model-driven architecture. |
| OMG | Object Management Group | Standards consortium behind ODM, DOL, BPMN, UML, and DPROD. |
| OpenTelemetry | — | Open standard for traces, metrics, and logs used to observe services and agents. |
| Operational area | Operational area | One of four groupings of domains: Business, Commercial, HR Service, and Technical Operations. |
| ORSD | Ontology Requirements Specification Document | NeOn Methodology template for ontology requirements (see Section A.4). |
| OWL | Web Ontology Language | W3C language for defining ontologies with formal semantics (OWL 2 DL is a decidable profile). |
| PCF | Process Classification Framework | See APQC PCF. |
| Persona | — | A governed description of a group of users who share a role, vocabulary, and information needs (Section 3.8). Not a security role. |
| PGVector | PostgreSQL vector extension | Adds vector storage and similarity search to PostgreSQL (used in Appendix A.9). |
| Polyhierarchy | — | A concept with more than one broader concept; limited |

| Term | Meaning | Description |
|---|---|---|
| | | to two in Company taxonomies (Section 3.9). |
| Polymorphic term | Polymorphic term | Concept or class built from more than one primitive; written as a phrase, never as a new word. |
| Polymorphism | — | A taxonomy concept that draws meaning from several vocabulary terms or parents (Section 3.11). |
| Primitive | Primitive | Unique, irreducible vocabulary term from which concepts and classes are built; owned by one domain or the core. |
| PROV-O | PROV Ontology | W3C vocabulary for recording provenance: who changed what, when, and from which sources. |
| PT / NPT | Preferred Term / Non-Preferred Term | ISO 25964 names for a concept's preferred label and its alternate labels (skos:prefLabel, skos:altLabel). |
| R2RML | RDB to RDF Mapping Language | W3C language for exposing relational databases as virtual RDF graphs. |
| RA | Registration Authority | Organization accountable for registering and maintaining an artifact (ISO/IEC 11179-6), recorded as dcterms:publisher. |
| RAG | Retrieval-Augmented Generation | Grounding LLM answers in retrieved, governed content; RAG 2.0 is the federated design in Section 2.1. |
| RDF / RDFS | Resource Description Framework / RDF Schema | W3C graph data model of subject-predicate-object triples, and its basic schema vocabulary. |
| RDF 1.2 / RDF-star | Resource Description Framework 1.2 | W3C update to RDF that adds triple terms, so properties can be attached to individual statements. |
| RDF Studio | — | Visual modeling tool used to design ontologies, taxonomies, and R2RML mappings (Appendix E). |
| Release event | Release event | Event on knowledge.release topics announcing a published, withdrawn or rolled-back release; the downstream contract with consumers (Section 2.7). |
| Release manifest | Release manifest | Graph listing the component graphs of an assembly release and the asset versions they require. |
| RML | RDF Mapping Language | Declarative language for mapping structured sources (CSV, JSON, XML, databases) to RDF. |
| RO | OBO Relations Ontology | Library of BFO-aligned relations such as has input, has output, and has active participant. |
| SemVer | Semantic Versioning | MAJOR.MINOR.PATCH version numbering used for all knowledge artifacts. |
| Service assembly | Service assembly | Services built from reusable process and step components, each in its own named graph (Section 3.12). |
| SG | Semantic Governance | The discipline and operating model described in this document. |
| SHACL | Shapes Constraint Language | W3C language for validating RDF data against rules (shapes); used here as a policy gate. |
| SIPOCM | Supplier, Input, Process, Output, Customer, Measure | The description every process step carries, in terms of the core ontology. |

| Term | Meaning | Description |
|---|---|---|
| SKL | Semantic Knowledge Lifecycle; Semantic Knowledge Layer | Six-stage lifecycle for semantic assets: ideation, development, testing, deployment, maintenance, retirement. In Section 2.6 and Appendix D, SKL also names the Semantic Knowledge Layer, the governed graph that holds ontologies, vocabularies, rules, and asset descriptions. |
| SKOS | Simple Knowledge Organization System | W3C vocabulary for taxonomies, thesauri, and classification schemes. |
| SKOS-XL | SKOS eXtension for Labels | W3C extension that makes labels resources, so a label can be linked to a persona, source, or date. |
| SLM | Small (Specialized) Language Model | A smaller, domain-tuned model. In this document, "virtual SLM" means a domain vector partition used with one LLM (Section 2.1). |
| SLO / SLA | Service Level Objective / Agreement | Target and committed levels of service, monitored in the target operating model. |
| SME | Subject-Matter Expert | A business user with deep domain knowledge. |
| SN | Scope Note | Note on what a term includes and excludes, recorded as skos:scopeNote (ISO 25964). |
| SOW | Statement of Work | Contractual description of scope, deliverables, and timeline (Appendix D). |
| SOX | Sarbanes-Oxley Act | U.S. law governing financial reporting controls (Section 404 covers internal controls). |
| SPARQL | SPARQL Protocol and RDF Query Language | W3C query language for RDF graphs. |
| SSOT | Single Source of Truth | One governed, versioned definition of each rule, constraint, logic component, and calculation (Section 3.5). |
| Star topology | — | Federation pattern in which every domain connects to the core and never directly to another domain. |
| Consumer subscription | Subscription | A consumer’s registration for release and asset events, delivered by notify-consumers (Section 2.7). |
| SWRL | Semantic Web Rule Language | Rule language that extends OWL with if-then rules. |
| TBX | TermBase eXchange | ISO 30042 format and data categories for terminology; the source of term types such as acronym and abbreviation. |
| TLO | Top-Level Ontology | Domain-neutral ontology that domain ontologies align to (ISO/IEC 21838-1); the Company uses BFO. |
| TOM | Target Operating Model | How knowledge engineering is run, monitored, and improved (Section 3.6). |
| TriG | TriG RDF syntax | Text format for RDF datasets with named graphs (.trig files). |
| TT | Top Term | ISO 25964 name for a concept with no broader concept, recorded as skos:topConceptOf. |
| UI / UX | User Interface / User Experience | The screens users interact with, and the quality of that interaction. |

| Term | Meaning | Description |
|---|---|---|
| UML | Unified Modeling Language | OMG standard for software and system models. |
| VAEM | Vocabulary for Attaching Essential Metadata | QUDT vocabulary that supplies vaem:namespace, the namespace of a vocabulary, taxonomy, or ontology. |
| Value gap | Value gap | Difference between a desired outcome's target and the delivered outcome's measured baseline on the same job step. |
| Value map | Value map | Separate assembly that bridges process steps to job steps and holds delivered outcomes. |
| VANN | Vocabulary for Annotating Vocabulary Descriptions | Vocabulary that supplies vann:preferredNamespacePrefix. |
| W3C | World Wide Web Consortium | Standards body for RDF, OWL, SHACL, SPARQL, SKOS, and DCAT. |